LAPORAN TUGAS AKHIR

SISTEM PAKAR UNTUK DIAGNOSIS PENYAKIT GIGI DAN MULUT PADA ANAK DENGAN METODE *FORWARD CHAINING* BERBASIS *WEBSITE*

SKRIPSI

Diajukan Sebagai Pedoman Pelaksanaan Penilaian Tugas Akhir
pada Jurusan Informatika Fakultas Teknik Universitas Jenderal Soedirman

Oleh:
BRIAN CAHYA PURNAMA
H1D022009

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS TEKNIK
JURUSAN INFORMATIKA
PURWOKERTO

2025

LEMBAR PENGESAHAN

Tugas Akhir dengan Judul:

SISTEM PAKAR UNTUK DIAGNOSIS PENYAKIT GIGI DAN MULUT PADA ANAK DENGAN METODE *FORWARD CHAINING* BERBASIS *WEBSITE*

Oleh:

BRIAN CAHYA PURNAMA

H1D022009

Diajukan untuk memenuhi salah satu persyaratan memperoleh gelar Sarjana Komputer pada Jurusan Informatika Fakultas Teknik Universitas Jenderal Soedirman

Diterima dan disetujui

Pada tanggal : 16 Desember 2025

Dosen Pembimbing I	Dosen Pembimbing II
Ir. Ipung Permadi, S.Si., M.Cs.	**Mochammad Agri Triansyah, S.Kom., M.Kom.**
NIP. 198311162008121005	NIP. 199408122023211023

Mengetahui,

Dekan Fakultas Teknik

Prof. Dr. Eng. Ir. Agus Martoyo, S.T., M.T., IPU., ASEAN Eng.

NIP. 197109202006041001

LEMBAR PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan di bawah ini:

Nama : Brian Cahya Purnama

NIM : H1D022009

Program Studi : Informatika

Dengan ini, saya menyatakan bahwa dalam skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu perguruan tinggi dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka. Demikian pernyataan ini yang saya buat dengan sebenar-benarnya tanpa ada unsur paksaan dari pihak manapun.

Purwokerto, 16 Desember 2025



Brian Cahya Purnama

HALAMAN PERSEMBAHAN

Tugas akhir ini penulis persembahkan kepada orang-orang yang selalu mendampingi penulis selama masa perkuliahan hingga penyusunan tugas akhir, antara lain:

1. Keluarga penulis.
2. Sahabat-sahabat penulis.
3. Teman-teman Informatika angkatan 2022 Universitas Jenderal Soedirman.
4. Serta teman-teman penulis yang telah memberikan dukungan dan doa dalam pengerjaan tugas akhir ini.

HALAMAN MOTTO

"Mastery comes through consistent practice."

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan tugas akhir penelitian dengan judul "Sistem Pakar untuk Diagnosis Penyakit Gigi dan Mulut pada Anak dengan Metode *Forward Chaining* Berbasis *Website*." Laporan tugas akhir ini disusun sebagai salah satu syarat untuk menempuh Tugas Akhir pada Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.

Dalam penyusunan laporan tugas akhir ini, penulis mendapatkan berbagai dukungan, arahan, serta masukan dari banyak pihak. Oleh karena itu, dengan penuh hormat dan rasa syukur, penulis menyampaikan terima kasih kepada:

1. Bapak Prof. Dr. Eng. Ir. Agus Maryoto, ST., M.T., IPU., ASEAN Eng selaku Dekan Fakultas Teknik, Universitas Jenderal Soedirman.
2. Bapak Dr. Ir. Lasmedi Afuan, S.T., M.Cs., IPM., selaku Ketua Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.
3. Bapak Bangun Wijayanto, S. T., M.Cs., selaku Dosen Pembimbing Akademik yang telah memberikan bimbingan, masukan, serta arahan selama masa perkuliahan.
4. Bapak Ir. Ipung Permadi, S.Si., M.Cs., selaku Dosen Pembimbing I dan Bapak Mochammad Agri Triansyah, S.Kom., M.Kom., selaku Dosen Pembimbing II yang dengan penuh perhatian telah memberikan bimbingan dan arahan selama penyusunan laporan tugas akhir ini.
5. Orang tua dan keluarga tercinta atas doa, semangat, serta dukungan yang tiada harganya.

6. Klinik Victory Purwokerto sebagai mitra penelitian yang telah memberikan izin, dukungan, serta fasilitas dalam pelaksanaan pengumpulan data dan proses validasi sistem.
7. drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani, selaku pakar dari Klinik Victory Purwokerto yang telah meluangkan waktu dan keahliannya untuk memberikan masukan serta validasi dalam penelitian ini.
8. Sahabat-sahabat penulis yang telah memberikan dukungan penuh selama proses penyusunan tugas akhir ini.
9. Teman-teman seperjuangan di Jurusan Informatika angkatan 2022, serta seluruh pihak yang telah memberikan dukungan dan bantuan, baik secara langsung maupun tidak langsung, yang tidak dapat disebutkan satu per satu.

Penulis menyadari laporan tugas akhir ini memiliki keterbatasan, sehingga kritik dan saran sangat diharapkan untuk perbaikan ke depan. Semoga karya ini dapat berkontribusi pada pengembangan ilmu, khususnya dalam sistem pakar berbasis *website* pada bidang medis.

Purwokerto, 16 Desember 2025



Brian Cahya Purnama

DAFTAR ISI

LEMBAR PENGESAHAN i
LEMBAR PERNYATAAN KEASLIAN SKRIPSI ii
HALAMAN PERSEMBAHAN iii
HALAMAN MOTTO iv
KATA PENGANTAR v
DAFTAR ISI vii
DAFTAR GAMBAR ix
DAFTAR TABEL xi
ABSTRAK xii
ABSTRACT xiii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 5
1.3 Batasan Masalah 6
1.4 Tujuan Penelitian 7
1.5 Manfaat Penelitian 7
BAB II TINJAUAN PUSTAKA 9
2.1 Sistem Pakar 9
2.2 Metode *Forward Chaining* 11
2.3 Konsep *Website* 12
2.4 PHP 12
2.5 *Framework* Laravel 13
2.6 Perangkat Lunak 13
2.7 Metode *Waterfall* 15
2.8 Pemodelan Sistem 17
2.9 Penyakit Gigi dan Mulut pada Anak 20
2.10 *Black-Box Testing* 27
2.11 Penelitian Sejenis 28
BAB III METODOLOGI PENELITIAN 30
3.1 Waktu dan Tempat Penelitian 30
3.2 Alat Penelitian 30

3.3 Data Penelitian 31
3.4 Tahapan Penelitian 32
BAB IV HASIL DAN PEMBAHASAN 38
4.1 *Requirements Analysis* 38
4.1.1 Analisis Pengetahuan 38
4.1.2 Analisis Kebutuhan Pengguna 65
4.2 *Design System* 67
4.2.1 Perancangan Dokumentasi Sistem 67
4.2.2 Perancangan *Mockup* Sistem 92
4.3 *Implementation* 109
4.4 *Testing* 169
4.5 *Maintenance* 182
BAB V PENUTUP 184
5.1 Kesimpulan 184
5.2 Saran 185
DAFTAR PUSTAKA 186
LAMPIRAN 191
Lampiran 1 Lembar Verifikasi Aturan Pakar Pertama 191
Lampiran 2 Lembar Verifikasi Aturan Pakar Kedua 192
Lampiran 3 Berita Acara Pengujian Pakar Pertama 193
Lampiran 4 Berita Acara Pengujian Pakar Kedua 194

DAFTAR GAMBAR

Gambar 1. Lingkungan Sistem Pakar 10
Gambar 2. Metode *Waterfall* 15
Gambar 3. Tahapan Penelitian 32
Gambar 4. Pembangunan Sistem Metode *Waterfall* 33
Gambar 5. Alur *Forward Chaining* 35
Gambar 6. Karies Gigi 41
Gambar 7. Pulpitis 42
Gambar 8. Abses Gigi 44
Gambar 9. Gingivitis 45
Gambar 10. Persistensi Gigi Sulung 47
Gambar 11. Trauma Gigi 48
Gambar 12. Fluorosis Dental 49
Gambar 13. Hipoplasia Email 50
Gambar 14. Stomatitis Aphthosa 52
Gambar 15. Maloklusi 53
Gambar 16. *Use Case Diagram* Sistem Pakar Gigi dan Mulut Anak 68
Gambar 17. *Activity Diagram Login as User* 69
Gambar 18. *Activity Diagram* Melakukan Diagnosis 70
Gambar 19. *Activity Diagram* Mencetak Hasil Diagnosis 72
Gambar 20. *Activity Diagram* Melihat Riwayat Diagnosis 73
Gambar 21. *Activity Diagram Login to Dashboard* 74
Gambar 22. *Activity Diagram* Mengelola Data Pengguna 76
Gambar 23. *Activity Diagram* Mengelola Data *Approval* Pakar 78
Gambar 24. *Activity Diagram Register as Pakar* 80
Gambar 25. *Activity Diagram* Mengelola Data Penyakit 82
Gambar 26. *Activity Diagram* Mengelola Data Gejala 85
Gambar 27. *Activity Diagram* Mengelola Data Aturan 87
Gambar 28. *Activity Diagram* Mengelola Data Riwayat Diagnosis 89
Gambar 29. *Activity Diagram Logout* 90
Gambar 30. *Class Diagram* Sistem Pakar Gigi dan Mulut Anak 91
Gambar 31. *Mockup* Halaman Daftar 92
Gambar 32. *Mockup* Halaman Masuk 93
Gambar 33. *Mockup* Halaman Beranda 94
Gambar 34. *Mockup* Halaman Diagnosis 95
Gambar 35. *Mockup* Halaman Hasil Diagnosis 96
Gambar 36. *Mockup* Halaman Riwayat Diagnosis 97
Gambar 37. *Mockup* Halaman Info Penyakit 98
Gambar 38. *Mockup* Halaman Detail Info Penyakit 99
Gambar 39. *Mockup Dashboard* Admin 100
Gambar 40. *Mockup* Halaman Tentang 101
Gambar 41. *Mockup* Halaman Kelola Pengguna 102
Gambar 42. *Mockup* Halaman Kelola *Approval* Pakar 103

Gambar 43. *Mockup Dashboard* Pakar 104
Gambar 44. *Mockup* Halaman Kelola Penyakit 105
Gambar 45. *Mockup* Halaman Kelola Gejala 106
Gambar 46. *Mockup* Halaman Kelola Aturan 107
Gambar 47. *Mockup* Halaman Kelola Riwayat Diagnosis 108
Gambar 48. Halaman Beranda 110
Gambar 49. Halaman Daftar 111
Gambar 50. Halaman Masuk 114
Gambar 51. Halaman Diagnosis 116
Gambar 52. Halaman Diagnosis Tidak Ditemukan 117
Gambar 53. Halaman Diagnosis Ditemukan 118
Gambar 54. Halaman Hasil Diagnosis 123
Gambar 55. Halaman Riwayat Diagnosis 125
Gambar 56. Cetak PDF 126
Gambar 57. Halaman Tentang 129
Gambar 58. Halaman Info Penyakit 130
Gambar 59. Halaman Detail Info Penyakit 132
Gambar 60. *Dashboard* Admin 133
Gambar 61. Halaman Daftar Data Pengguna 136
Gambar 62. Halaman Tambah Data Pengguna 138
Gambar 63. Halaman Edit Data Pengguna 140
Gambar 64. Halaman Daftar Data *Approval* Pakar 142
Gambar 65. Setujui Pakar 143
Gambar 66. Tolak Pakar 144
Gambar 67. *Dashboard* Pakar 145
Gambar 68. Halaman Daftar Data Penyakit 148
Gambar 69. Halaman Tambah Data Penyakit 150
Gambar 70. Halaman Edit Data Penyakit 152
Gambar 71. Halaman Daftar Data Gejala 154
Gambar 72. Halaman Tambah Data Gejala 156
Gambar 73. Halaman Edit Data Gejala 157
Gambar 74. Halaman Daftar Data Aturan 159
Gambar 75. Mode Edit Data Aturan 160
Gambar 76. Halaman Tambah Data Aturan 163
Gambar 77. Halaman Daftar Data Riwayat Diagnosis 165
Gambar 78. Detail Data Riwayat Diagnosis 166

DAFTAR TABEL

Tabel 1. Komponen *Use Case Diagram* 18
Tabel 2. Komponen *Activity Diagram* 19
Tabel 3. Komponen *Class Diagram* 20
Tabel 4. Penelitian Sejenis 28
Tabel 5. Gejala Penyakit Gigi dan Mulut pada Anak 39
Tabel 6. Jenis Penyakit Gigi dan Mulut pada Anak 40
Tabel 7. Tabel Keputusan Penyakit Gigi dan Mulut pada Anak 55
Tabel 8. Ilustrasi Proses *Pruning* 65
Tabel 9. Analisis Kebutuhan Pengguna 66
Tabel 10. Pengujian Fitur Administrator 170
Tabel 11. Pengujian Fitur Pakar 172
Tabel 12. Pengujian Fitur Klien 179

ABSTRAK

SISTEM PAKAR UNTUK DIAGNOSIS PENYAKIT GIGI DAN MULUT PADA ANAK DENGAN METODE *FORWARD CHAINING* BERBASIS *WEBSITE*

Brian Cahya Purnama

H1D022009

Penyakit gigi dan mulut merupakan masalah kesehatan yang sering terjadi pada anak dengan prevalensi tinggi di Indonesia, namun deteksi dini masih terhambat oleh keterbatasan pengetahuan orang tua dan akses terhadap tenaga medis gigi. Penelitian ini mengembangkan sistem pakar berbasis *website* untuk diagnosis awal penyakit gigi dan mulut pada anak menggunakan metode *forward chaining*. Sistem dikembangkan dengan *framework* Laravel dan MySQL, melibatkan basis pengetahuan dari pakar Klinik Victory Purwokerto. Sistem mengidentifikasi 10 jenis penyakit berdasarkan 38 gejala yang direpresentasikan dalam aturan *IF-THEN* dengan struktur logika *AND-OR*. Mesin inferensi menerapkan lima teknik *pruning* untuk meningkatkan efisiensi diagnosis, mengurangi rata-rata pertanyaan menjadi 5-10 dari maksimal 38 gejala. Sistem melayani tiga kategori pengguna: administrator, pakar, dan klien dengan fungsi berbeda sesuai peran masing-masing. Pengujian *black-box testing* terhadap seluruh fitur menunjukkan semua skenario menghasilkan *output* sesuai harapan, memvalidasi sistem memenuhi kebutuhan fungsional yang ditetapkan. Sistem ini berfungsi sebagai alat bantu diagnosis awal untuk deteksi dini penyakit gigi dan mulut pada anak, meskipun tidak menggantikan pemeriksaan klinis langsung oleh dokter gigi profesional.

Kata Kunci: Diagnosis, *Forward Chaining*, Laravel, Penyakit Gigi Mulut Anak, *Pruning*, Sistem Pakar, *Website*

ABSTRACT

EXPERT SYSTEM FOR DIAGNOSING DENTAL AND ORAL DISEASES IN CHILDREN USING THE FORWARD CHAINING METHOD WEBSITE-BASED

Brian Cahya Purnama

H1D022009

Dental and oral diseases are common health problems among children with a high prevalence in Indonesia, but early detection is still hampered by limited parental knowledge and access to dental professionals. This study developed a website-based expert system for the initial diagnosis of dental and oral diseases in children using the forward chaining method. The system was developed with the Laravel framework and MySQL, involving a knowledge base from experts at the Victory Purwokerto Clinic. The system identifies 10 types of diseases based on 38 symptoms represented in IF-THEN rules with AND-OR logic structures. The inference engine applies five pruning techniques to improve diagnosis efficiency, reducing the average number of questions to 5-10 from a maximum of 38 symptoms. The system serves three categories of users: administrators, experts, and clients with different functions according to their respective roles. Black-box testing of all features showed that all scenarios produced expected outputs, validating that the system meets the defined functional requirements. This system serves as an initial diagnostic tool for early detection of dental and oral diseases in children, although it does not replace direct clinical examination by a professional dentist.

Keywords: *Diagnosis, Expert System, Forward Chaining, Laravel, Pediatric Dental Diseases, Pruning, Website*

BAB I
PENDAHULUAN

1.1 Latar Belakang

Kesehatan gigi dan mulut merupakan bagian integral dari kesehatan tubuh secara keseluruhan yang harus diperhatikan sejak dini, khususnya pada anak-anak yang berada dalam fase pertumbuhan dan perkembangan. Data dari Riset Kesehatan Dasar (Riskesdas) menunjukkan bahwa hampir 89% penduduk Indonesia mengalami masalah dengan gigi yang berlubang (*decay*), gigi yang dicabut karena karies atau sisa akar (*missing*), dan gigi yang ditumpat (*filling*). Selain itu, sebanyak 92,6% dari penduduk Indonesia berusia 5-9 tahun juga mengalami masalah DMF (*decay, missing,* dan *filling*), sementara itu angka tersebut sebesar 73,4% untuk penduduk Indonesia yang berusia 10-14 tahun (Wibowo et al., 2023). Fakta ini menunjukkan bahwa penyakit gigi pada anak masih menjadi masalah kesehatan masyarakat yang cukup tinggi dan perlu mendapat perhatian lebih. Tingginya kasus penyakit gigi pada anak tidak hanya berdampak pada kesehatan fisik, tetapi juga dapat memengaruhi kualitas hidup, proses belajar, dan kepercayaan diri anak.

Tingginya prevalensi penyakit gigi pada anak tidak lepas dari faktor perilaku, seperti kebiasaan mengonsumsi makanan manis dan lengket (kariogenik) yang kaya akan karbohidrat sederhana (Mulyati et al., 2022). Konsumsi makanan kariogenik yang berkelanjutan tanpa diiringi perhatian higienitas yang memadai secara progresif akan mengikis email gigi (Safira, J. R. D., & Larasati, 2022). Kebiasaan makan makanan manis ini, yang sering menjadi jajanan favorit anak-anak, sangat berpengaruh terhadap terbentuknya

gigi berlubang (Bukhari et al., 2025). Selain itu, banyak orang tua yang belum memahami cara merawat gigi anak sejak dini, sehingga masalah gigi anak sering baru diketahui ketika sudah parah dan menyebabkan sakit.

Salah satu tantangan utama dalam penanganan kesehatan gigi anak adalah keterbatasan akses terhadap diagnosis dini yang akurat. Di Indonesia, distribusi tenaga medis gigi yang tidak merata menyebabkan banyak kasus penyakit gigi anak terlambat terdeteksi. Keterbatasan pengetahuan orang tua dalam mengenali gejala awal berbagai jenis penyakit gigi anak, seperti karies, gingivitis, trauma, atau fluorosis, turut memperparah kondisi ini. Kondisi serupa juga ditemukan di Klinik Victory Purwokerto, sebagian besar pasien anak datang dengan kondisi yang sudah memerlukan penanganan intensif. Padahal, deteksi dini dan pengananan sedini mungkin sangat penting untuk mencegah komplikasi dan kerusakan yang semakin parah, khususnya pada anak dalam fase pertumbuhan dan perkembangan gigi (Isnaini, 2023). Untuk mendukung upaya deteksi dini tersebut, diperlukan alat bantu yang dapat memfasilitasi diagnosis awal secara tepat dan mudah diakses, salah satunya melalui penerapan sistem pakar.

Sistem pakar merupakan cabang dari kecerdasan buatan yang dirancang untuk meniru kemampuan penalaran dan pengetahuan seorang pakar dalam menyelesaikan masalah di bidang tertentu secara cerdas dan efektif (Hakim, 2020). Dalam konteks medis, sistem pakar telah banyak diterapkan untuk membantu diagnosis penyakit, memberikan rekomendasi penanganan, serta menjadi alat bantu pengajaran bagi tenaga medis dalam praktik klinis (Manek et al., 2024). Sistem pakar memerlukan basis pengetahuan yang komprehensif

dan metode inferensi yang mampu menghasilkan kesimpulan berdasarkan fakta atau gejala yang dimasukkan pengguna. Penerapan sistem pakar untuk diagnosis penyakit gigi dan mulut pada anak diharapkan dapat memberikan diagnosis awal secara cepat dan akurat, sehingga mendukung upaya deteksi dini dan pencegahan komplikasi lebih lanjut.

Berbagai penelitian sebelumnya telah mengembangkan sistem pakar untuk diagnosis penyakit gigi, namun sebagian besar masih bersifat umum. Salah satunya menggunakan metode *forward chaining* untuk diagnosis penyakit gigi, tetapi cakupan penyakit yang dibahas belum difokuskan pada anak. Padahal, kelompok usia anak memiliki karakteristik dan kebutuhan diagnosis yang berbeda (Ahmad Kurniawan & Febriana Sesunan, 2022). Penelitian lain mengembangkan sistem pakar diagnosis penyakit gigi dengan metode *forward chaining* dan menghasilkan tingkat akurasi sebesar 89% berdasarkan pengujian 39 data uji, namun masih berfokus pada penyakit gigi secara umum dan belum diarahkan pada kasus gigi dan mulut khusus anak (Wa Rumiyanti, Andi Tenri Sumpala, 2023). Oleh karena itu, penelitian ini berfokus pada diagnosis penyakit gigi dan mulut khusus anak dengan melibatkan pengetahuan klinis dari dokter gigi di Klinik Victory Purwokerto sebagai basis aturan sistem. Pemilihan metode *forward chaining* pada penelitian ini didasarkan pada kemampuannya dalam melakukan penalaran yang berangkat dari gejala menuju diagnosis, yang sangat sesuai dengan karakteristik diagnosis pada anak karena gejala klinis menjadi informasi awal yang tersedia dari pengamatan orang tua atau tenaga kesehatan. Metode ini memungkinkan sistem untuk menelusuri berbagai

kemungkinan penyakit secara sistematis berdasarkan fakta yang ada, mirip dengan proses diagnosis yang dilakukan dokter gigi dalam praktik klinis.

Metode *forward chaining* dinilai sangat sesuai untuk aplikasi diagnosis karena proses penalarannya berangkat dari gejala yang nyata dialami pasien menuju kemungkinan penyakit tertentu (Saputri, 2024). Karakteristik ini sejalan dengan kebutuhan diagnosis medis pada anak, karena informasi awal yang tersedia biasanya berupa gejala atau keluhan yang diamati orang tua maupun tenaga kesehatan. Metode ini memungkinkan sistem melakukan penalaran secara bertahap berdasarkan fakta, mirip dengan cara dokter gigi menganalisis gejala sebelum menetapkan diagnosis akhir. Pendekatan yang bersifat *data-driven* ini lebih efektif diterapkan pada kasus diagnosis gigi dan mulut anak yang sering dimulai dari pengumpulan gejala klinis (Faisal et al., 2024). Dibandingkan metode *backward chaining* yang lebih sesuai untuk mencari penjelasan dari hipotesis tertentu, *forward chaining* lebih unggul untuk diagnosis awal karena mampu menelusuri berbagai kemungkinan penyakit berdasarkan gejala yang diamati sejak awal. Hal ini sejalan dengan hasil pengujian menggunakan uji beta terhadap 70 responden yang menunjukkan bahwa skor total kepuasan pengguna terhadap kecepatan sistem dan keakurasian data pada metode *forward chaining* mencapai 59.328, lebih tinggi dibandingkan skor metode *backward chaining* sebesar 55.385 (Kusuma, Abdi Pandu, 2019).

Sistem pakar ini dapat diimplementasikan melalui berbagai jenis aplikasi seperti aplikasi *mobile*, aplikasi *desktop*, maupun aplikasi berbasis *website*. Dalam penelitian ini, pemilihan *platform website* dipilih sebagai basis

pengembangan sistem pakar dengan beberapa pertimbangan penting.. *Website* memiliki aksesibilitas universal, dapat diakses melalui berbagai perangkat (*smartphone, tablet,* komputer) dan dapat digunakan baik oleh tenaga medis maupun orang tua tanpa memerlukan instalasi khusus (Dan & Tunanetra, 2023). *Platform website* juga memungkinkan integrasi dengan sistem informasi klinik yang sudah ada, dapat menjangkau populasi lebih luas, serta memudahkan pembaruan data, sehingga menjadikannya media yang tepat untuk penerapan sistem pakar.

Dengan diterapkannya metode *forward chaining* dalam sistem pakar berbasis *website*, sistem ini diharapkan mampu memberikan diagnosis awal yang lebih cepat dan akurat terhadap penyakit gigi dan mulut pada anak. Sistem ini berpotensi menjadi sarana deteksi dini yang dapat diakses oleh orang tua. Apabila sistem ini berjalan optimal, maka kualitas pelayanan di klinik dapat meningkat, waktu penanganan menjadi lebih efisien, dan risiko keterlambatan penanganan akibat keterbatasan tenaga ahli dapat diminimalkan. Dengan demikian, sistem pakar ini diharapkan dapat membantu mencegah penyakit gigi dan meningkatkan kesehatan gigi dan mulut anak secara keseluruhan.

1.2 Rumusan Masalah

Berdasarkan paparan dalam latar belakang, beberapa permasalahan pokok yang menjadi fokus penelitian ini dapat diidentifikasi sebagai berikut:

a. Bagaimana mengimplementasikan metode *forward chaining* dalam sistem pakar untuk membantu diagnosis awal penyakit gigi dan mulut pada anak di Klinik Victory Purwokerto?

b. Apa saja jenis penyakit gigi dan mulut anak yang dapat dideteksi oleh sistem pakar ini, dan bagaimana aturan (*rule base*) dapat disusun untuk menghasilkan diagnosis yang akurat?

1.3 Batasan Masalah

Agar penelitian ini dapat dilakukan secara lebih terarah, maksimal, dan spesifik, maka ditetapkan beberapa batasan masalah sebagai berikut:

a. Penelitian ini secara khusus memfokuskan pada diagnosis sepuluh jenis penyakit gigi dan mulut yang dijumpai pada anak, di antaranya adalah karies gigi, gingivitis, pulpitis, abses gigi, persistensi gigi sulung, trauma gigi, fluorosis dental, hipoplasia email, stomatitis aphthosa, serta maloklusi.
b. Metode yang digunakan dalam sistem pakar ini adalah *forward chaining*, dengan pendekatan berbasis aturan (*rule-based system*) untuk menelusuri gejala menuju kemungkinan penyakit.
c. Sistem dikembangkan khusus untuk *platform website* (*web-based application*) dan tidak mencakup pengembangan aplikasi *mobile* atau *desktop*.
d. Sistem ini dirancang untuk digunakan oleh tiga jenis pengguna, yaitu administrator, pakar (dokter gigi), dan klien (pasien/orang tua/pengguna umum).
e. Basis pengetahuan (*knowledge base*) dalam sistem ini diperoleh dari literatur ilmiah, literatur medis yang relevan, serta wawancara dengan pakar dokter gigi yaitu drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani dari Klinik Victory Purwokerto, namun tidak mencakup data historis pasien.

f. Sistem menerima *input* berupa gejala-gejala yang dialami pengguna dan menghasilkan *output* berupa hasil diagnosis awal, penjelasan penyakit, serta pencegahan dan perawatan tindak lanjut.

g. Sistem berfungsi sebagai alat bantu diagnosis awal dan tidak menggantikan pemeriksaan klinis langsung oleh dokter gigi profesional.

1.4 Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut:

a. Mengimplementasikan metode *forward chaining* dalam sistem pakar untuk membantu proses diagnosis awal penyakit gigi dan mulut pada anak di Klinik Victory Purwokerto.

b. Mengidentifikasi dan menetapkan jenis-jenis penyakit gigi dan mulut anak yang dapat dideteksi oleh sistem, serta menyusun aturan (*rule base*) sebagai dasar penarikan kesimpulan diagnosis yang akurat.

1.5 Manfaat Penelitian

Melalui penelitian ini, diharapkan dapat diperoleh berbagai manfaat yang dirinci sebagai berikut:

a. Bagi Klinik dan Tenaga Medis

Mempercepat proses diagnosis sekaligus mengurangi beban kerja tenaga medis, karena identifikasi penyakit dilakukan melalui pengolahan gejala secara sistematis dan otomatis. Dengan demikian, waktu yang dibutuhkan untuk menetapkan diagnosis awal dapat diminimalkan dibandingkan apabila dilakukan secara manual.

b. Bagi Peneliti

Menerapkan konsep sistem pakar dalam bidang kecerdasan buatan melalui implementasi metode *forward chaining* ke dalam praktik nyata sekaligus memenuhi syarat kelulusan di Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman. Selain itu, penelitian ini juga mengasah keterampilan pengembangan sistem dengan teknologi modern seperti *framework* Laravel.

c. Bagi Masyarakat atau Pembaca

Memberikan sarana deteksi dini yang mudah diakses untuk mengenali gejala penyakit gigi dan mulut pada anak, sehingga orang tua dapat lebih cepat mengambil langkah pencegahan maupun penanganan. Selain itu, sistem pakar ini menyajikan informasi kesehatan gigi dan mulut anak secara sederhana dan mudah dipahami, sekaligus memberikan wawasan mengenai penerapan sistem pakar dalam bidang kesehatan.

d. Bagi Institusi

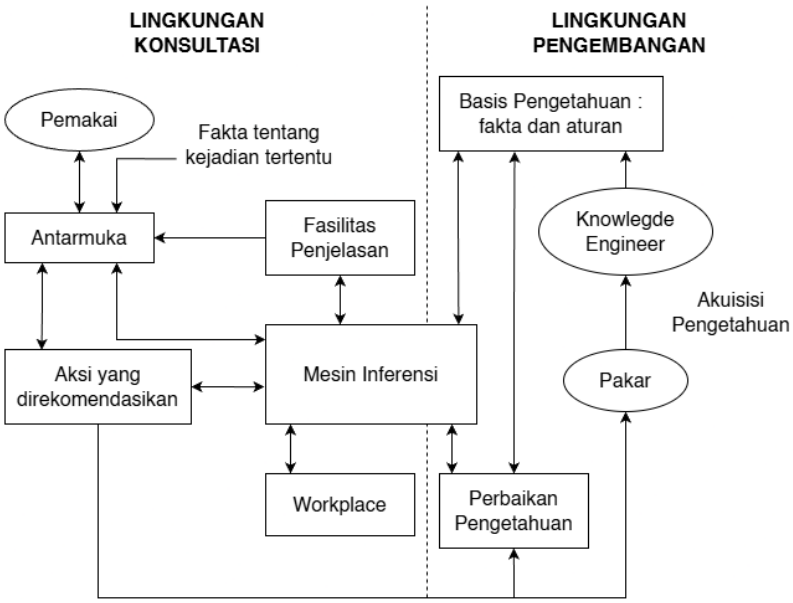
Memberikan kontribusi dalam pengembangan riset di bidang sistem pakar dan kesehatan di Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman. Penelitian ini dapat menjadi referensi akademis bagi penelitian sejenis serta memperkuat kolaborasi antara institusi pendidikan dengan layanan kesehatan dalam penerapan teknologi untuk meningkatkan kualitas pelayanan medis.

BAB II
TINJAUAN PUSTAKA

2.1 Sistem Pakar

Sistem pakar merupakan aplikasi berbasis komputer yang dirancang untuk meniru cara berpikir seorang ahli dalam memecahkan masalah pada bidang tertentu. Program ini menggabungkan basis pengetahuan (*knowledge base*) dengan mekanisme inferensi untuk menghasilkan solusi layaknya seorang pakar. Sebagai bentuk kecerdasan buatan, sistem pakar berperan sebagai konsultan atau penasihat cerdas yang dapat memberikan saran maupun diagnosis berdasarkan kumpulan pengetahuan dari para ahli. Dengan demikian, sistem pakar memungkinkan orang awam untuk memperoleh bantuan dalam menyelesaikan masalah yang biasanya hanya dapat ditangani oleh pakar (Matondang, 2023).

Secara arsitektural, sistem pakar terdiri dari dua lingkungan utama yaitu lingkungan pengembangan dan lingkungan konsultasi. Lingkungan pengembangan ditujukan untuk pakar atau *knowledge engineer* yang bertanggung jawab dalam membangun dan memperbarui basis aturan serta komponen-komponen sistem pakar. Sementara itu, lingkungan konsultasi berfungsi sebagai sistem konsultasi yang digunakan oleh pengguna non-ahli untuk berinteraksi langsung dengan sistem dalam memperoleh solusi atas permasalahan yang dihadapi (Afdhal & Mayola, 2022). Kedua lingkungan ini saling terhubung dan membentuk arsitektur sistem pakar yang terintegrasi, sebagaimana dapat dilihat pada Gambar 1.



Gambar 1. Lingkungan Sistem Pakar

Dalam struktur dasarnya, sistem pakar memiliki beberapa komponen utama yang bekerja secara terintegrasi. Komponen-komponen tersebut meliputi antarmuka pengguna (*user interface*) yang berfungsi sebagai media komunikasi antara pengguna dan sistem, basis pengetahuan (*knowledge base*) yang menyimpan pengetahuan dalam bentuk aturan atau fakta, dan mesin inferensi (*inference engine*) yang bertanggung jawab untuk menarik kesimpulan dengan menerapkan teknik penalaran seperti *forward chaining* atau *backward chaining*. Selain ketiga komponen utama tersebut, sistem pakar juga dilengkapi dengan komponen pendukung seperti akuisisi pengetahuan untuk mengumpulkan dan mentransfer keahlian dari pakar ke dalam sistem, *workplace* atau *blackboard* sebagai area memori kerja untuk merekam kejadian yang sedang berlangsung, fasilitas penjelasan untuk memberikan penjelasan tentang proses penalaran sistem, serta modul perbaikan pengetahuan yang memungkinkan sistem untuk

menganalisis dan meningkatkan kinerjanya secara berkelanjutan (Alam et al., 2021).

2.2 Metode *Forward Chaining*

Forward chaining merupakan metode inferensi dengan pendekatan *data-driven*, di mana penalaran dimulai dari fakta atau informasi yang tersedia untuk mencapai kesimpulan. Aturan-aturan dalam basis pengetahuan dievaluasi secara berurutan, dimulai dari bagian *IF* menuju *THEN* (Suryono, S., & Dewi, 2023). Jika premis sesuai dengan kondisi yang ada, maka aturan dieksekusi dan menghasilkan fakta baru yang ditambahkan ke basis data. Fakta baru ini digunakan kembali untuk mengevaluasi aturan berikutnya hingga diperoleh kesimpulan akhir (Ikhsanto et al., 2022). Dengan cara ini, sistem pakar dapat melakukan penalaran secara bertahap berdasarkan data masukan yang diberikan pengguna.

Metode ini disebut pencarian maju (*forward search*) karena bergerak dari premis menuju kesimpulan akhir. Prosesnya sesuai untuk aplikasi diagnosis, di mana gejala yang diketahui pengguna diproses untuk menelusuri kemungkinan penyakit. Selain itu, *forward chaining* juga efektif untuk kasus dengan struktur pohon pengetahuan yang lebar, karena inferensi dilakukan dengan menguji aturan satu per satu dan mengeksekusi hanya sekali jika kondisi terpenuhi (Mulyani et al., 2023).

2.3 Konsep *Website*

Website merupakan kumpulan halaman digital yang berisi informasi dalam berbagai bentuk, seperti teks, gambar, animasi, suara, maupun video, serta kombinasi dari semuanya, yang dapat diakses melalui jaringan internet (Andini et al., 2025). Konsep *website* pertama kali diperkenalkan oleh Tim Berners-Lee, ilmuwan asal Inggris yang menciptakannya untuk mempermudah para peneliti dalam berbagi informasi di lingkungan kerjanya. Sebelum tahun 1990-an, Berners-Lee merancang tiga teknologi utama yang menjadi dasar perkembangan *website*, yaitu *Hyper Text Markup Language* (HTML) sebagai bahasa *markup*, *Uniform Resource Identifier* (URI) yang kini dikenal sebagai *Uniform Resource Locator* (URL) untuk mengidentifikasi sumber daya, serta *Hyper Text Transfer Protocol* (HTTP) sebagai protokol jaringan yang memungkinkan pertukaran data antar komputer, termasuk dokumen, gambar, dan video (Kusumawardani, Dwi Mustika, 2023).

2.4 PHP

PHP (PHP: *Hypertext Preprocessor*) merupakan bahasa pemrograman sisi server yang pertama kali diperkenalkan oleh Rasmus Lerdorf pada tahun 1994 dan hingga kini berkembang sebagai salah satu bahasa paling populer untuk pengembangan *website*. Bersifat *open-source*, PHP memungkinkan pembuatan konten *website* dinamis, pengolahan formulir, pengelolaan sesi pengguna, serta integrasi dengan berbagai basis data, terutama MySQL. Bahasa ini juga mendukung banyak protokol internet, menyediakan beragam fungsi bawaan, serta terus mengalami perkembangan mencakup dukungan pemrograman berorientasi objek, peningkatan kinerja, dan perbaikan keamanan. Walaupun

menghadapi persaingan dari bahasa pemrograman *website* lainnya, PHP tetap banyak digunakan karena kemudahan, dukungan *hosting* yang luas, serta ekosistem komunitas yang aktif (Sugito & Shafira, 2023).

2.5 *Framework* Laravel

Laravel merupakan salah satu *framework* pengembangan aplikasi *website* yang pertama kali diperkenalkan pada tahun 2011 dan hingga kini masih banyak digunakan oleh para pengembang. *Framework* ini dikenal bersifat dinamis, memiliki dokumentasi yang lengkap, serta terus diperbarui. Arsitektur utamanya berbasis *Model-View-Controller* (MVC) dan didukung oleh *tool* baris perintah bernama Artisan yang memudahkan pengelolaan paket (*packaging bundle*) maupun instalasi melalui *command prompt* (cmd) (Habibi, 2020).

2.6 Perangkat Lunak

Perangkat lunak atau *software* merupakan komponen yang tidak berwujud secara fisik, sehingga tidak dapat disentuh maupun dilihat, tetapi dapat dijalankan untuk mengoperasikan komputer. Perangkat lunak dapat dipahami sebagai sekumpulan instruksi yang ketika dijalankan akan menghasilkan fungsi, fitur, serta kinerja sesuai kebutuhan pengguna (Sari, 2021). Dalam perancangan dan pembangunan sistem ini, peneliti memanfaatkan beberapa perangkat lunak, antara lain:

a. Virtual Studio Code

Visual Studio Code adalah *text editor* bersifat *open-source* yang mendukung berbagai bahasa pemrograman serta dilengkapi dengan beragam ekstensi. Perangkat lunak ini dikembangkan oleh Microsoft dan dapat dijalankan pada

berbagai sistem operasi, termasuk Linux, MacOS, dan Windows (Firnando et al., 2023).

b. Laragon

Dalam memulai desain serta pemrograman *website*, diperlukan persiapan *website* server untuk keperluan pengujian dan memastikan program berjalan sesuai yang diharapkan. Salah satu aplikasi yang banyak digunakan karena kestabilannya adalah Laragon, yang menyediakan lingkungan pengembangan terpadu. Laragon telah dilengkapi dengan berbagai paket pendukung seperti Apache, Nginx, MySQL, PHP, Node.js, dan komponen lainnya (Kholil, M., & Akhsani, 2021).

c. MySQL

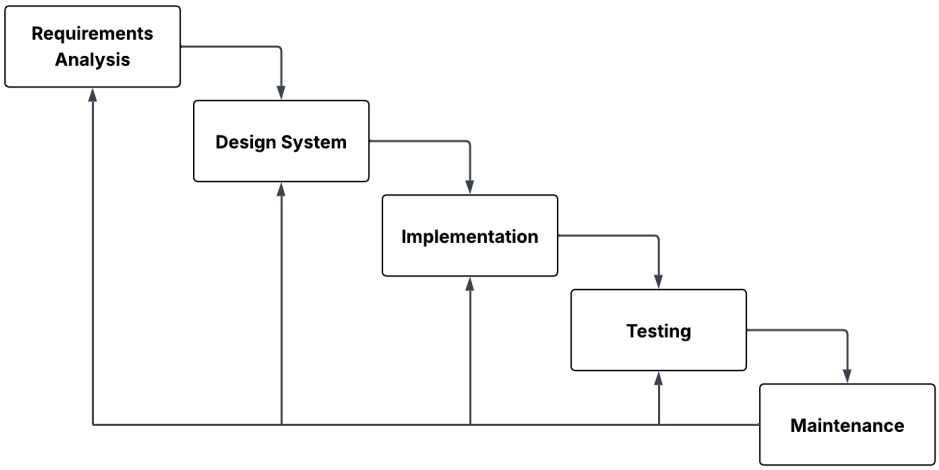
MySQL adalah sistem manajemen basis data relasional (RDBMS) *open-source* yang populer, dikenal karena kecepatan, keandalan, dan fleksibilitasnya dalam mengelola data terstruktur. Dikembangkan oleh Oracle Corporation, MySQL mendukung bahasa *query* standar SQL dan dapat digunakan pada berbagai *platform*, menjadikannya pilihan utama untuk banyak aplikasi *website* dan sistem manajemen konten (Aqilah et al., 2023).

d. PHPMyAdmin

PHPMyAdmin adalah aplikasi berbasis *website* yang dikembangkan dengan PHP untuk mempermudah administrasi basis data MySQL melalui antarmuka grafis (GUI). Perangkat lunak gratis ini memungkinkan pengguna melakukan tugas seperti membuat, mengedit, menghapus tabel, serta menjalankan perintah SQL secara langsung (Sari Siregar et al., 2024).

2.7 Metode *Waterfall*

Metode *waterfall* merupakan model pengembangan perangkat lunak yang bersifat sistematis dan berurutan, dimulai dari analisis kebutuhan pengguna, kemudian dilanjutkan dengan tahap perencanaan, permodelan, pembangunan, hingga penyerahan sistem kepada pengguna (Nur Adiya et al., 2024). Proses ini diakhiri dengan dukungan atau pemeliharaan terhadap perangkat lunak yang telah dihasilkan. Kelebihan dari metode *waterfall* terletak pada pembagian tahapan yang jelas dan terstruktur, sehingga memudahkan pengelolaan serta pengendalian dalam proses pengembangan. Tahapan dalam metode *waterfall* dapat dilihat pada Gambar 2.



Gambar 2. Metode *Waterfall*

Metode *waterfall* menurut Sommerville memiliki lima tahapan yang terdiri dari (Adi Kurniyanti & Murdiani, 2022):

a. *Requirements Analysis* (Analisis Kebutuhan)

Pada tahap ini dilakukan proses komunikasi untuk membantu pengembang memahami perangkat lunak yang diinginkan pengguna sekaligus menetapkan batasan sistem. Pemahaman tersebut umumnya diperoleh melalui wawancara,

diskusi, maupun survei langsung. Data yang terkumpul kemudian dianalisis guna memperoleh informasi yang diperlukan.

b. *Design System* (Desain Sistem)

Setelah tahap spesifikasi kebutuhan selesai, langkah berikutnya adalah menyiapkan desain sistem. Tahap ini berfungsi untuk merancang dan mendefinisikan arsitektur sistem secara menyeluruh sehingga dapat menjadi acuan dalam proses pengembangan selanjutnya.

c. *Implementation* (Implementasi)

Pada tahap ini, sistem dibangun dalam bentuk program-program kecil yang disebut unit atau modul. Setiap unit dikembangkan secara terpisah, kemudian digabungkan dan diintegrasikan pada tahap berikutnya hingga membentuk sistem yang utuh.

d. *Testing* (Pengujian)

Unit yang telah dikembangkan pada tahap sebelumnya digabungkan ke dalam sistem setelah melalui proses pengujian masing-masing unit. Setelah integrasi selesai, dilakukan pengujian menyeluruh terhadap sistem untuk memastikan tidak terdapat kesalahan atau kegagalan.

e. *Maintenance* (Pemeliharaan)

Pada tahap terakhir, perangkat lunak yang telah selesai dikembangkan dijalankan dan dipelihara. Pemeliharaan mencakup perbaikan terhadap kesalahan yang tidak terdeteksi pada tahap sebelumnya serta pengembangan atau peningkatan sistem sesuai kebutuhan baru yang muncul.

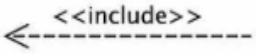
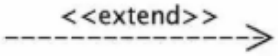
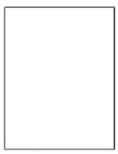
2.8 Pemodelan Sistem

Pemodelan sistem merupakan tahap penting dalam analisis dan perancangan perangkat lunak karena memberikan gambaran yang jelas mengenai alur kerja, interaksi, serta struktur sistem yang akan dibangun. Dengan pemodelan, pengembang dapat memahami kebutuhan sistem secara lebih terstruktur dan memvisualisasikan hubungan antar komponen sebelum diimplementasikan. Dalam penelitian ini, pemodelan sistem dilakukan menggunakan pendekatan berorientasi objek dengan *Unified Modeling Language* (UML). UML adalah sebuah standar pemodelan visual yang digunakan untuk mendeskripsikan, merancang, dan mendokumentasikan sistem perangkat lunak sehingga aspek fungsional maupun strukturalnya dapat direpresentasikan secara komprehensif (Hidayati et al., 2023). Diagram yang digunakan dalam penelitian ini meliputi *use case diagram*, *sequence diagram*, dan *class diagram* untuk menggambarkan kebutuhan fungsional, alur proses, serta struktur kelas dalam sistem.

a. *Use Case Diagram* (Diagram Use Case)

Use case diagram digunakan untuk menggambarkan hubungan antara aktor dengan fungsionalitas sistem (Ramdany, 2024). Diagram ini menampilkan apa saja layanan yang disediakan sistem serta bagaimana interaksi antara pengguna dengan sistem. Komponen dalam *use case diagram* dapat dilihat pada Tabel 1.

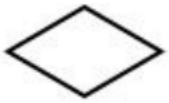
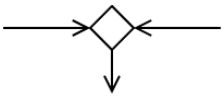
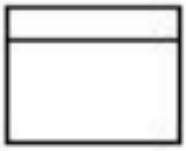
Tabel 1. Komponen *Use Case Diagram*

Gambar	Nama	Keterangan
	*Actor*	Menspesifikasikan himpunan peran yang pengguna mainkan ketika berinteraksi dengan *use case.*
	*Include*	Menunjukkan bahwa suatu *use case* seluruhnya merupakan fungsionalitas dari *use case* lainnya.
	*Extend*	Menunjukkan bahwa suatu *use case* merupakan tambahan fungsional dari *use case* lainnya jika suatu kondisi terpenuhi.
	Association	Apa yang menghubungkan antara objek satu dengan objek lainnya.
	*Use Case*	Deskripsi dari urutan aksi-aksi yang ditampilkan sistem yang menghasilkan suatu hasil yang terukur bagi suatu aktor.
	*System*	Menspesifikasikan paket yang menampilkan sistem secara terbatas.

b. *Activity Diagram* (Diagram Aktivitas)

Activity diagram menggambarkan alur aktivitas atau proses dalam sistem dari awal hingga akhir. Diagram ini menunjukkan urutan aktivitas, keputusan, aliran kontrol yang menggambarkan bagaimana suatu proses berjalan (Wayahdi & Ruziq, 2023). Komponen *activity diagram* dapat dilihat pada Tabel 2.

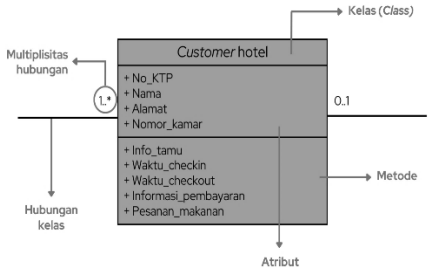
Tabel 2. Komponen *Activity Diagram*

Gambar	Nama	Keterangan
	Status Awal	Sebuah diagram aktivitas memiliki sebuah status awal.
	Aktivitas	Aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja.
	Percabangan/*Decision*	Percabangan dimana ada pilihan aktivitas yang lebih dari satu.
	Penggabungan/*Merge*	Penggabungan dimana lebih dari satu aktivitas digabungkan menjadi satu.
	Status Akhir	Status akhir yang dilakukan sistem, sebuah diagram aktivitas memiliki sebuah status akhir.
	*Swimlane*	*Swimlane* memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi.

c. *Class Diagram* (Diagram Kelas)

Class diagram berfungsi untuk menunjukkan struktur kelas dalam sistem, termasuk atribut, operasi (*method*), serta hubungan antar kelas. Diagram ini penting karena merepresentasikan model data dan logika program yang akan diimplementasikan (Wayahdi & Ruziq, 2023). Komponen dalam *class diagram* dapat dilihat pada Tabel 3.

Tabel 3. Komponen *Class Diagram*

Gambar	Nama	Keterangan
	Nama *Class*	Bagian atas class yang berisi identitas kelas, ditulis dengan huruf kapital di awal kata.
	Atribut	Bagian tengah class yang berisi properti atau variabel data yang dimiliki kelas.
	Metode	Bagian bawah class yang berisi operasi atau fungsi yang dapat dilakukan kelas.
	Hubungan	Garis yang menghubungkan antar class menunjukkan relasi atau hubungan antar kelas dalam sistem.
	Kardinalitas	Notasi jumlah hubungan antar kelas, seperti 1 (*one*), * (*many*), 1..* (*one to many*), 0..1 (*zero or one*), 1..1 (*one to one*).

2.9 Penyakit Gigi dan Mulut pada Anak

Kesehatan gigi dan mulut merupakan komponen penting dari kesehatan anak secara menyeluruh yang dapat mempengaruhi kualitas hidup mereka. Penyakit gigi dan mulut pada anak menjadi salah satu masalah kesehatan yang paling sering dijumpai pada masa pertumbuhan. Berbeda dengan orang dewasa, anak-anak berada dalam fase pertumbuhan dan perkembangan yang dinamis, termasuk perkembangan struktur rongga mulut, dari gigi sulung ke gigi permanen (Florentinsa, 2023). Kerentanan pada periode ini membuat kesehatan gigi dan mulut menjadi fondasi penting untuk kesehatan di masa mendatang.

Penyakit gigi dan mulut pada anak tidak hanya terbatas pada kerusakan gigi, tetapi juga mencakup kelainan pada jaringan lunak seperti gusi dan lidah (Lestari, 2025). Faktor-faktor seperti pola makan, kebiasaan menjaga kebersihan diri yang belum terbentuk, serta struktur anatomi gigi sulung yang lebih rapuh menjadi penyebab utama tingginya prevalensi penyakit ini. Kesehatan mulut anak yang terganggu seringkali berkaitan dengan munculnya rasa sakit, kesulitan mengunyah, gangguan nutrisi, hingga dampak psikologis seperti menurunnya kepercayaan diri.

Terdapat beberapa penyakit gigi dan mulut yang umum terjadi pada anak-anak, di antaranya:

a. Karies Gigi

Karies gigi, yang umum dikenal sebagai gigi berlubang, merupakan penyakit pada jaringan keras gigi yang paling sering terjadi pada anak-anak. Penyakit ini disebabkan oleh proses demineralisasi email gigi akibat asam yang dihasilkan oleh metabolisme bakteri plak, seperti Streptococcus mutans, dari sisa makanan yang mengandung gula. Proses kerusakan ini cenderung berjalan lebih cepat pada anak-anak karena struktur gigi sulung memiliki lapisan email yang lebih tipis dan rentan dibandingkan gigi permanen. Gejala karies gigi pada anak bervariasi tergantung pada tingkat keparahan, mulai dari bercak putih pada permukaan gigi sebagai tanda awal demineralisasi, munculnya lubang kecil hingga besar pada gigi, perubahan warna gigi menjadi coklat atau kehitaman, hingga keluhan nyeri yang dapat bersifat spontan atau dipicu oleh rangsangan tertentu seperti makanan manis, dingin, atau panas (Karima, 2023).

b. Gingivitis

Gingivitis atau radang gusi adalah kondisi peradangan pada jaringan gusi yang merupakan tahap awal dari penyakit periodontal (di sekeliling gigi) dan sering dijumpai pada anak-anak. Penyebab utamanya adalah akumulasi plak bakteri di sepanjang garis gusi akibat kebersihan mulut yang kurang optimal, di mana plak tersebut akan mengiritasi jaringan gusi. Gejala yang paling umum pada anak meliputi gusi yang terlihat lebih merah dari biasanya, bengkak, dan mudah berdarah saat menyikat gigi. Berbeda dengan penyakit gusi tingkat lanjut, gingivitis bersifat *reversibel*, artinya kondisi ini dapat disembuhkan sepenuhnya dengan meningkatkan kebiasaan menjaga kebersihan gigi dan mulut untuk menghilangkan plak secara efektif (Pontoluli et al., 2021).

c. Pulpitis

Pulpitis adalah peradangan pada pulpa gigi, yaitu jaringan lunak di bagian tengah gigi yang berisi pembuluh darah dan saraf. Kondisi ini umumnya merupakan kelanjutan dari karies gigi yang dalam dan tidak dirawat, di mana bakteri dari lubang gigi berhasil menembus lapisan email dan dentin hingga mengiritasi pulpa. Gejala pulpitis bervariasi tergantung pada tingkat keparahannya; pada tahap awal (pulpitis *reversibel*), anak mungkin merasakan nyeri tajam yang singkat saat mengonsumsi sesuatu yang dingin atau manis. Namun, jika peradangan berlanjut menjadi parah (pulpitis *ireversibel*), rasa nyeri akan menjadi spontan, berdenyut, dan bertahan lama bahkan setelah rangsangan dihilangkan. Pulpitis yang tidak ditangani dapat menyebabkan kematian jaringan pulpa (*nekrosis*) dan infeksi lebih lanjut seperti abses gigi (Safa Haqiqah & Dyah Ayu Murika Sari, 2025).

d. Abses Gigi

Abses gigi adalah penumpukan nanah yang terlokalisasi di sekitar akar gigi sebagai akibat dari infeksi bakteri yang parah. Kondisi ini merupakan komplikasi serius yang terjadi ketika pulpitis tidak dirawat, menyebabkan jaringan pulpa mati (nekrosis) dan infeksi menyebar keluar dari ujung akar ke tulang di sekitarnya. Gejala khas pada anak meliputi nyeri hebat yang konstan dan berdenyut, gigi menjadi sangat sakit saat disentuh atau digunakan untuk mengunyah, serta pembengkakan pada gusi dan wajah. Sering kali, kondisi ini juga disertai dengan munculnya benjolan pada gusi yang mengeluarkan nanah (fistula) atau bahkan demam. Abses gigi adalah kondisi darurat yang memerlukan penanganan segera untuk mencegah penyebaran infeksi yang lebih luas dan kerusakan pada benih gigi permanen yang sedang berkembang (Ida Ayu Bulan Kencana Dewi, 2025).

e. Persistensi Gigi Sulung

Persistensi gigi sulung adalah suatu kondisi di mana gigi sulung tidak tanggal pada waktunya, meskipun gigi permanen penggantinya telah atau sedang erupsi ke dalam rongga mulut. Penyebab paling umum adalah jalur erupsi gigi permanen yang menyimpang, sehingga tidak mendorong akar gigi sulung dan proses resorpsi akar tidak terjadi secara alami. Secara klinis, kondisi ini sering tampak sebagai "gigi baris ganda" (*shark teeth*), terutama pada gigi seri bawah, di mana gigi permanen tumbuh di belakang gigi sulung yang masih kokoh. Apabila tidak ditangani, persistensi dapat menyebabkan gigi permanen terperangkap dalam posisi yang salah, memicu masalah maloklusi (gigi berjejal), dan menciptakan area yang sulit dibersihkan. Penanganan untuk

kondisi ini umumnya adalah pencabutan gigi sulung yang persisten untuk memberi jalan bagi gigi permanen menempati posisinya yang benar (C. I. Dewi et al., 2024).

f. Trauma Gigi

Trauma gigi merupakan cedera yang terjadi pada gigi dan struktur pendukungnya akibat benturan atau kecelakaan, yang sering dialami anak-anak karena aktivitas fisik yang tinggi dan koordinasi motorik yang belum sempurna. Trauma dapat berupa retak email ringan hingga avulsi (gigi terlepas sepenuhnya), dengan penyebab utama meliputi jatuh saat bermain, kecelakaan sepeda, atau olahraga kontak, dimana gigi anterior atas paling sering terkena karena posisinya yang menonjol. Gejala trauma gigi bervariasi mulai dari nyeri pada gigi, mobilitas atau pergeseran posisi gigi, perubahan warna gigi menjadi keabu-abuan atau kehitaman yang menandakan kematian pulpa, pembengkakan dan perdarahan gusi, hingga fraktur mahkota atau akar gigi, sehingga memerlukan penanganan segera untuk mencegah komplikasi jangka panjang seperti infeksi atau gangguan perkembangan gigi permanen (A. R. Dewi et al., 2021).

g. Fluorosis Dental

Fluorosis dental merupakan kondisi perubahan warna dan tekstur gigi yang disebabkan oleh konsumsi *fluoride* berlebihan selama masa pembentukan gigi (amelogenesis), terutama pada anak-anak usia 0-8 tahun ketika gigi permanen sedang berkembang. Kondisi ini terjadi akibat paparan *fluoride* kronis yang melebihi batas aman, baik dari air minum yang mengandung *fluoride* tinggi, pasta gigi ber-*fluoride* yang tertelan secara berlebihan, atau suplemen *fluoride*

yang tidak tepat dosisnya. Gejala fluorosis dental bervariasi dari ringan hingga berat, dimulai dari bercak putih atau garis-garis putih halus pada permukaan gigi (fluorosis ringan), perubahan warna menjadi kuning hingga coklat (fluorosis sedang), hingga pada kasus berat dapat terjadi hipoplasia email dengan permukaan gigi yang kasar, berlubang-lubang kecil (*pitting*), dan mudah patah, yang tidak hanya mempengaruhi estetika tetapi juga fungsi gigi dalam jangka panjang (Syuhriah Syuhriah et al., 2024).

h. Hipoplasia Email

Hipoplasia email merupakan kelainan perkembangan gigi yang ditandai dengan pembentukan email yang tidak sempurna, sehingga menghasilkan lapisan email yang tipis, tidak rata, atau bahkan tidak terbentuk pada beberapa bagian permukaan gigi. Kondisi ini dapat disebabkan oleh berbagai faktor selama masa pembentukan gigi, antara lain infeksi sistemik pada masa kehamilan atau bayi (seperti rubella, sifilis kongenital), defisiensi nutrisi (terutama kalsium, fosfor, dan vitamin A, C, D), trauma lokal pada gigi sulung yang dapat mempengaruhi benih gigi permanen di bawahnya, atau paparan bahan kimia tertentu. Gejala hipoplasia email tampak sebagai bercak putih, kuning, atau coklat pada permukaan gigi, permukaan gigi yang kasar atau berlekuk-lekuk, email yang mudah aus atau patah, peningkatan sensitivitas gigi terhadap rangsangan panas, dingin, atau manis, serta risiko tinggi terjadinya karies karena struktur email yang defektif memberikan perlindungan yang tidak optimal terhadap serangan bakteri dan asam (SYAM, 2024).

i. Stomatitis Aphthosa

Stomatitis aphthosa atau sariawan merupakan peradangan pada mukosa mulut yang ditandai dengan terbentuknya ulser (luka) kecil, bulat atau oval, berwarna putih atau kekuningan dengan tepi kemerahan yang terasa sangat nyeri. Kondisi ini sering terjadi pada anak-anak dan dapat disebabkan oleh berbagai faktor seperti trauma minor akibat tergigit, sikat gigi yang kasar, stres fisik atau emosional, defisiensi vitamin B12, asam folat, atau zat besi, gangguan sistem imun, infeksi virus, atau faktor genetik. Gejala stomatitis aphthosa meliputi munculnya satu atau beberapa ulser di dalam mulut (bibir, pipi bagian dalam, lidah, atau gusi), rasa nyeri yang hebat terutama saat makan, minum, atau berbicara, kesulitan menelan makanan terutama yang asam atau pedas, demam ringan pada beberapa kasus, dan pembengkakan kelenjar getah bening di leher, dengan gejala yang biasanya berlangsung 7-14 hari dan dapat sembuh sendiri namun cenderung kambuh secara berulang (Fresno Argadianti et al., 2020).

j. Maloklusi

Maloklusi merupakan kondisi ketidakselarasan atau ketidakharmonisan hubungan antara gigi-gigi pada rahang atas dan bawah ketika mulut tertutup, yang menyebabkan susunan gigi tidak normal dan dapat mempengaruhi fungsi pengunyahan, bicara, serta estetika wajah. Kondisi ini dapat disebabkan oleh faktor genetik (bentuk dan ukuran rahang yang diturunkan), kebiasaan buruk anak seperti mengisap jempol, dot, atau empeng dalam waktu lama, kehilangan gigi sulung secara prematur, persistensi gigi sulung, atau gangguan perkembangan rahang. Gejala maloklusi bervariasi tergantung jenisnya, antara

lain gigi berjejal (*crowding*), gigi renggang (*spacing*), gigi tonggos (*overjet*), gigitan dalam (*deep bite*), gigitan terbuka (*open bite*), gigitan silang (*crossbite*), kesulitan mengunyah atau menggigit makanan dengan efektif, gangguan bicara seperti cadel, nyeri pada sendi temporomandibular (TMJ), serta masalah estetika yang dapat mempengaruhi kepercayaan diri anak, sehingga memerlukan evaluasi ortodontik untuk menentukan waktu dan jenis perawatan yang tepat (Aaliyah et al., 2024).

2.10 *Black-Box Testing*

Black-box testing merupakan metode pengujian perangkat lunak yang berfokus pada fungsi sistem tanpa memperhatikan struktur internal maupun kode program. Pengujian dilakukan dengan memberikan berbagai *input* dan mengamati *output* yang dihasilkan untuk memastikan kesesuaiannya dengan spesifikasi yang telah ditentukan. Tujuan dari metode ini adalah memverifikasi bahwa perangkat lunak berjalan sesuai kebutuhan, di mana skenario uji disusun berdasarkan persyaratan sistem dan hasilnya dibandingkan dengan *output* yang diharapkan (P. P. Adi et al., 2020).

2.11 Penelitian Sejenis

Penelitian ini mengacu pada studi-studi terdahulu yang relevan sebagai rujukan, pertimbangan, sekaligus dasar perbandingan dalam pengembangannya. Kajian penelitian serupa disajikan pada Tabel 4.

Tabel 4. Penelitian Sejenis

No	Peneliti (Tahun)	Judul	Hasil	Perbedaan
1	Siti Sholikhah, Dedy Kurniadi, Andi Riansyah (2021)	Sistem Pakar Menggunakan Metode *Forward Chaining* untuk Diagnosa Hama dan Penyakit Tanaman Padi	Sistem pakar dengan metode *forward chaining* dan aturan *IF-THEN* guna membantu petani mendiagnosis hama dan penyakit pada tanaman padi.	Penelitian dilakukan dengan subjek penelitian pertanian (tanaman padi).
2	Gallant Setiawan, Galang Setia Budi (2023)	Implementasi Metode *Forward Chaining* Pada Sistem Pakar Untuk Penyakit DBD	Penerapan metode *forward chaining* dalam Sistem pakar untuk diagnosis penyakit DBD. Sistem memberikan diagnosa dengan *rule* yang membantu deteksi dini dan penanganan penyakit DBD.	Penelitian dilakukan dengan subjek penelitian penyakit (DBD) pada populasi umum.
3	Raditha Larasaty, Putri Taqwa Prasetyaningrum (2024)	Sistem Pakar Diagnosa Gangguan Kecemasan Pada Difabel Menggunakan Metode *Forward Chaining* Berbasis *Web*	Sistem pakar berbasis *web* untuk diagnosis gangguan kecemasan pada difabel. Pengujian terhadap 15 pasien menunjukkan tingkat akurasi 100% dan sistem sesuai dengan hasil pakar.	Penelitian dilakukan dengan subjek penelitian kesehatan mental (gangguan kecemasan) pada difabel

No	Peneliti (Tahun)	Judul	Hasil	Perbedaan
4	Brian Cahya Purnama (2025)	Sistem Pakar untuk Diagnosis Penyakit Gigi dan Mulut pada Anak dengan Metode *Forward Chaining* Berbasis *Website*	Mengembangkan sistem pakar berbasis *website* untuk diagnosis awal penyakit gigi dan mulut pada anak berdasarkan gejala-gejala yang dimasukkan oleh pengguna menggunakan metode *forward chaining*, dengan melibatkan pengetahuan klinis dokter gigi di Klinik Victory Purwokerto.	Penelitian dilakukan dengan subjek penelitian yang berfokus khusus pada penyakit gigi dan mulut anak dengan melibatkan pakar klinis secara langsung dengan menggunakan metode *forward chaining.*

BAB III
METODOLOGI PENELITIAN

3.1 Waktu dan Tempat Penelitian

Waktu penelitian berlangsung selama empat bulan, terhitung dari bulan September hingga Desember 2025. Penelitian ini dilaksanakan di Klinik Victory Purwokerto.

3.2 Alat Penelitian

Penelitian ini memanfaatkan berbagai alat, baik perangkat keras maupun perangkat lunak, untuk mendukung proses perancangan, pembangunan, dan pengujian sistem. Adapun alat penelitian yang digunakan adalah sebagai berikut:

a. Perangkat Keras

HP Pavilion Gaming Laptop 15-ec2xxx dengan spesifikasi:

- Prosesor : AMD Ryzen 5 5600H
- RAM : 16,0 GB
- Penyimpanan : SSD 512 GB
- GPU : NVIDIA GeForce GTX 1650

b. Perangkat Lunak

- Sistem Operasi Windows 11 Home
- Visual Studio Code
- Laragon
- MySQL
- PHPMyAdmin
- Draw.io

- Microsoft Word

3.3 Data Penelitian

Beberapa teknik pengumpulan data penelitian digunakan untuk mendapatkan informasi yang akurat dan sesuai dengan kebutuhan sistem (Romdona, S., Junista, S. S., & Gunawan, 2025):

a. Studi Literatur

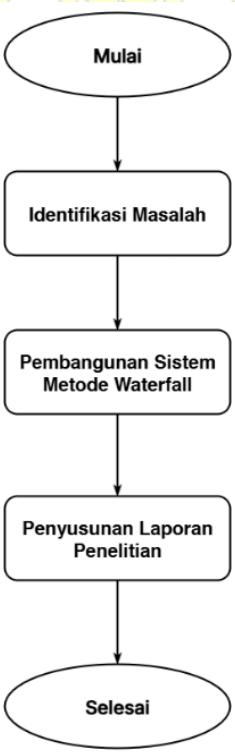
Peneliti melakukan studi literatur dengan menelaah berbagai sumber ilmiah seperti buku kedokteran gigi anak, jurnal penelitian, artikel akademik, dan laporan kesehatan nasional. Studi literatur ini bertujuan untuk memperkuat landasan teori, memperluas cakupan data mengenai penyakit gigi dan mulut pada anak.

b. Wawancara

Sebelum dan selama proses pengembangan aplikasi, peneliti melakukan wawancara dengan pakar. Pakar yang diwawancarai adalah drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani dari Klinik Victory Purwokerto, yang memiliki pengalaman di bidangnya. Melalui wawancara tersebut diharapkan dapat diperoleh data penyakit gigi dan mulut anak yang menjadi fokus diagnosis, data gejala yang umum dialami pasien anak, serta data aturan (*rule base*) yang akan digunakan sebagai dasar proses inferensi dan mendukung pengembangan dalam sistem pakar.

3.4 Tahapan Penelitian

Penelitian berjudul "Sistem Pakar untuk Diagnosis Penyakit Gigi dan Mulut pada Anak dengan Metode *Forward Chaining* Berbasis *Website*" ini menerapkan metode pengembangan perangkat lunak *waterfall*. Pemilihan metode ini didasarkan pada sifatnya yang sistematis dan berurutan, sehingga setiap tahap dapat dilaksanakan secara jelas mulai dari analisis kebutuhan hingga pemeliharaan sistem. Pendekatan ini dianggap sesuai karena sistem pakar yang dikembangkan menuntut kejelasan spesifikasi sejak awal, baik terkait data penyakit, gejala, maupun aturan (*rule base*), serta memastikan sistem yang dihasilkan memenuhi kebutuhan yang telah dirancang. Tahapan penelitian secara keseluruhan digambarkan pada Gambar 3 dan dijelaskan lebih lanjut pada uraian berikut:



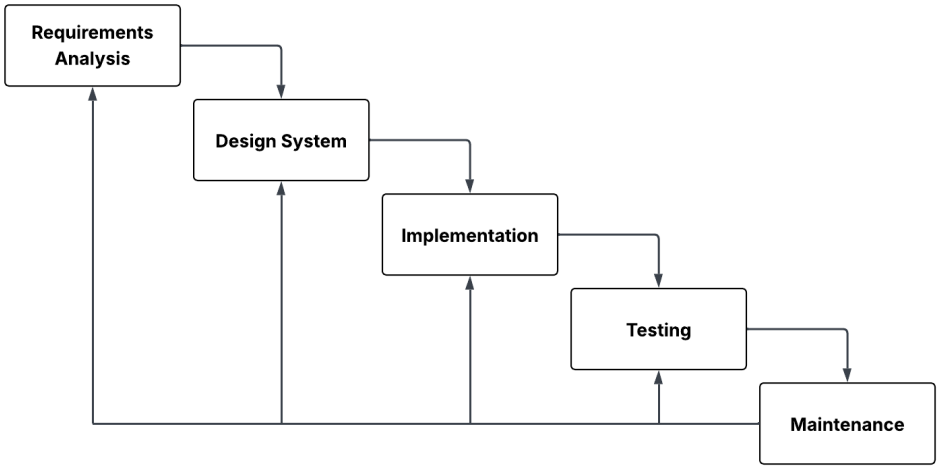
Gambar 3. Tahapan Penelitian

a. Identifikasi Masalah

Tahap identifikasi masalah dilakukan untuk menentukan fokus penelitian berdasarkan permasalahan yang ada dalam diagnosis penyakit gigi dan mulut pada anak. Masalah yang diidentifikasi antara lain keterbatasan pengetahuan orang tua dalam mengenali gejala awal penyakit gigi dan mulut pada anak dan belum adanya sistem yang dapat membantu memberikan informasi diagnosis awal. Dari identifikasi masalah ini kemudian ditetapkan rumusan masalah, tujuan penelitian, dan batasan sistem yang akan dikembangkan.

b. Pembangunan Sistem Metode *Waterfall*

Tahap pembangunan sistem pakar untuk diagnosis penyakit gigi dan mulut pada anak akan dilakukan menggunakan metode *waterfall*. Terdapat lima tahapan *waterfall* seperti pada Gambar 4 yang akan dilakukan secara berurutan dan dijelaskan lebih lanjut pada uraian berikut:



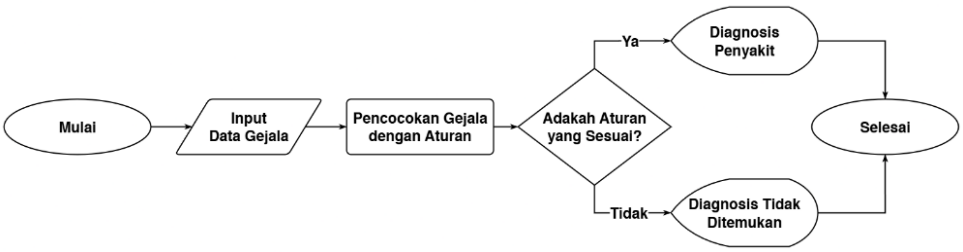
Gambar 4. Pembangunan Sistem Metode *Waterfall*

- *Requirements Analysis.* Pada tahap ini akan dilakukan pengumpulan dan pendefinisian kebutuhan sistem secara detail. Aktivitas yang akan dilakukan meliputi studi literatur untuk mengkaji penelitian terdahulu tentang penyakit

gigi dan mulut pada anak, serta wawancara dengan pakar, yaitu drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani dari Klinik Victory Purwokerto untuk mengidentifikasi jenis-jenis penyakit yang umum terjadi, gejala-gejala yang terkait, dan aturan (*rule base*) yang akan digunakan dalam sistem pakar. Hasil dari tahap ini akan menjadi dasar spesifikasi kebutuhan untuk tahap selanjutnya.

- *Design System*. Tahap ini akan melakukan perancangan arsitektur perangkat lunak berdasarkan spesifikasi kebutuhan yang telah diperoleh dari tahap sebelumnya. Desain yang akan dibuat mencakup struktur basis pengetahuan, alur inferensi menggunakan metode *forward chaining*, serta antarmuka pengguna berbasis *website*. Perancangan akan dilakukan menggunakan model UML (*Unified Modeling Language*) yang terdiri dari *use case diagram, activity diagram,* dan *class diagram* untuk memvisualisasikan aliran data dan interaksi antara pengguna dengan sistem, sehingga memberikan gambaran yang jelas tentang cara kerja sistem.
- *Implementation.* Pada tahap implementasi akan dilakukan pembangunan perangkat lunak berdasarkan desain yang telah dibuat pada tahap sebelumnya. Sistem pakar akan dikembangkan menggunakan *framework* Laravel sebagai *platform* utama dengan teknologi pendukung lainnya. Pembangunan sistem pakar ini mencakup tiga komponen utama. Pertama, akuisisi pengetahuan (*knowledge acquisition*), yaitu proses pengumpulan dan ekstraksi pengetahuan dari pakar, dalam hal ini drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani, yang mencakup data penyakit gigi dan mulut pada anak, gejala-gejala yang terkait, serta hubungan antara gejala dan

penyakit. Pengetahuan ini diperoleh melalui wawancara dan studi literatur yang telah dilakukan pada tahap *requirements analysis*. Kedua, representasi pengetahuan (*knowledge representation*), dimana pengetahuan yang telah diperoleh kemudian direpresentasikan dalam bentuk basis pengetahuan menggunakan aturan *IF-THEN*. Setiap aturan merepresentasikan hubungan antara kombinasi gejala (premis) dengan penyakit tertentu (konklusi), yang disimpan dalam basis data sistem untuk digunakan dalam proses diagnosis. Hasil representasi ini dapat menggambarkan pengetahuan secara terstruktur melalui tabel keputusan, yaitu matriks dua dimensi yang berisi kondisi gejala dan tujuan penyakit sebagai deskripsi kaidah diagnosis. Ketiga, mesin inferensi (*inference engine*), yaitu komponen yang bertugas untuk melakukan penalaran berdasarkan fakta (gejala yang dimasukkan pengguna) dan aturan yang ada dalam basis pengetahuan. Mesin inferensi yang digunakan menerapkan metode *forward chaining* dengan teknik *pruning*, yaitu penalaran maju dari fakta menuju konklusi dengan memangkas cabang pencarian yang tidak relevan untuk menghasilkan diagnosis penyakit secara lebih efisien. Antarmuka pengguna akan dibuat agar mudah diakses dan dipahami melalui perangkat berbasis *website*.



Gambar 5. Alur *Forward Chaining*

Proses *forward chaining* yang diterapkan dalam sistem pakar ini digambarkan secara visual pada Gambar 5. Proses diawali dengan pengguna memasukkan data gejala yang dialami dengan menjawah "ya" atau "tidak" pada setiap gejala yang ditampilkan sistem. Sistem kemudian melakukan pencocokan antara gejala yang dimasukkan dengan aturan-aturan dalam basis pengetahuan, dimana setiap aturan merupakan kombinasi gejala yang menjadi syarat untuk mendiagnosis suatu penyakit tertentu. Dalam proses evaluasi, sistem menerapkan struktur logika *AND-OR* sebagaimana terdefinisi dalam setiap aturan penyakit. Untuk operator *AND* antar grup, sistem memeriksa apakah setiap kelompok gejala wajib (*mandatory group*) terpenuhi, dimana minimal satu gejala dalam setiap grup tersebut harus dijawab "ya" oleh pengguna. Sementara untuk operator *OR* dalam satu grup, sistem hanya memerlukan minimal satu gejala dari beberapa gejala dalam grup tersebut yang terpenuhi agar grup tersebut dianggap valid. Sistem akan mengevaluasi apakah gejala yang dimasukkan memenuhi syarat dari aturan yang tersedia. Apabila ditemukan aturan dengan seluruh syarat terpenuhi, yaitu semua grup *AND* terpenuhi dan setidaknya satu kondisi *OR* dalam setiap grup terpenuhi, sistem menghasilkan diagnosis penyakit yang sesuai. Sebaliknya, jika tidak ada aturan yang terpenuhi setelah semua gejala dievaluasi, sistem akan menampilkan bahwa diagnosis tidak ditemukan. Kondisi ini terjadi ketika kombinasi gejala yang dimasukkan tidak sesuai dengan pola aturan dalam basis pengetahuan, misalnya terdapat grup wajib (*AND*) yang tidak terpenuhi atau kombinasi gejala berada di luar cakupan pengetahuan dari pakar. Sistem akan memberikan pesan bahwa diagnosis

tidak dapat diberikan dan menyarankan pengguna berkonsultasi dengan tenaga medis profesional untuk menghindari diagnosis yang salah.

- *Testing*. Tahap pengujian akan dilakukan untuk memastikan perangkat lunak berjalan sesuai dengan kebutuhan dan spesifikasi yang telah ditentukan. Metode yang akan digunakan adalah *black-box testing*, di mana pengujian difokuskan pada fungsi *input* dan *output* sistem tanpa melihat struktur internal kode. Pengujian ini akan mencakup validasi hasil diagnosis dan fungsionalitas antarmuka pengguna untuk memastikan sistem dapat berjalan dengan baik.
- *Maintenance.* Pada tahap akhir, perangkat lunak yang telah selesai dikembangkan dijalankan dan memasuki fase pemeliharaan. Pemeliharaan yang akan dilakukan meliputi perbaikan terhadap kesalahan yang belum terdeteksi pada tahap sebelumnya serta pengembangan sistem lebih lanjut sesuai kebutuhan baru yang muncul.

c. Penyusunan Laporan Penelitian

Tahap penyusunan laporan penelitian dilakukan secara paralel dengan proses pembangunan sistem. Pada tahap ini, setiap aktivitas penelitian mulai dari analisis kebutuhan, perancangan, implementasi, hingga pengujian didokumentasikan secara sistematis. Laporan memuat analisis dan pembahasan hasil yang diperoleh, evaluasi kinerja sistem, serta kesimpulan yang menjawab rumusan masalah dan tujuan penelitian. Selain itu, disertakan pula saran untuk pengembangan lebih lanjut agar sistem dapat terus disempurnakan.

BAB IV
HASIL DAN PEMBAHASAN

4.1 *Requirements Analysis*

Pada penelitian ini, tahap *requirements analysis* dilakukan melalui studi literatur dan wawancara dengan pakar untuk mengumpulkan informasi yang diperlukan guna memastikan sistem pakar diagnosis penyakit gigi dan mulut pada anak sesuai dengan kebutuhan pengguna sebelum memasuki tahap perancangan dan implementasi. Analisis ini mencakup dua aspek utama, yaitu analisis pengetahuan dan analisis kebutuhan pengguna.

4.1.1 Analisis Pengetahuan

Pada tahap ini, peneliti mengumpulkan data terkait penyakit gigi dan mulut pada anak yang mencakup gejala, deskripsi penyakit, penyebab, serta solusi. Data diperoleh melalui studi literatur dari sumber-sumber medis seperti buku dan referensi jurnal ilmiah, kemudian divalidasi melalui konfirmasi langsung dengan pakar, yaitu drg. Nurulita Ashary dan drg. Catur Aditya Ramadhani dari Klinik Victory Purwokerto. Proses analisis ini dibagi menjadi tiga tahapan utama, yaitu:

a. Akuisisi Pengetahuan

Pada tahap akuisisi pengetahuan, pengetahuan yang diperoleh dari studi literatur dan wawancara dengan pakar ditransformasikan ke dalam bentuk tabel terstruktur yang dapat diproses. Pengetahuan yang menjadi data penelitian mencakup gejala-gejala penyakit gigi dan mulut pada anak sebagaimana tercantum pada Tabel 5, serta jenis-jenis penyakit gigi dan mulut anak yang disajikan pada Tabel 6.

Tabel 5. Gejala Penyakit Gigi dan Mulut pada Anak

Kode Gejala	Deskripsi Gejala
G01	Ada bercak kecoklatan/kehitaman di gigi
G02	Ada lubang di gigi
G03	Gigi sensitif dan rapuh
G04	Gigi berbayang abu-abu
G05	Sering terselip sisa makanan
G06	Sakit apabila terkena minuman/makanan dingin atau manis
G07	Gigi berlubang besar
G08	Ada sakit spontan dan terus-menerus
G09	Gusi bengkak/pembesaran
G10	Gusi keluar nanah
G11	Terdapat cairan atau rasa asin pada sekitar gigi
G12	Gusi kemerahan
G13	Gusi mudah berdarah
G14	Terasa sakit di area tersebut
G15	Luka di gusi
G16	Gigi permanen sudah tumbuh di belakang gigi susu
G17	Gigi terlihat berjejal atau tidak rapi
G18	Gigi goyang
G19	Gigi susu masih kuat
G20	Ada riwayat jatuh atau terbentur
G21	Ada gigi yang patah/retak/lepas
G22	Ada luka di area trauma
G23	Ada bercak putih di gigi
G24	Pernah tertelan *fluoride* (kandungan dalam pasta gigi/obat kumur) pada saat sikat gigi
G25	Mengonsumsi air dengan kadar *fluoride* tinggi
G26	Gigi kecoklatan
G27	Ada bercak putih kekuningan di gigi
G28	Anak memiliki riwayat kekurangan nutrisi saat kehamilan
G29	Ibu memiliki riwayat konsumsi antibiotik Tetracycline saat hamil
G30	Anak pernah terbentur pada saat fase pertumbuhan gigi anak
G31	Ada ulkus (luka) berwarna putih di area mulut
G32	Ada riwayat tergigit/terbentur benda tajam di area mulut
G33	Ada riwayat alergi (makanan, obat, atau bahan tertentu)
G34	Sedang stress/kelelahan/terkena penyakit infeksi

Kode Gejala	Deskripsi Gejala
G35	Memiliki kebiasan buruk (menghisap jempol atau bernapas lewat mulut)
G36	Gigitan terbalik (gigi bawah berada di depan gigi atas saat menggigit)
G37	Rahang atas atau bawah terlalu maju atau mundur
G38	Gigitan terbuka (ada celah antara gigi atas dan bawah saat mulut tertutup/menggigit)

Tabel 6. Jenis Penyakit Gigi dan Mulut pada Anak

Kode Penyakit	Nama Penyakit
P00	Diagnosis Tidak Ditemukan
P01	Karies Gigi
P02	Pulpitis
P03	Abses Gigi
P04	Gingivitis
P05	Persistensi Gigi Sulung
P06	Trauma Gigi
P07	Fluorosis Dental
P08	Hipoplasia Email
P09	Stomatitis Aphthosa
P10	Maloklusi

Tabel 6 mengklasifikasikan 10 jenis penyakit gigi dan mulut yang sering ditemukan pada anak, dengan kode P01 hingga P10, serta satu kategori P00 untuk kasus yang tidak teridentifikasi. Setiap penyakit memerlukan pemahaman yang baik mengenai karakteristiknya agar dapat ditangani dengan tepat. Penjelasan berikut mencakup definisi, penyebab utama, dan solusi penanganan untuk masing-masing penyakit:

- P01 Karies Gigi (Gigi Berlubang)



Gambar 6. Karies Gigi

Sebagaimana terlihat pada Gambar 6, karies gigi adalah kerusakan pada jaringan keras gigi yang disebabkan oleh aktivitas bakteri di dalam mulut. Bakteri tersebut memanfaatkan sisa makanan, terutama yang mengandung gula, lalu menghasilkan asam yang perlahan-lahan merusak email gigi. Jika tidak ditangani, kerusakan dapat berlanjut hingga menembus dentin dan menyebabkan gigi berlubang.

Kondisi ini merupakan salah satu masalah gigi yang paling umum dialami anak-anak maupun orang dewasa. Selain menimbulkan rasa tidak nyaman, karies gigi juga dapat mengganggu fungsi mengunyah, estetika, dan kesehatan gigi secara keseluruhan. Karies gigi dapat disebabkan oleh beberapa faktor, di antaranya:

- Konsumsi makanan/minuman manis atau asam secara berlebihan
- Jarang menyikat gigi atau teknik menyikat gigi yang salah
- Penumpukan plak dan karang gigi
- Produksi air liur yang sedikit (mulut kering)
- Tidak rutin periksa ke dokter gigi

Pencegahan karies gigi dapat dilakukan dengan:

- Menyikat gigi 2x sehari pagi setelah makan dan malam sebelum tidur

- Menyikat gigi secara perlahan dan minimal 2 menit dengan teknik memutar
- Menggunakan *dental floss* atau benang gigi untuk membersihkan sela-sela gigi
- Mengurangi konsumsi makanan tinggi gula dan asam
- Gunakan obat kumur antibakteri setelah menyikat gigi
- Konsumsi air putih yang cukup untuk menjaga kelembaban mulut
- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Ganti sikat gigi setiap 3 bulan

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan penambalan gigi

- P02 Pulpitis (Peradangan Saraf Gigi)



Gambar 7. Pulpitis

Sebagaimana terlihat pada Gambar 7, pulpitis adalah peradangan pada pulpa gigi, yaitu jaringan lunak yang berisi saraf dan pembuluh darah di dalam gigi. Peradangan ini biasanya terjadi ketika karies sudah cukup dalam hingga mencapai saraf gigi (*pulpa*), menyebabkan saraf menjadi bengkak, meradang, dan sangat sensitif.

Kondisi ini ditandai dengan rasa sakit spontan atau nyeri tajam saat gigi terkena rangsangan dingin, panas, atau manis. Jika tidak segera ditangani,

pulpitis dapat berkembang menjadi lebih serius hingga menimbulkan infeksi atau kematian saraf gigi. Pulpitis dapat disebabkan oleh beberapa faktor, di antaranya:

- Karies gigi yang dalam dan tidak ditangani
- Trauma pada gigi (misalnya retak atau patah)
- Perawatan gigi yang tidak sempurna (tambalan bocor)
- Infeksi yang menyebar dari jaringan sekitar gigi

Pencegahan pulpitis dapat dilakukan dengan:

- Menyikat gigi 2x sehari pagi setelah makan dan malam sebelum tidur
- Menyikat gigi secara perlahan dan minimal 2 menit dengan teknik memutar
- Menggunakan *dental floss* atau benang gigi untuk membersihkan sela-sela gigi
- Mengurangi konsumsi makanan tinggi gula dan asam
- Gunakan obat kumur antibakteri setelah menyikat gigi
- Konsumsi air putih yang cukup untuk menjaga kelembaban mulut
- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Ganti sikat gigi setiap 3 bulan

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Periksa ke dokter gigi untuk tambal gigi, perawatan akar gigi, atau pencabutan
- Kompres dingin atau minum obat pereda nyeri untuk mengurangi sakit sementara
- Hindari makanan/minuman terlalu panas atau dingin

- P03 Abses Gigi (Gusi Bengkak)



Gambar 8. Abses Gigi

Sebagaimana terlihat pada Gambar 8, abses gigi adalah infeksi bakteri yang menyebabkan terbentuknya kantong nanah di sekitar akar gigi. Biasanya, kondisi ini terjadi akibat karies gigi yang tidak ditangani hingga menyebabkan kerusakan mendalam atau karena gigi yang sudah mati sarafnya.

Infeksi pada abses gigi dapat menimbulkan rasa sakit hebat, pembengkakan, hingga munculnya cairan bernanah dengan rasa tidak sedap di mulut. Jika dibiarkan, abses dapat menyebar ke jaringan lain di sekitar mulut dan menimbulkan komplikasi yang lebih serius. Abses gigi dapat disebabkan oleh beberapa faktor, di antaranya:

- Karies gigi atau sisa akar gigi yang parah dan tidak dirawat
- Perawatan saluran akar yang gagal
- Cedera pada gigi yang menyebabkan saraf mati
- Infeksi dari gusi atau jaringan sekitarnya

Pencegahan abses gigi dapat dilakukan dengan:

- Menyikat gigi 2x sehari pagi setelah makan dan malam sebelum tidur
- Menyikat gigi secara perlahan dan minimal 2 menit dengan teknik memutar

- Menggunakan *dental floss* atau benang gigi untuk membersihkan sela-sela gigi
- Mengurangi konsumsi makanan tinggi gula dan asam
- Gunakan obat kumur antibakteri setelah menyikat gigi
- Konsumsi air putih yang cukup untuk menjaga kelembaban mulut
- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Ganti sikat gigi setiap 3 bulan

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi perawatan saluran akar atau pencabutan
- Berkumur dengan air garam hangat sebagai pertolongan pertama
- Hindari makanan terlalu panas/dingin dan keras
- Segera ke dokter gigi jika ada tanda infeksi

- P04 Gingivitis (Gusi Bengkak dan Berdarah)



Gambar 9. Gingivitis

Sebagaimana terlihat pada Gambar 9, gingivitis adalah peradangan pada jaringan gusi yang disebabkan oleh penumpukan plak dan bakteri di sekitar gigi. Plak yang tidak dibersihkan dengan baik akan mengiritasi gusi, sehingga menjadi merah, bengkak, dan mudah berdarah saat menyikat gigi.

Kondisi ini merupakan tahap awal dari penyakit gusi. Meski sering dianggap sepele, gingivitis yang tidak ditangani dapat berkembang menjadi

periodontitis, yang dapat merusak jaringan penyangga gigi dan menyebabkan gigi goyah atau tanggal. Gingivitis dapat disebabkan oleh beberapa faktor, di antaranya:

- Penumpukan plak dan karang gigi
- Kebersihan mulut yang buruk
- Perubahan hormon
- Kekurangan vitamin C

Pencegahan gingivitis dapat dilakukan dengan:

- Menyikat gigi 2x sehari pagi setelah makan dan malam sebelum tidur
- Menyikat gigi secara perlahan dan minimal 2 menit dengan teknik memutar
- Menggunakan *dental floss* atau benang gigi untuk membersihkan sela-sela gigi
- Mengurangi konsumsi makanan tinggi gula dan asam
- Gunakan obat kumur antibakteri setelah menyikat gigi
- Konsumsi air putih yang cukup untuk menjaga kelembaban mulut
- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Ganti sikat gigi setiap 3 bulan
- Pijat gusi dengan lembut saat menyikat gigi

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan pembersihan karang gigi
- Berkumur dengan larutan antiseptik ringan atau obat kumur antiseptic

- P05 Persistensi Gigi Sulung (Gigi Kesundulan)



Gambar 10. Persistensi Gigi Sulung

Sebagaimana terlihat pada Gambar 10, persistensi gigi sulung adalah kondisi ketika gigi susu tidak tanggal pada waktunya meskipun gigi tetap sudah mulai tumbuh. Hal ini dapat menyebabkan gigi tetap tumbuh di posisi yang salah, misalnya muncul di belakang gigi susu.

Jika dibiarkan, kondisi ini bisa menimbulkan masalah susunan gigi yang tidak rapi dan mengganggu fungsi gigitan anak. Oleh karena itu, pemantauan tumbuh kembang gigi anak sejak dini sangat penting dilakukan. Persistensi gigi sulung dapat disebabkan oleh beberapa faktor, di antaranya:

- Gigi susu yang akarnya tidak mau memendek
- Posisi gigi tetap yang tidak tepat
- Faktor keturunan
- Kurangnya rangsangan untuk gigi susu tanggal
- Makanan yang terlalu lembut sehingga gigi susu tidak terpakai
- Kehilangan gigi susu terlalu lambat

Pencegahan persistensi gigi sulung dapat dilakukan dengan:

- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Pantau pertumbuhan gigi anak secara berkala

- Berikan makanan yang membantu proses tanggalnya gigi susu secara alami

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan pencabutan gigi persistensi

- P06 Trauma Gigi (Gigi Patah/Lepas karena Kecelakaan)



Gambar 11. Trauma Gigi

Sebagaimana terlihat pada Gambar 11, trauma gigi adalah kerusakan pada gigi akibat benturan fisik, seperti terjatuh, kecelakaan, atau terkena benda keras. Benturan tersebut dapat menyebabkan gigi retak, patah sebagian, atau bahkan lepas seluruhnya.

Selain gigi, trauma juga sering melibatkan jaringan lunak di sekitarnya, misalnya gusi atau bibir yang terluka. Kondisi ini membutuhkan penanganan segera agar kerusakan tidak bertambah parah dan gigi dapat diselamatkan. Trauma gigi dapat disebabkan oleh beberapa faktor, di antaranya:

- Jatuh atau terbentur saat bermain/berolahraga
- Kecelakaan lalu lintas
- Kebiasaan menggigit benda keras
- Benturan fisik pada wajah atau mulut

Pencegahan trauma gigi dapat dilakukan dengan:

- Menghindari aktivitas yang dapat memicu trauma gigi

- Gunakan pelindung gigi (*mouth guard*) saat berolahraga kontak
- Hindari menggigit benda keras
- Gunakan pengaman saat berkendara

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi pasca trauma
- Segera ke dokter gigi dalam 30 menit jika gigi lepas
- Kompres dingin untuk mengurangi pembengkakan

- P07 Fluorosis Dental (Bercak Putih karena *Fluoride* Berlebih)



Gambar 12. Fluorosis Dental

Sebagaimana terlihat pada Gambar 12, fluorosis dental adalah kondisi munculnya bercak-bercak putih pada gigi akibat paparan *fluoride* yang berlebihan ketika gigi masih berkembang. Biasanya terjadi pada masa kanak-kanak ketika anak sering menelan pasta gigi atau mengonsumsi air dengan kadar *fluoride* tinggi.

Meskipun tidak menimbulkan rasa sakit, fluorosis dapat mengganggu estetika gigi. Pada kasus yang berat, gigi bisa berubah warna menjadi kecoklatan dan lebih rapuh. Fluorosis dental dapat disebabkan oleh beberapa faktor, di antaranya:

- Menelan pasta gigi terlalu sering saat kecil
- Air minum yang kandungan *fluoride* nya tinggi
- Suplemen *fluoride* berlebihan
- Penggunaan pasta gigi dewasa pada anak kecil
- Tidak diawasi saat sikat gigi
- Konsumsi makanan/minuman tinggi *fluoride*

Pencegahan fluorosis dental dapat dilakukan dengan:

- Menggunakan pasta gigi sesuai dengan umur (ukuran beras untuk anak kurang dari 3 tahun)
- Ketika menyikat gigi diawasi oleh orang tua
- Ajarkan anak untuk tidak menelan pasta gigi
- Pantau asupan *fluoride* dari sumber lain (air minum, suplemen)

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan penambalan gigi

- P08 Hipoplasia Email (Lapisan Gigi Tidak Sempurna)



Gambar 13. Hipoplasia Email

Sebagaimana terlihat pada Gambar 13, hipoplasia email adalah kelainan pada lapisan terluar gigi atau email yang tidak terbentuk dengan sempurna saat gigi berkembang. Kondisi ini dapat disebabkan oleh berbagai faktor, seperti kekurangan nutrisi, demam tinggi, atau infeksi pada masa pertumbuhan gigi.

Gigi dengan hipoplasia biasanya tampak bercak putih atau kekuningan, lebih sensitif, dan rapuh. Hal ini membuat gigi lebih mudah mengalami karies jika tidak dirawat dengan baik. Hipoplasia email dapat disebabkan oleh beberapa faktor, di antaranya:

- Kekurangan nutrisi (kalsium, vitamin D, fosfor)
- Infeksi atau demam tinggi pada masa pertumbuhan gigi
- Gangguan selama kehamilan (misalnya ibu sakit atau konsumsi obat tertentu)
- Trauma pada gigi yang sedang tumbuh
- Konsumsi antibiotik Tetracycline saat hamil

Pencegahan hipoplasia email dapat dilakukan dengan:

- Mengkonsumsi makanan yang tinggi kalsium, fosfor, dan vitamin D
- Hindari demam tinggi dan infeksi selama kehamilan dan masa kanak-kanak
- Konsumsi makanan bergizi seimbang selama kehamilan
- Hindari obat-obatan tertentu selama kehamilan tanpa konsultasi dokter

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan penambalan gigi
- Penggunaan pasta gigi khusus untuk gigi sensitive

- P09 Stomatitis Aphthosa (Sariawan)



Gambar 14. Stomatitis Aphthosa

Sebagaimana terlihat pada Gambar 14, stomatitis aphthosa adalah luka kecil pada jaringan lunak di dalam mulut yang biasanya berbentuk bulat atau oval berwarna putih kekuningan dengan tepi kemerahan. Luka ini dapat muncul di pipi bagian dalam, lidah, atau bibir.

Penyebabnya beragam, mulai dari tergigit, stres, kekurangan vitamin, hingga penurunan daya tahan tubuh. Meski sering sembuh dengan sendirinya, stomatitis dapat menimbulkan rasa nyeri sehingga mengganggu makan dan berbicara. Stomatitis aphthosa dapat disebabkan oleh beberapa faktor, di antaranya:

- Tidak sengaja menggigit pipi, bibir, atau lidah
- Stress dan kelelahan
- Kekurangan vitamin B, C, atau zinc
- Sistem kekebalan tubuh menurun
- Infeksi virus atau jamur
- Alergi terhadap makanan tertentu
- Perubahan hormon

Pencegahan stomatitis dapat dilakukan dengan:

- Menghindari kontak dengan makanan yang tajam, keras, dan terlalu panas/pedas
- Rutin mengkonsumsi makanan mengandung banyak vitamin B, C, dan zinc
- Menghindari stress berlebihan
- Menjaga kebersihan mulut dengan baik
- Hindari makanan pedas dan asam saat mulut sedang luka
- Konsumsi banyak air putih

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan medikasi atau pengobatan
- Berkumur dengan larutan antiseptik ringan atau obat kumur antiseptik
- Gunakan obat pereda nyeri topikal jika diperlukan
- Konsumsi makanan lunak selama masa penyembuhan

- P10 Maloklusi (Gigi Tidak Rapi)



Gambar 15. Maloklusi

Sebagaimana terlihat pada Gambar 15, maloklusi adalah kondisi ketika susunan gigi tidak sejajar dengan baik atau gigitan tidak normal. Bentuknya bisa berupa gigi berjejal, tonggos, rahang maju/mundur, gigitan terbuka, atau gigitan terbalik.

Kondisi ini bisa disebabkan faktor keturunan maupun kebiasaan buruk, seperti menghisap jempol atau bernapas lewat mulut. Jika tidak ditangani, maloklusi dapat mempengaruhi fungsi mengunyah, bicara, serta penampilan estetik seseorang. Maloklusi dapat disebabkan oleh beberapa faktor, di antaranya:

- Faktor keturunan dari orangtua
- Kebiasaan menghisap jempol terlalu lama
- Kebiasaan bernafas lewat mulut
- Kebiasaan bertopang dagu dalam jangka waktu lama
- Menggigit kuku atau benda lain
- Kehilangan gigi susu terlalu dini
- Ukuran rahang yang tidak sesuai dengan ukuran gigi
- *Tongue thrusting* (kebiasaan mendorong gigi dengan lidah)

Pencegahan maloklusi dapat dilakukan dengan:

- Menghindari kebiasaan buruk, seperti bernafas lewat mulut, menggigit kuku, dan menggigit benda-benda tertentu
- Hentikan kebiasaan mengisap jempol setelah usia 2 tahun
- Rutin periksa ke dokter gigi setiap 6 bulan sekali
- Deteksi dini masalah pertumbuhan rahang dan gigi

Jika sudah terjadi, perawatan yang tepat perlu segera dilakukan dengan:

- Melakukan pemeriksaan ke dokter gigi dan pemasangan behel gigi atau perawatan preventif sejak dini
- Retainer untuk mempertahankan hasil perawatan

b. Representasi Pengetahuan

Pengetahuan yang telah diperoleh pada tahap akuisisi pengetahuan kemudian direpresentasikan ke dalam tabel keputusan untuk mempermudah implementasi dalam sistem pakar. Tabel keputusan menyajikan hubungan antara gejala (kondisi/premis) dengan penyakit (kesimpulan/konklusi) dalam bentuk matriks dua dimensi, sehingga memudahkan pemetaan aturan diagnosis ke dalam basis pengetahuan sistem pakar. Representasi pengetahuan dalam bentuk tabel keputusan untuk setiap penyakit gigi dan mulut anak dapat dilihat pada Tabel 7.

Tabel 7. Tabel Keputusan Penyakit Gigi dan Mulut pada Anak

Kode Gejala	**Kode Penyakit**									
	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10
G01	✓	✓								
G02	✓	✓	✓							
G03	✓	✓	✓			✓	✓	✓		
G04	✓	✓	✓							
G05	✓	✓	✓							
G06		✓	✓							
G07		✓	✓							
G08		✓	✓							
G09			✓	✓		✓				
G10			✓	✓						
G11			✓							
G12			✓	✓		✓				
G13				✓						
G14				✓		✓			✓	
G15				✓	✓					
G16					✓					
G17					✓					✓
G18					✓	✓				
G19					✓					
G20						✓				

Kode Gejala	Kode Penyakit									
	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10
G21						✓				
G22						✓				
G23							✓			
G24							✓			
G25							✓			
G26							✓			
G27								✓		
G28								✓		
G29								✓		
G30								✓		
G31									✓	
G32									✓	
G33									✓	
G34									✓	
G35										✓
G36										✓
G37										✓
G38										✓

Untuk dapat diproses oleh komputer, pengetahuan yang telah direpresentasikan dalam tabel keputusan ditransformasikan ke dalam bentuk aturan berbasis pengetahuan (*rule-based*). Setiap aturan ditulis dalam format *IF-THEN*, yang menyatakan bahwa jika sejumlah gejala tertentu terpenuhi (premis), maka sistem akan menghasilkan kesimpulan berupa diagnosis penyakit tertentu (konklusi). Representasi *rule-based* ini menjadi dasar bagi mesin inferensi sistem pakar untuk memproses data masukan dari pengguna dan melakukan penalaran secara otomatis menggunakan metode *forward chaining*.

Dalam implementasinya, aturan-aturan dalam sistem disusun menggunakan struktur logika *AND* dan *OR* yang merepresentasikan:

- *AND* antar grup: Seluruh kelompok gejala dalam satu penyakit harus terpenuhi. Artinya, minimal satu gejala *OR* dalam setiap grup wajib (*AND*) dialami agar diagnosis dapat dilanjutkan ke grup berikutnya.
- *OR* dalam satu grup: Cukup satu dari beberapa gejala dalam satu grup yang dialami, maka grup tersebut dianggap terpenuhi.

Struktur logika *AND-OR* ini memungkinkan sistem melakukan evaluasi diagnosis secara fleksibel namun tetap akurat. Berikut adalah aturan *IF-THEN* untuk diagnosis penyakit gigi dan mulut pada anak yang diimplementasikan dalam sistem:

- Aturan P01 Karies Gigi

  IF Ada bercak kecoklatan/kehitaman di gigi (G01)

  AND Ada lubang di gigi (G02)

  AND (

  Gigi sensitif dan rapuh (G03)

  OR Gigi berbayang abu-abu (G04)

  OR Sering terselip sisa makanan (G05)

)

  THEN Karies Gigi (P01)

- Aturan P02 Pulpitis

  IF Ada bercak kecoklatan/kehitaman di gigi (G01)

  AND Ada lubang di gigi (G02)

  AND Gigi sensitif dan rapuh (G03)

  AND Sakit apabila terkena minuman/makanan dingin atau manis (G06)

  AND (

Gigi berbayang abu-abu (G04)

OR Sering terselip sisa makanan (G05)

OR Gigi berlubang besar (G07)

OR Ada sakit spontan dan terus-menerus (G08)

)

THEN Pulpitis (P02)

- Aturan P03 Abses Gigi

  IF Ada lubang di gigi (G02)

  AND Gusi bengkak/pembesaran (G09)

  AND (

  Gigi sensitif dan rapuh (G03)

  OR Gigi berbayang abu-abu (G04)

  OR Sering terselip sisa makanan (G05)

  OR Sakit apabila terkena minuman/makanan dingin atau manis (G06)

  OR Gigi berlubang besar (G07)

  OR Ada sakit spontan dan terus-menerus (G08)

  OR Gusi keluar nanah (G10)

  OR Terdapat cairan atau rasa asin pada sekitar gigi (G11)

  OR Gusi kemerahan (G12)

)

  THEN Abses Gigi (P03)

- Aturan P04 Gingivitis

  IF Gusi bengkak/pembesaran (G09)

AND Gusi kemerahan (G12)

AND Gusi mudah berdarah (G13)

AND (

Gusi keluar nanah (G10)

OR Terasa sakit di area tersebut (G14)

OR Luka di gusi (G15)

)

THEN Gingivitis (P04)

- Aturan P05 Persistensi Gigi Sulung

  IF Gigi permanen sudah tumbuh di belakang gigi susu (G16)

  AND Gigi terlihat berjejal atau tidak rapi (G17)

  AND (

  Luka di gusi (G15)

  OR Gigi goyang (G18)

  OR Gigi susu masih kuat (G19)

)

  THEN Persistensi Gigi Sulung (P05)

- Aturan P06 Trauma Gigi

  IF Ada riwayat jatuh atau terbentur (G20)

  AND Ada gigi yang patah/retak/lepas (G21)

  AND (

  Gigi sensitif dan rapuh (G03)

  OR Gusi bengkak/pembesaran (G09)

  OR Gusi kemerahan (G12)

OR Terasa sakit di area tersebut (G14)

OR Gigi goyang (G18)

OR Ada luka di area trauma (G22)

)

THEN Trauma Gigi (P06)

- Aturan P07 Fluorosis Dental

  IF Ada bercak putih di gigi (G23)

  AND Pernah tertelan *fluoride* (kandungan dalam pasta gigi/obat kumur) pada saat sikat gigi (G24)

  AND (

  Gigi sensitif dan rapuh (G03)

  OR Mengonsumsi air dengan kadar *fluoride* tinggi (G25)

  OR Gigi kecoklatan (G26)

)

  THEN Fluorosis Dental (P07)

- Aturan P08 Hipoplasia Email

  IF Gigi sensitif dan rapuh (G03)

  AND Ada bercak putih kekuningan di gigi (G27)

  AND (

  Anak memiliki riwayat kekurangan nutrisi saat kehamilan (G28)

  OR Ibu memiliki riwayat konsumsi antibiotik Tetracycline saat hamil (G29)

  OR Anak pernah terbentur pada saat fase pertumbuhan gigi anak (G30)

)

THEN Hipoplasia Email (P08)

- Aturan P09 Stomatitis Aphthosa

  IF Terasa sakit di area tersebut (G14)

  AND Ada ulkus (luka) berwarna putih di area mulut (G31)

  AND (

  Ada riwayat tergigit/terbentur benda tajam di area mulut (G32)

  OR Ada riwayat alergi (makanan, obat, atau bahan tertentu) (G33)

  OR Sedang stress/kelelahan/terkena penyakit infeksi (G34)

)

  THEN Stomatitis Aphthosa (P09)

- Aturan P10 Maloklusi

  IF Gigi terlihat berjejal atau tidak rapi (G17)

  AND (

  Memiliki kebiasan buruk (menghisap jempol atau bernapas lewat mulut) (G35)

  OR Gigitan terbalik (gigi bawah berada di depan gigi atas saat menggigit) (G36)

  OR Rahang atas atau bawah terlalu maju atau mundur (G37)

  OR Gigitan terbuka (ada celah antara gigi atas dan bawah saat mulut tertutup/menggigit) (G38)

)

  THEN Maloklusi (P10)

c. Mesin Inferensi

Mesin inferensi merupakan komponen inti sistem pakar yang bertugas melakukan penalaran untuk menghasilkan diagnosis berdasarkan gejala yang dimasukkan oleh pengguna. Proses inferensi dalam sistem ini menggunakan metode *forward chaining*, di mana sistem menelusuri gejala yang dimasukkan pengguna secara bertahap untuk menemukan diagnosis yang sesuai berdasarkan aturan (*rule-based*) yang telah dibuat pada tahap representasi pengetahuan. Sistem akan terus mengevaluasi jawaban yang diberikan dan mencocokkannya dengan struktur logika *AND-OR* dalam aturan yang ada hingga semua gejala relevan selesai ditanyakan. Pada akhir proses, sistem akan menghasilkan kesimpulan berupa diagnosis penyakit yang sesuai dengan kombinasi gejala yang dialami, atau memberikan informasi bahwa diagnosis tidak dapat ditentukan apabila tidak ada aturan yang terpenuhi.

Untuk meningkatkan efisiensi proses diagnosis, sistem ini menerapkan teknik *pruning* (pemangkasan) yang bertujuan mengurangi jumlah pertanyaan yang diajukan kepada pengguna dengan cara mengeliminasi kandidat penyakit yang tidak relevan secara bertahap. Teknik *pruning* ini diterapkan dalam beberapa strategi:

- *Strict Pruning* (Pemangkasan Ketat)

  Sistem melakukan pemangkasan ketat dengan mengeliminasi penyakit dari daftar kandidat jika pengguna menjawab "Ya" pada gejala yang bukan merupakan bagian dari aturan penyakit tersebut. Misalnya, jika pengguna menjawab "Ya" pada gejala G31 yang hanya terkait dengan P09

(Stomatitis), maka semua penyakit lain seperti P01 (Karies Gigi), P02 (Pulpitis), dan lainnya akan langsung dipangkas dari daftar kandidat.

- *Mandatory Pruning* (Pemangkasan Gejala Wajib)

  Dalam struktur aturan yang menggunakan logika *AND*, terdapat gejala-gejala wajib yang harus dipenuhi untuk suatu penyakit. Sistem akan melakukan pemangkasan terhadap penyakit jika pengguna menjawab "Tidak" pada gejala wajib tersebut. Sebagai contoh, P01 (Karies Gigi) memiliki gejala wajib G01 dan G02. Jika pengguna menjawab "Tidak" pada salah satu gejala wajib ini, maka P01 akan langsung dipangkas dari kandidat, dan sistem tidak perlu menanyakan gejala lainnya yang terkait dengan P01.

- *Optional Pruning* (Pemangkasan Gejala Opsional)

  Untuk aturan dengan logika *OR* (gejala opsional), sistem akan melakukan pemangkasan jika semua gejala opsional dalam satu grup dijawab "Tidak" oleh pengguna. Misalnya, jika suatu penyakit memiliki grup gejala opsional (G04 *OR* G05 *OR* G07) dan pengguna menjawab "Tidak" untuk ketiga gejala tersebut, maka penyakit tersebut akan dipangkas karena tidak ada satupun gejala opsional yang terpenuhi.

- *Real-time Pruning* dengan *Early Stopping*

  Sistem melakukan validasi kandidat penyakit secara *real-time* setelah setiap jawaban yang diberikan pengguna. Jika setelah menjawab suatu gejala tidak ada lagi kandidat penyakit yang tersisa (semua penyakit telah dipangkas), maka sistem akan menghentikan proses diagnosis lebih awal

(*early stopping*) dan memberikan pesan bahwa tidak ada penyakit yang cocok dengan kombinasi gejala yang dipilih.

- *Strategic Pruning* dengan Prioritas Gejala

  Sistem juga menerapkan pemangkasan strategis dengan menggunakan prioritas gejala untuk menangani kasus penyakit dengan gejala yang *overlapping*. Gejala-gejala diberi urutan prioritas berdasarkan kemampuannya dalam membedakan antar penyakit. Sistem akan menanyakan gejala dengan prioritas tinggi terlebih dahulu, yaitu gejala yang dapat secara efektif memisahkan kandidat penyakit. Contohnya pada kasus P01 (Karies Gigi) dan P02 (Pulpitis) yang memiliki gejala awal yang sama (G01, G02, G03). Sistem tidak akan langsung mengkonfirmasi P01 meskipun semua gejala wajibnya terpenuhi, jika P02 masih menjadi kandidat. Dalam hal ini, sistem akan menunda keputusan dan terlebih dahulu menanyakan G06 sebagai gejala pembeda (*discriminating symptom*) yang memiliki prioritas tinggi. Jika pengguna menjawab "Tidak" pada G06, maka P02 akan dipangkas dan sistem mengkonfirmasi P01. Sebaliknya, jika pengguna menjawab "Ya" pada G06, maka P01 akan dipangkas dan sistem melanjutkan dengan gejala konfirmasi untuk P02.

Dengan menerapkan kombinasi teknik *pruning* ini, sistem dapat mengurangi rata-rata jumlah pertanyaan dari maksimal 38 gejala menjadi sekitar 5-10 pertanyaan saja, tergantung pada kombinasi gejala yang dialami pengguna. Hal ini tidak hanya meningkatkan efisiensi waktu diagnosis, tetapi juga meningkatkan pengalaman pengguna dengan menghindari pertanyaan

yang tidak relevan, serta meningkatkan akurasi diagnosis melalui eliminasi kandidat penyakit yang tidak sesuai secara sistematis.

Untuk memberikan gambaran yang lebih jelas mengenai proses *pruning* dalam sistem ini, Tabel 8 berikut menunjukkan ilustrasi eliminasi kandidat penyakit secara bertahap berdasarkan jawaban pengguna terhadap setiap gejala yang ditanyakan.

Tabel 8. Ilustrasi Proses *Pruning*

Pertanyaan	Jawaban	Kandidat Penyakit	Jenis *Pruning*
Awal	-	P01, P02, P03, P04, P05, P06, P07, P08, P09, P10	-
G01	Ya	P01, P02	*Strict Pruning* (P03-P10 dipangkas)
G02	Ya	P01, P02	Tetap
G03	Ya	P01, P02	Tetap
G06	Tidak	P01	*Strategic Pruning* (P02 dipangkas)
HASIL	-	P01 (Karies Gigi)	*Confirmed*

4.1.2 Analisis Kebutuhan Pengguna

Analisis kebutuhan pengguna dilakukan untuk mengidentifikasi fungsi dan fitur yang diperlukan oleh setiap jenis pengguna dalam sistem pakar. Sistem pakar ini dirancang untuk melayani tiga kategori pengguna, yaitu administrator, pakar (dokter gigi), dan klien (pasien/orang tua/pengguna umum), dengan kebutuhan fungsional yang berbeda-beda sesuai peran masing-masing. Rincian kebutuhan fungsional untuk setiap jenis pengguna dapat dilihat pada Tabel 9.

Tabel 9. Analisis Kebutuhan Pengguna

Pengguna	Kebutuhan	
	Kode	**Deskripsi**
Admin	[URS-A-01]	Admin dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid untuk masuk ke *dashboard* admin
	[URS-A-02]	Admin dapat melakukan manajemen data pengguna dalam sistem
	[URS-A-03]	Admin dapat mengelola proses persetujuan untuk akun pakar yang mendaftar dalam sistem
	[URS-A-04]	Admin dapat *logout* dari sistem dengan aman untuk mengakhiri sesi
Pakar	[URS-P-01]	Pakar dapat mendaftarkan diri ke dalam sistem dengan mengisi formulir registrasi dan menunggu persetujuan dari admin
	[URS-P-02]	Pakar dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang telah disetujui untuk masuk ke *dashboard* pakar
	[URS-P-03]	Pakar dapat melakukan manajemen data penyakit dalam sistem
	[URS-P-04]	Pakar dapat melakukan manajemen data gejala dalam sistem
	[URS-P-05]	Pakar dapat melakukan manajemen aturan diagnosis (*knowledge base*) yang menghubungkan gejala-gejala dengan penyakit tertentu
	[URS-P-06]	Pakar dapat melihat dan menganalisis riwayat diagnosis yang telah dilakukan oleh sistem
	[URS-P-07]	Pakar dapat *logout* dari sistem dengan aman untuk mengakhiri sesi
Klien	[URS-K-01]	Klien dapat mendaftarkan diri ke dalam sistem untuk mendapatkan akun dan mengakses fitur-fitur yang memerlukan autentikasi
	[URS-K-02]	Klien dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang telah terdaftar untuk menggunakan fitur-fitur yang memerlukan autentikasi

Pengguna	Kebutuhan	
	Kode	**Deskripsi**
Klien	[URS-K-03]	Klien dapat melihat informasi lengkap mengenai berbagai penyakit yang ada dalam sistem tanpa harus *login* terlebih dahulu
	[URS-K-04]	Klien dapat melakukan proses diagnosis mandiri dengan menjawab pertanyaan-pertanyaan terkait gejala yang dialami untuk mendapatkan hasil diagnosis dari sistem
	[URS-K-05]	Klien yang telah *login* dapat melihat riwayat diagnosis yang pernah dilakukan sebelumnya untuk *tracking*
	[URS-K-06]	Klien dapat menyimpan atau mencetak hasil diagnosis dalam format PDF untuk dokumentasi pribadi
	[URS-K-07]	Klien dapat *logout* dari sistem dengan aman untuk mengakhiri sesi

4.2 *Design System*

Tahap *design system* merupakan tahap perancangan arsitektur dan antarmuka sistem pakar berdasarkan hasil analisis kebutuhan yang telah dilakukan pada tahap sebelumnya. Perancangan ini mencakup dua aspek utama, yaitu perancangan dokumentasi sistem menggunakan model UML (*Unified Modeling Language*) untuk memvisualisasikan struktur dan alur kerja sistem, serta perancangan *mockup* sistem untuk memberikan gambaran antarmuka pengguna.

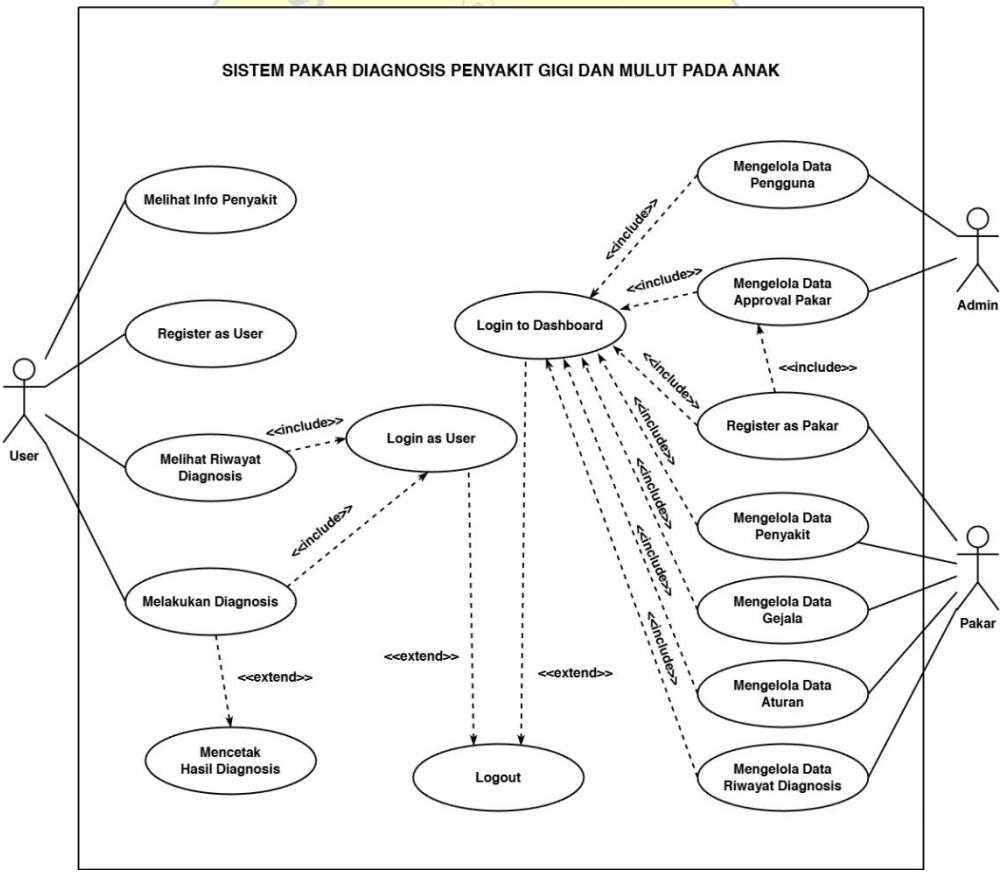
4.2.1 Perancangan Dokumentasi Sistem

Perancangan dokumentasi sistem dilakukan menggunakan model UML (*Unified Modeling Language*) untuk memvisualisasikan arsitektur dan alur kerja sistem pakar secara terstruktur. Dokumentasi ini mencakup tiga jenis

diagram utama yang masing-masing memiliki fungsi spesifik dalam menggambarkan aspek-aspek sistem, yaitu:

a. *Use Case Diagram*

Use case diagram digunakan untuk menggambarkan interaksi antara pengguna (*actor*) dengan sistem serta fungsi-fungsi utama yang dapat dilakukan oleh setiap jenis pengguna. Diagram ini menunjukkan peran dari tiga kategori pengguna, yaitu administrator, pakar, dan klien, beserta fitur-fitur yang dapat diakses oleh masing-masing pengguna sesuai dengan hak aksesnya. *Use case diagram* sistem pakar diagnosis penyakit gigi dan mulut pada anak dapat dilihat pada Gambar 16.

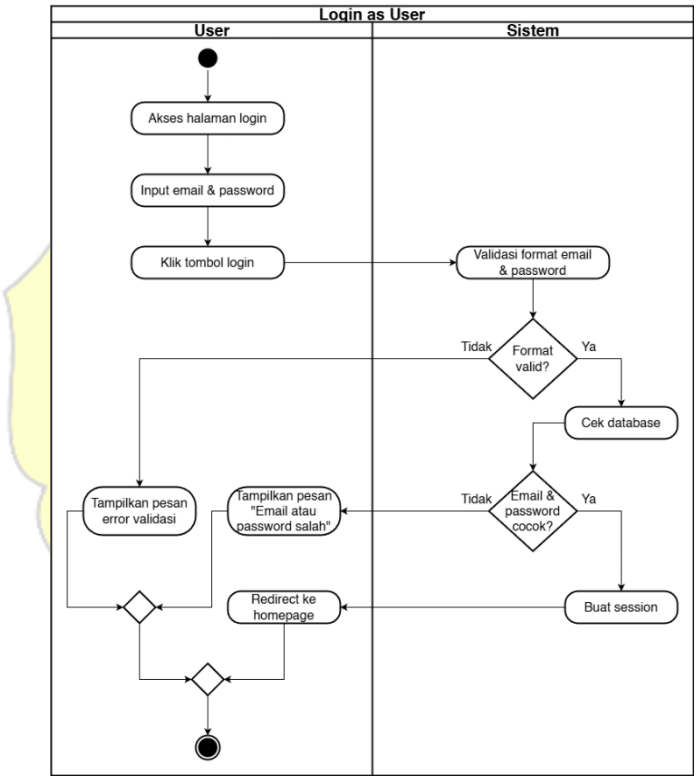


Gambar 16. *Use Case Diagram* Sistem Pakar Gigi dan Mulut Anak

b. *Activity Diagram*

Activity diagram digunakan untuk menggambarkan alur proses atau aktivitas yang terjadi dalam sistem secara sekuensial, mulai dari awal hingga akhir suatu proses. Diagram ini menunjukkan langkah-langkah yang dilakukan oleh pengguna dan respons sistem dalam menjalankan fungsi-fungsi tertentu seperti:

- *Login as User*

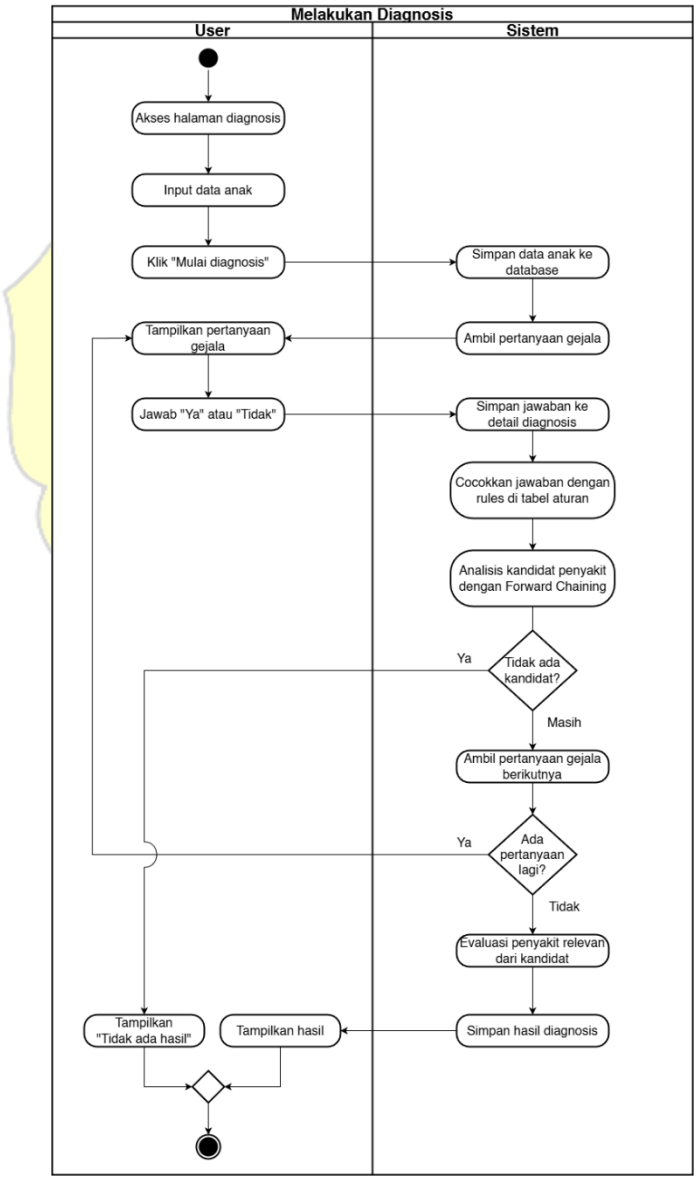


Gambar 17. *Activity Diagram Login as User*

Berdasarkan Gambar 17, alur aktivitas *login* melibatkan interaksi antara pengguna (*User*) dan Sistem. Proses dimulai ketika pengguna mengakses halaman *login*, memasukkan email dan *password*, lalu menekan tombol *login*. Aksi tersebut memicu sistem untuk memvalidasi format data, jika format tidak sesuai, sistem akan langsung menampilkan pesan *error* validasi

kepada pengguna. Sebaliknya, jika format valid, sistem akan memeriksa kecocokan data tersebut di dalam *database*. Apabila email dan *password* terverifikasi cocok, sistem akan membuat sesi (*session*) dan mengarahkan pengguna ke halaman beranda (*homepage*), namun jika data tidak cocok, sistem akan menampilkan pesan peringatan bahwa email atau *password* salah.

- Melakukan Diagnosis

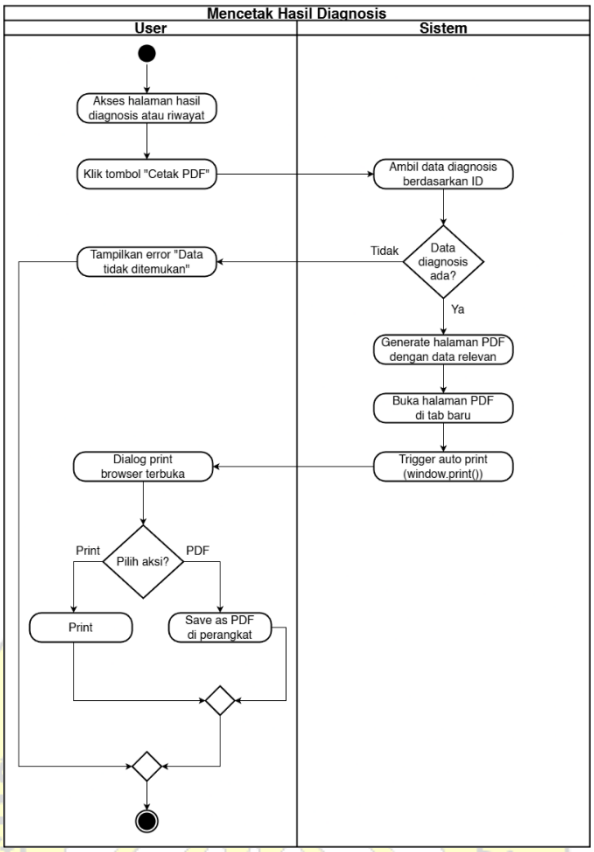


Gambar 18. *Activity Diagram* Melakukan Diagnosis

Berdasarkan Gambar 18, alur proses dimulai ketika *user* mengakses halaman diagnosis, memasukkan data anak, dan mengklik "Mulai diagnosis". Sistem kemudian menyimpan data anak ke *database* dan mengambil pertanyaan gejala untuk ditampilkan kepada *user*. *User* menjawab setiap pertanyaan dengan "Ya" atau "Tidak", dan sistem menyimpan jawaban tersebut ke detail diagnosis. Proses tanya jawab ini berlangsung berulang hingga semua gejala ditanyakan.

Setelah semua pertanyaan dijawab, sistem mencocokkan jawaban dengan *rules* di tabel aturan dan melakukan analisis kandidat penyakit menggunakan metode *Forward Chaining*. Jika tidak ditemukan kandidat penyakit, sistem menampilkan "Tidak ada hasil" dan proses berakhir. Namun jika ditemukan kandidat, sistem akan melanjutkan dengan pertanyaan gejala tambahan yang lebih spesifik. Proses ini berlanjut hingga tidak ada pertanyaan lagi, kemudian sistem melakukan evaluasi penyakit yang paling relevan dari kandidat yang ada, menyimpan hasil diagnosis ke *database*, dan menampilkan hasil diagnosis kepada *user*.

- Mencetak Hasil Diagnosis



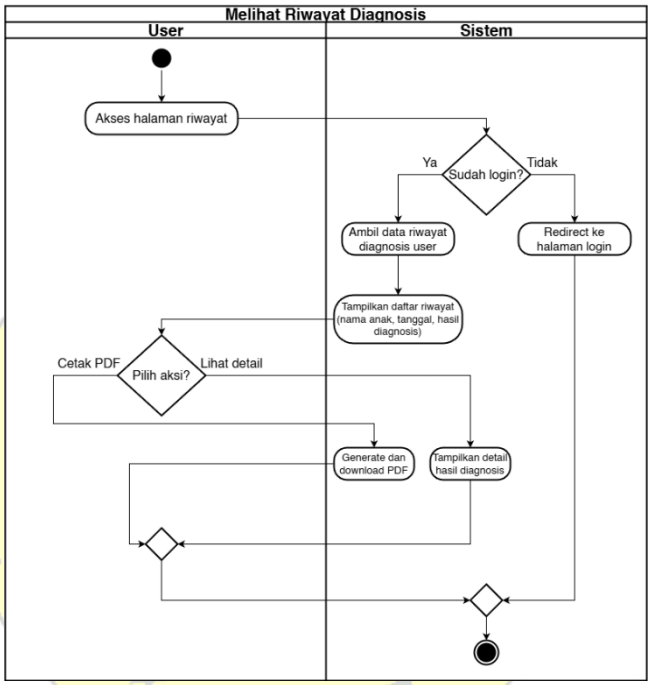
Gambar 19. *Activity Diagram* Mencetak Hasil Diagnosis

Berdasarkan Gambar 19, alur proses dimulai ketika *user* mengakses halaman hasil diagnosis atau riwayat dan mengklik tombol "Cetak PDF". Sistem kemudian mengambil data diagnosis berdasarkan ID yang dipilih dan melakukan pengecekan apakah data diagnosis tersebut ada atau tidak dalam *database*. Jika data tidak ditemukan, sistem akan menampilkan pesan *error* "Data tidak ditemukan" kepada *user* dan proses berakhir.

Jika data diagnosis ditemukan, sistem akan melanjutkan dengan men-*generate* halaman PDF yang berisi data relevan dari hasil diagnosis tersebut. Sistem kemudian membuka halaman PDF di tab baru *browser* dan secara otomatis memicu dialog *print browser* menggunakan fungsi window.print().

Pada dialog *print* yang terbuka, *user* diberikan pilihan aksi untuk memilih antara "*Print*" untuk langsung mencetak dokumen ke *printer*, atau "PDF" untuk menyimpan file sebagai PDF di perangkat. Setelah *user* memilih salah satu opsi dan menyelesaikan proses *print* atau *save*, maka proses mencetak hasil diagnosis selesai.

- Melihat Riwayat Diagnosis

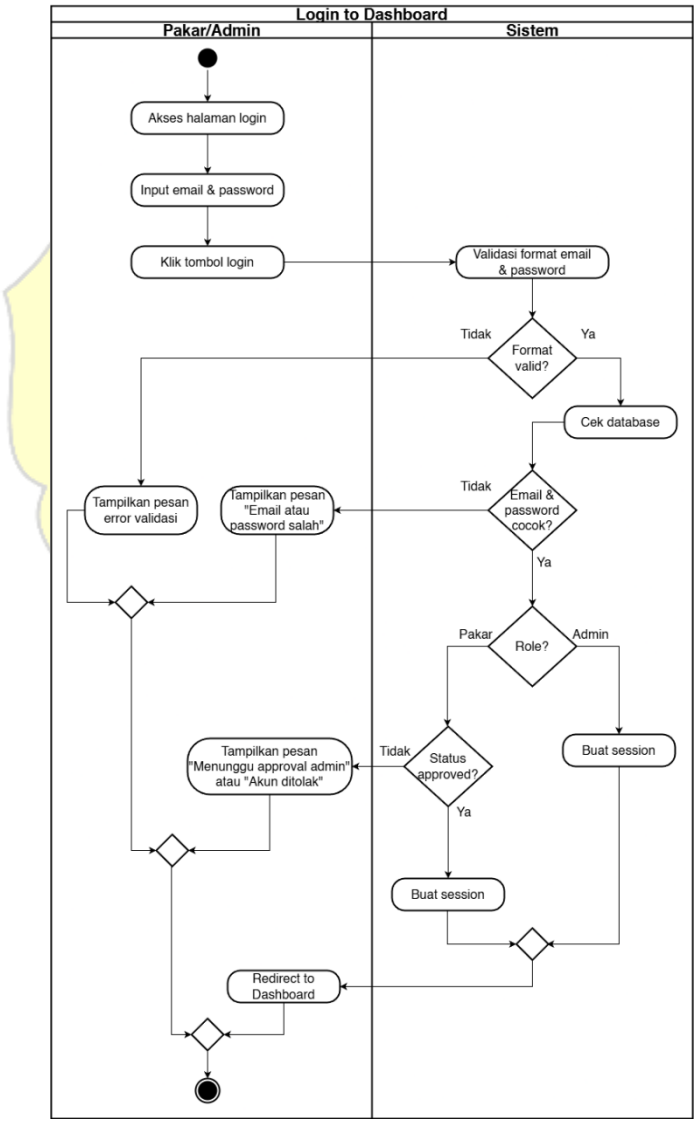


Gambar 20. *Activity Diagram* Melihat Riwayat Diagnosis

Berdasarkan Gambar 20, alur proses dimulai ketika *user* mengakses halaman riwayat dan sistem melakukan pengecekan apakah *user* sudah *login* atau belum. Jika *user* belum *login*, sistem akan melakukan *redirect* ke halaman *login* dan proses berakhir, namun jika *user* sudah *login*, sistem akan mengambil data riwayat diagnosis *user* dari *database* dan menampilkan daftar riwayat yang berisi informasi nama anak, tanggal, dan hasil diagnosis dalam bentuk *list*. Setelah daftar riwayat ditampilkan, *user*

dapat memilih aksi yang ingin dilakukan, yaitu "Cetak PDF" untuk men-*generate* dan mengunduh file PDF dari riwayat diagnosis yang dipilih, atau "Lihat detail" untuk menampilkan detail lengkap dari hasil diagnosis yang dipilih termasuk informasi gejala-gejala yang telah dijawab dan hasil diagnosis secara mendetail, kemudian setelah *user* selesai melakukan salah satu aksi tersebut, proses melihat riwayat diagnosis selesai.

- *Login to Dashboard*

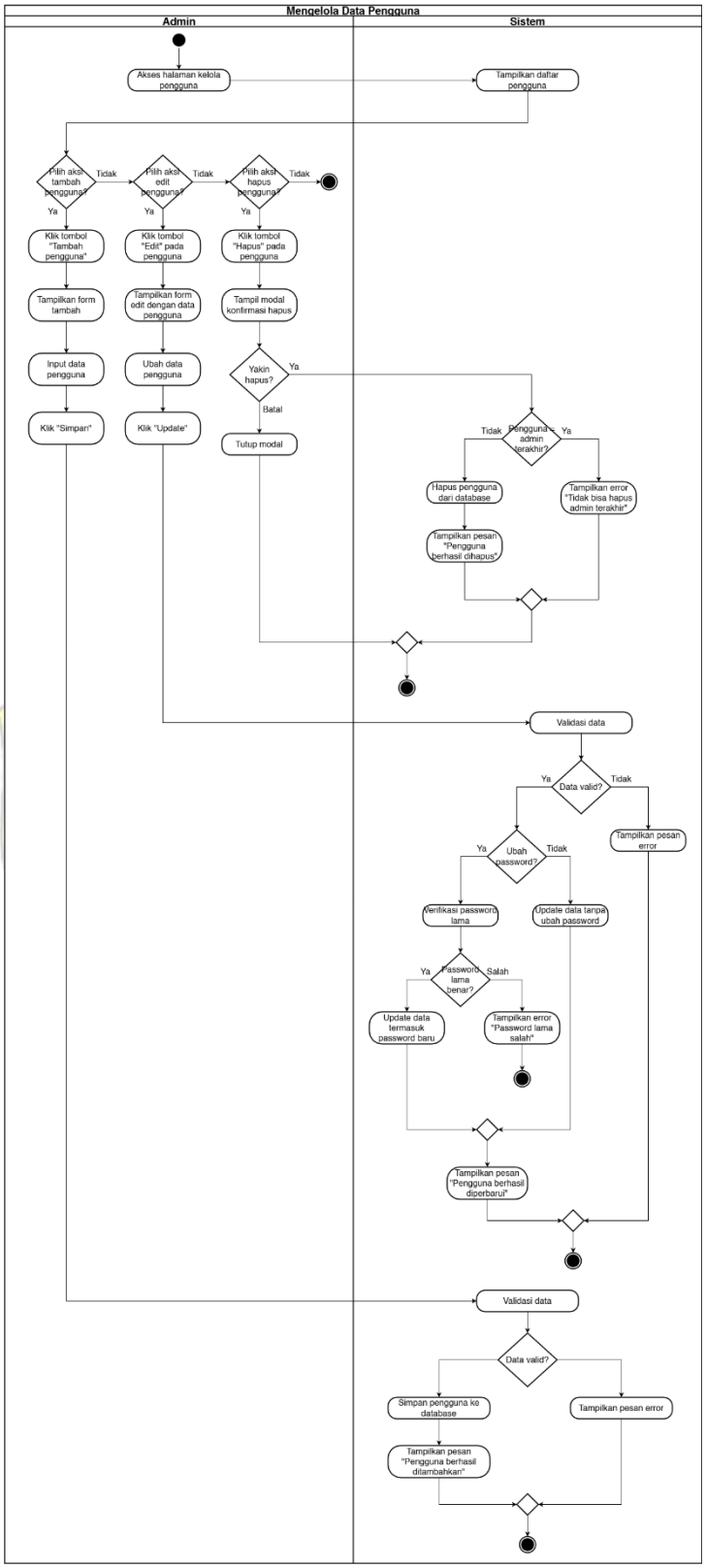


Gambar 21. *Activity Diagram Login to Dashboard*

Berdasarkan Gambar 21, alur proses dimulai ketika pakar atau admin mengakses halaman *login*, memasukkan email dan *password*, kemudian mengklik tombol *login*. Sistem melakukan validasi format email dan *password*, jika format tidak valid maka sistem menampilkan pesan *error* validasi dan proses berakhir, namun jika format valid maka sistem melanjutkan dengan mengecek *database* untuk mencocokkan email dan *password* yang dimasukkan. Jika email atau *password* tidak cocok, sistem menampilkan pesan "Email atau *password* salah" dan proses berakhir, namun jika cocok maka sistem mengecek *role user* tersebut apakah sebagai pakar atau admin.

Jika *role* adalah admin, sistem langsung membuat *session* untuk *user* dan melakukan *redirect* ke dashboard. Namun jika *role* adalah pakar, sistem melakukan pengecekan tambahan terhadap status *approved* dari akun pakar tersebut. Jika status pakar belum *approved*, sistem menampilkan pesan "Menunggu *approval* admin" atau "Akun ditolak" dan proses berakhir, namun jika status sudah *approved* maka sistem membuat *session* untuk *user* dan melakukan *redirect* ke *dashboard*, sehingga proses *login* selesai dan *user* dapat mengakses *dashboard* sesuai dengan *role* masing-masing.

- Mengelola Data Pengguna



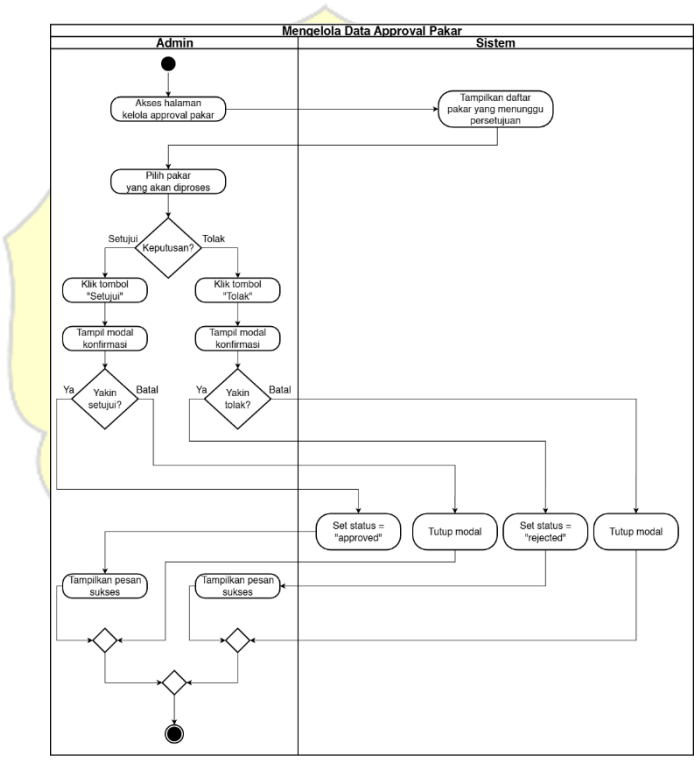
Gambar 22. *Activity Diagram* Mengelola Data Pengguna

Berdasarkan Gambar 22, alur proses dimulai ketika admin mengakses halaman kelola pengguna dan sistem menampilkan daftar pengguna yang ada. Admin kemudian dapat memilih tiga aksi utama yaitu "Tambah pengguna", "Edit pengguna", atau "Hapus pengguna". Jika admin memilih tambah pengguna, sistem menampilkan form tambah, admin mengisi data pengguna baru dan mengklik "Simpan", kemudian sistem melakukan validasi data. Jika admin memilih edit pengguna, sistem menampilkan form edit dengan data pengguna yang dipilih, admin mengubah data dan mengklik "*Update*", kemudian sistem juga melakukan validasi data. Jika admin memilih hapus pengguna, sistem menampilkan modal konfirmasi hapus, dan jika admin yakin dengan memilih "Ya" maka sistem akan mengecek apakah pengguna tersebut adalah admin terakhir atau bukan.

Pada proses hapus, jika pengguna yang akan dihapus adalah admin terakhir dalam sistem, maka sistem menampilkan *error* "Tidak bisa hapus admin terakhir" dan proses berakhir, namun jika bukan admin terakhir maka sistem menghapus pengguna dari *database* dan menampilkan pesan "Pengguna berhasil dihapus". Untuk proses tambah dan edit, setelah sistem melakukan validasi data, jika data tidak valid maka sistem menampilkan pesan *error* dan proses berakhir, namun jika data valid maka sistem melanjutkan dengan mengecek apakah admin ingin mengubah *password* atau tidak. Jika admin mengubah *password*, sistem melakukan verifikasi *password* lama, jika *password* lama salah maka sistem menampilkan *error* "*Password* lama salah" dan proses berakhir, namun jika benar maka sistem melakukan verifikasi apakah *password* baru sama dengan *password* lama,

jika sama maka sistem menampilkan *error* "*Password* lama salah", namun jika berbeda maka sistem meng-*update* data termasuk *password* baru. Jika admin tidak mengubah *password*, sistem langsung meng-*update* data tanpa *password* baru. Setelah proses tambah atau edit selesai, sistem menyimpan pengguna ke *database* dan menampilkan pesan "Pengguna berhasil disimpan" atau "Pengguna berhasil diperbarui", kemudian proses pengelolaan data pengguna selesai.

- Mengelola Data *Approval* Pakar

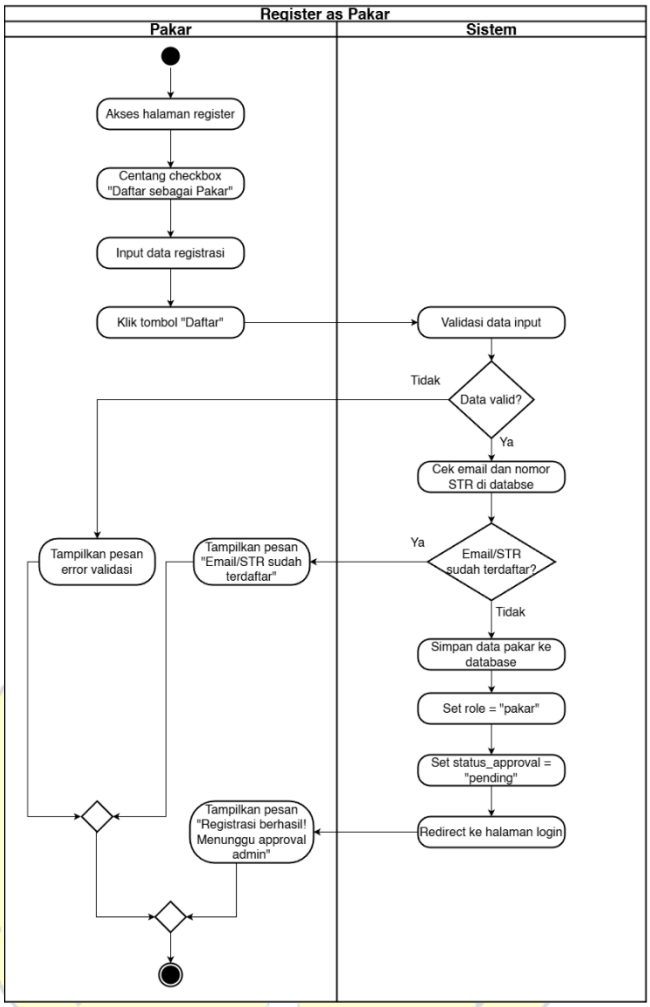


Gambar 23. *Activity Diagram* Mengelola Data *Approval* Pakar

Berdasarkan Gambar 23, alur proses dimulai ketika admin mengakses halaman kelola *approval* pakar dan sistem menampilkan daftar pakar yang menunggu persetujuan. Admin kemudian memilih pakar yang akan diproses dan menentukan keputusan apakah akan menyetujui atau menolak pendaftaran pakar tersebut. Jika admin memutuskan untuk menyetujui,

admin mengklik tombol "Setujui" dan sistem menampilkan modal konfirmasi, jika admin yakin dengan keputusannya maka sistem akan mengubah status pakar menjadi "*approved*" dan menutup modal, kemudian menampilkan pesan sukses kepada admin, namun jika admin membatalkan konfirmasi maka modal akan ditutup dan proses kembali ke daftar pakar. Jika admin memutuskan untuk menolak, admin mengklik tombol "Tolak" dan sistem menampilkan modal konfirmasi, jika admin yakin dengan keputusannya maka sistem akan mengubah status pakar menjadi "*rejected*" dan menutup modal, kemudian menampilkan pesan sukses kepada admin, namun jika admin membatalkan konfirmasi maka modal akan ditutup dan proses kembali ke daftar pakar. Setelah proses *approval* atau penolakan selesai dilakukan dan pesan sukses ditampilkan, proses pengelolaan data *approval* pakar selesai.

- *Register as Pakar*

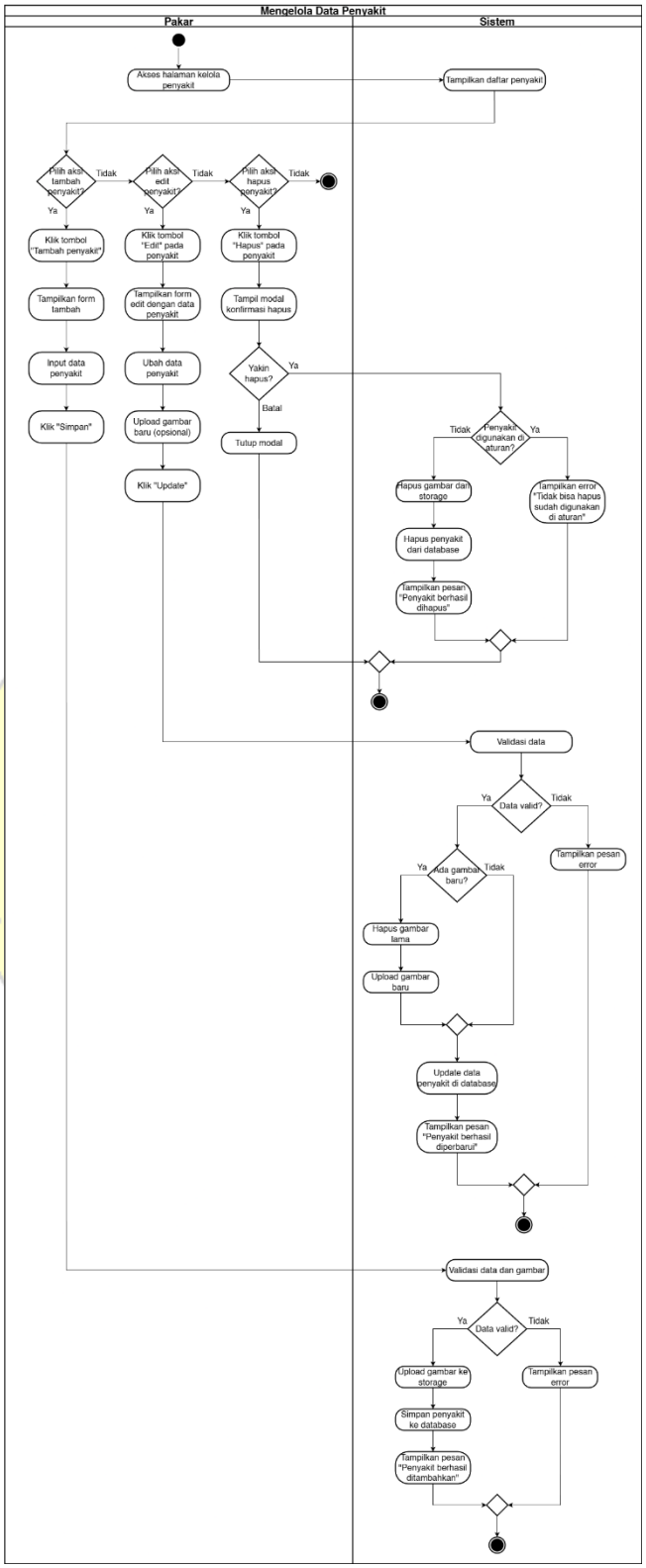


Gambar 24. *Activity Diagram Register as Pakar*

Berdasarkan Gambar 24, alur proses dimulai ketika calon pakar mengakses halaman *register*, mencentang *checkbox* "Daftar sebagai Pakar", memasukkan data registrasi yang diperlukan, dan mengklik tombol "Daftar". Sistem kemudian melakukan validasi terhadap data *input* yang diberikan, jika data tidak valid maka sistem menampilkan pesan *error* validasi dan proses berakhir, namun jika data valid maka sistem melanjutkan dengan mengecek email dan nomor STR (Surat Tanda Registrasi) di *database* untuk memastikan keunikan data tersebut. Jika

email atau nomor STR sudah terdaftar sebelumnya, sistem menampilkan pesan "Email/STR sudah terdaftar" dan proses berakhir, namun jika email dan nomor STR belum terdaftar maka sistem menyimpan data pakar ke *database*, mengatur *role* sebagai "pakar", dan mengatur status_approval sebagai "*pending*" yang menandakan akun pakar tersebut menunggu persetujuan dari admin. Setelah semua data tersimpan, sistem melakukan *redirect* ke halaman *login* dan menampilkan pesan "Registrasi berhasil! Menunggu *approval* admin" kepada calon pakar, sehingga proses registrasi sebagai pakar selesai dan calon pakar harus menunggu persetujuan dari admin sebelum dapat menggunakan sistem.

- Mengelola Data Penyakit



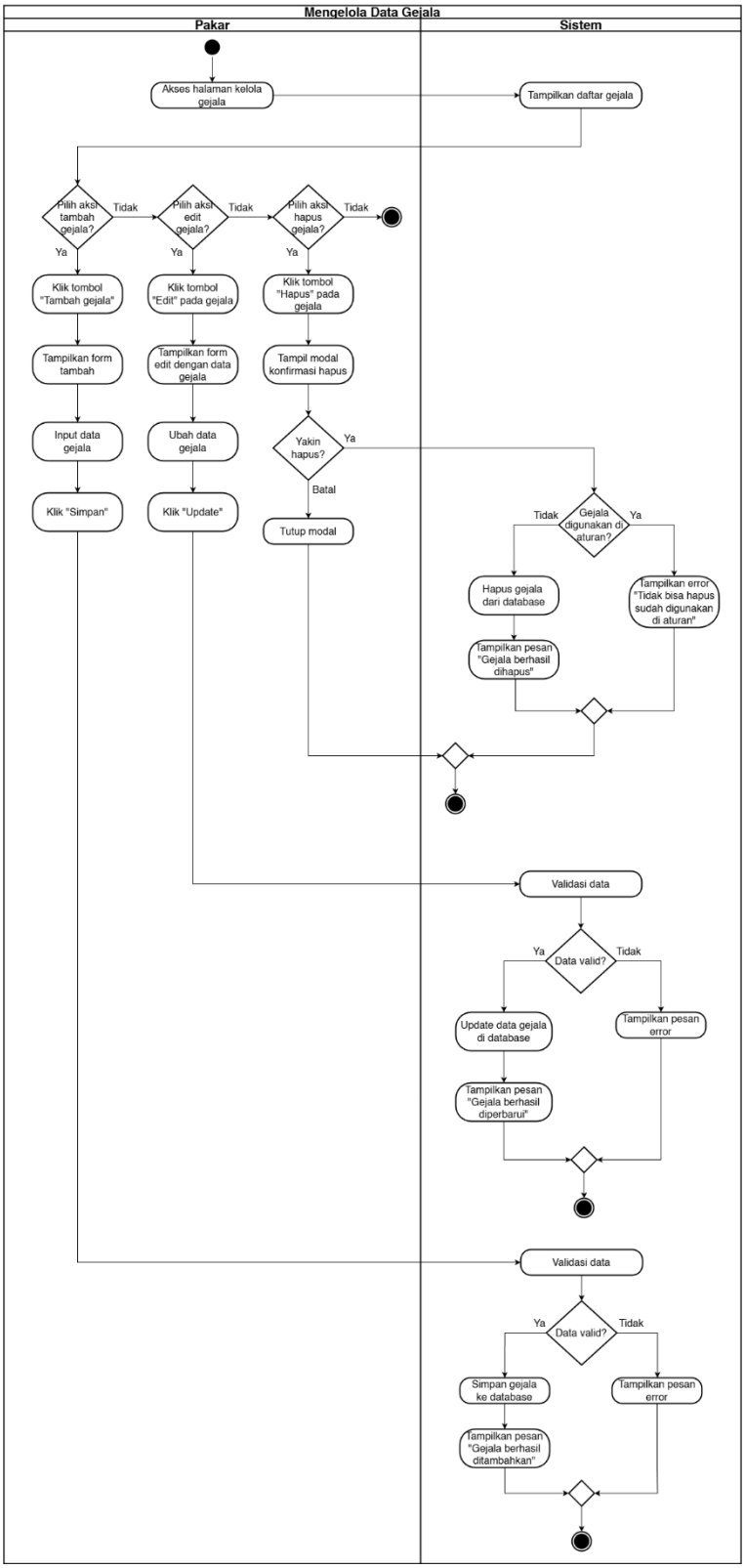
Gambar 25. *Activity Diagram* Mengelola Data Penyakit

Berdasarkan Gambar 25, alur proses dimulai ketika pakar mengakses halaman kelola penyakit dan sistem menampilkan daftar penyakit yang ada. Pakar kemudian dapat memilih tiga aksi utama yaitu "Tambah penyakit", "Edit penyakit", atau "Hapus penyakit". Jika pakar memilih tambah penyakit, sistem menampilkan form tambah, pakar mengisi data penyakit dan mengklik "Simpan", kemudian sistem melakukan validasi data dan gambar. Jika pakar memilih edit penyakit, sistem menampilkan form edit dengan data penyakit yang dipilih, pakar mengubah data penyakit dan dapat mengunggah gambar baru jika diperlukan, kemudian mengklik "*Update*", dan sistem melakukan validasi data dan gambar. Jika pakar memilih hapus penyakit, sistem menampilkan modal konfirmasi hapus, dan jika pakar yakin dengan memilih "Ya" maka sistem akan mengecek apakah penyakit tersebut digunakan dalam aturan diagnosis atau tidak.

Pada proses hapus, jika penyakit digunakan dalam aturan, sistem menampilkan *error* "Tidak bisa hapus sudah digunakan di aturan" dan proses berakhir, namun jika tidak digunakan maka sistem menghapus gambar dari *storage*, menghapus penyakit dari *database*, dan menampilkan pesan "Penyakit berhasil dihapus". Untuk proses tambah dan edit, setelah sistem melakukan validasi data, jika data tidak valid maka sistem menampilkan pesan *error* dan proses berakhir, namun jika data valid maka sistem melanjutkan dengan mengecek apakah ada gambar baru yang diunggah. Jika ada gambar baru, sistem menghapus gambar lama dari *storage* dan meng-*upload* gambar baru, kemudian meng-*update* data penyakit di *database* dan menampilkan pesan "Penyakit berhasil

diperbarui". Jika tidak ada gambar baru, sistem langsung meng-*update* data penyakit di *database* dan menampilkan pesan "Penyakit berhasil diperbarui". Untuk proses tambah penyakit baru, setelah validasi berhasil, sistem melakukan validasi data dan gambar sekali lagi, jika valid maka sistem meng-*upload* gambar ke *storage*, menyimpan data penyakit ke *database*, dan menampilkan pesan "Penyakit berhasil ditambahkan", kemudian proses pengelolaan data penyakit selesai.

- Mengelola Data Gejala

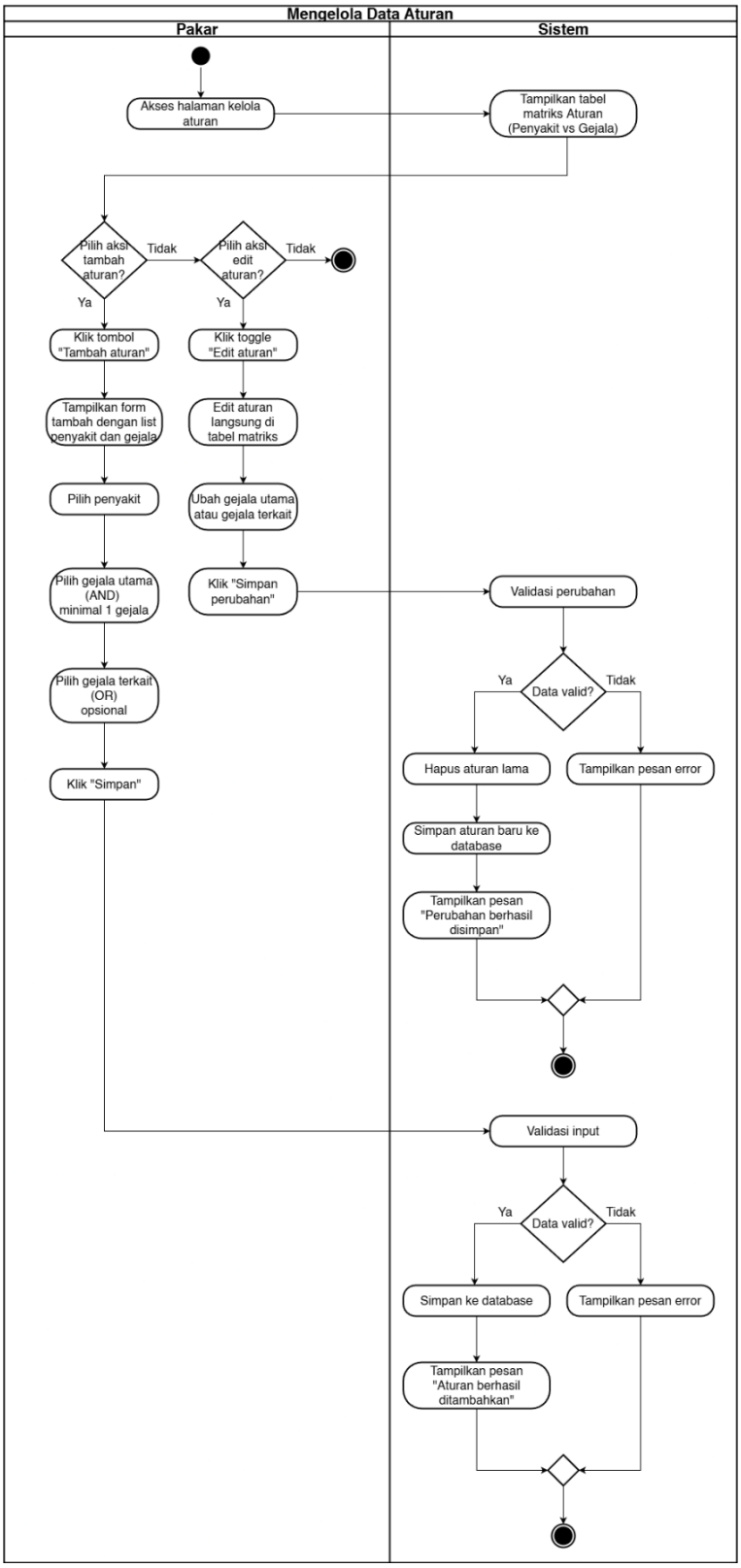


Gambar 26. *Activity Diagram* Mengelola Data Gejala

Berdasarkan Gambar 26, alur proses dimulai ketika pakar mengakses halaman kelola gejala dan sistem menampilkan daftar gejala yang ada. Pakar kemudian dapat memilih tiga aksi utama yaitu "Tambah gejala", "Edit gejala", atau "Hapus gejala". Jika pakar memilih tambah gejala, sistem menampilkan form tambah, pakar mengisi data gejala dan mengklik "Simpan", kemudian sistem melakukan validasi data. Jika pakar memilih edit gejala, sistem menampilkan form edit dengan data gejala yang dipilih, pakar mengubah data gejala dan mengklik "*Update*", kemudian sistem juga melakukan validasi data. Jika pakar memilih hapus gejala, sistem menampilkan modal konfirmasi hapus, dan jika pakar yakin dengan memilih "Ya" maka sistem akan mengecek apakah gejala tersebut digunakan dalam aturan diagnosis atau tidak.

Pada proses hapus, jika gejala digunakan dalam aturan, sistem menampilkan *error* "Tidak bisa hapus sudah digunakan di aturan" dan proses berakhir, namun jika tidak digunakan maka sistem menghapus gejala dari *database* dan menampilkan pesan "Gejala berhasil dihapus". Untuk proses edit, setelah sistem melakukan validasi data, jika data tidak valid maka sistem menampilkan pesan *error* dan proses berakhir, namun jika data valid maka sistem meng-*update* data gejala di *database* dan menampilkan pesan "Gejala berhasil diperbarui", kemudian proses selesai. Untuk proses tambah gejala baru, setelah validasi data berhasil, sistem menyimpan gejala ke *database* dan menampilkan pesan "Gejala berhasil ditambahkan", kemudian proses pengelolaan data gejala selesai.

- Mengelola Data Aturan

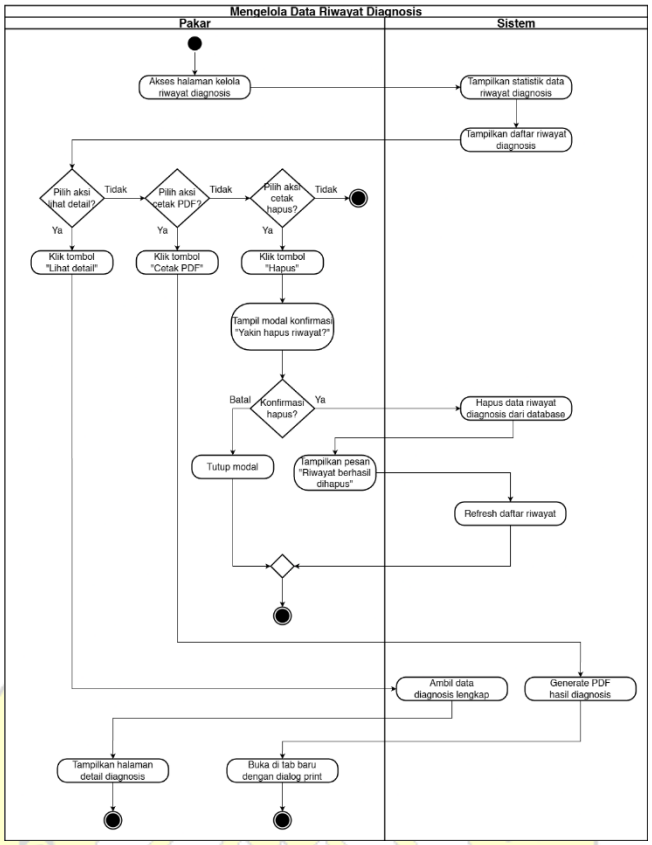


Gambar 27. *Activity Diagram* Mengelola Data Aturan

Berdasarkan Gambar 27, alur proses dimulai ketika pakar mengakses halaman kelola aturan dan sistem menampilkan tabel matriks aturan yang menunjukkan hubungan antara penyakit dan gejala. Pakar kemudian dapat memilih dua aksi yaitu "Tambah aturan" atau "Edit aturan". Jika pakar memilih tambah aturan, sistem menampilkan form tambah dengan *list* penyakit dan gejala yang tersedia, pakar memilih penyakit target, memilih gejala utama dengan operator *AND* yang minimal 1 gejala, memilih gejala terkait dengan operator *OR* yang bersifat opsional, dan mengklik "Simpan", kemudian sistem melakukan validasi *input*. Jika pakar memilih edit aturan, sistem mengaktifkan *toggle* "Edit aturan" dan pakar dapat mengedit aturan langsung di tabel matriks dengan mengubah gejala utama atau gejala terkait, kemudian mengklik "Simpan perubahan", dan sistem melakukan validasi perubahan.

Untuk proses edit, setelah sistem melakukan validasi perubahan, jika data tidak valid maka sistem menampilkan pesan *error* dan proses berakhir, namun jika data valid maka sistem menghapus aturan lama, menyimpan aturan baru ke *database*, dan menampilkan pesan "Perubahan berhasil disimpan", kemudian proses selesai. Untuk proses tambah aturan baru, setelah validasi *input* berhasil, jika data tidak valid maka sistem menampilkan pesan *error* dan proses berakhir, namun jika data valid maka sistem menyimpan aturan ke *database* dan menampilkan pesan "Aturan berhasil ditambahkan", kemudian proses pengelolaan data aturan selesai.

- Mengelola Data Riwayat Diagnosis

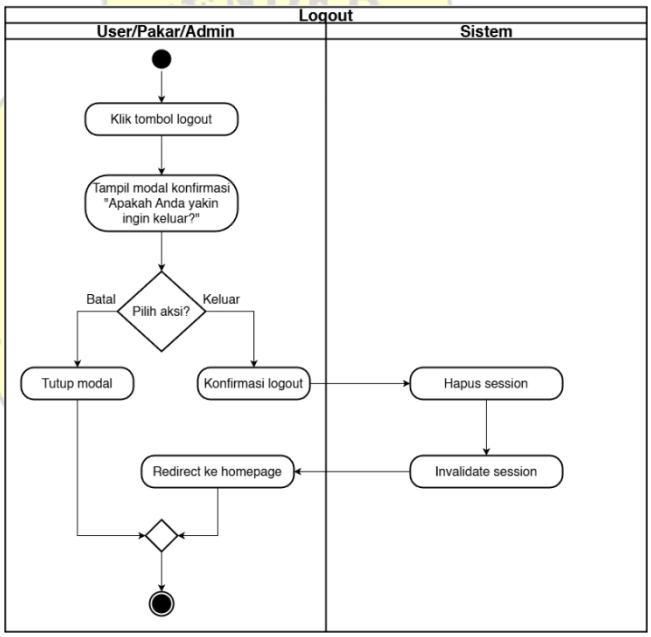


Gambar 28. *Activity Diagram* Mengelola Data Riwayat Diagnosis

Berdasarkan Gambar 28, alur proses dimulai ketika pakar mengakses halaman kelola riwayat diagnosis dan sistem menampilkan statistik data riwayat diagnosis serta menampilkan daftar riwayat diagnosis yang ada. Pakar kemudian dapat memilih tiga aksi utama yaitu "Lihat detail", "Cetak PDF", atau "Hapus". Jika pakar memilih lihat detail dengan mengklik tombol "Lihat detail", sistem akan mengambil data diagnosis lengkap dan menampilkan halaman detail diagnosis, kemudian proses selesai. Jika pakar memilih cetak PDF dengan mengklik tombol "Cetak PDF", sistem akan men-*generate* PDF hasil diagnosis dan membuka file tersebut di tab baru dengan dialog *print*, kemudian proses selesai.

Jika pakar memilih hapus dengan mengklik tombol "Hapus", sistem menampilkan modal konfirmasi dengan pertanyaan "Yakin hapus riwayat?", dan jika pakar membatalkan dengan memilih "Batal" maka sistem menutup modal dan proses berakhir, namun jika pakar mengkonfirmasi dengan memilih "Ya" maka sistem menghapus data riwayat diagnosis dari *database*, menampilkan pesan "Riwayat berhasil dihapus", dan melakukan *refresh* pada daftar riwayat untuk memperbarui tampilan data terbaru, kemudian proses pengelolaan data riwayat diagnosis selesai.

- *Logout*

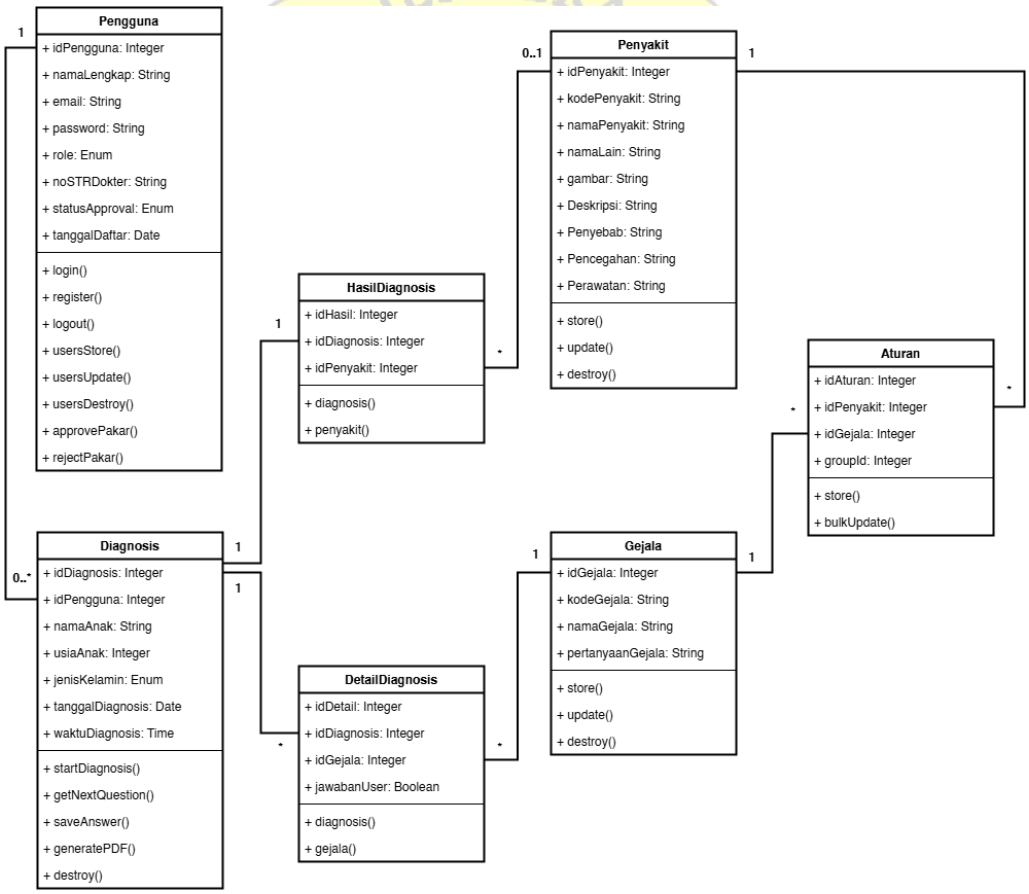


Gambar 29. *Activity Diagram Logout*

Berdasarkan Gambar 29, alur proses dimulai saat *user* mengklik tombol *logout* dan sistem menampilkan konfirmasi "Apakah Anda yakin ingin keluar?". Jika memilih "Batal", modal ditutup dan proses berakhir. Jika memilih "Keluar", sistem menghapus *session*, meng-*invalidate session*, dan *redirect* ke *homepage*.

c. *Class Diagram*

Class diagram digunakan untuk menggambarkan struktur data dalam sistem, termasuk entitas-entitas utama (*class*), atribut yang dimiliki setiap entitas, serta relasi atau hubungan antar entitas. Diagram ini menjadi dasar perancangan basis data dan implementasi struktur program dalam sistem pakar. *Class diagram* menggambarkan bagaimana data penyakit, gejala, aturan diagnosis, dan data pengguna diorganisasikan dan saling berelasi dalam sistem. *Class diagram* sistem pakar diagnosis penyakit gigi dan mulut pada anak dapat dilihat pada Gambar 30.

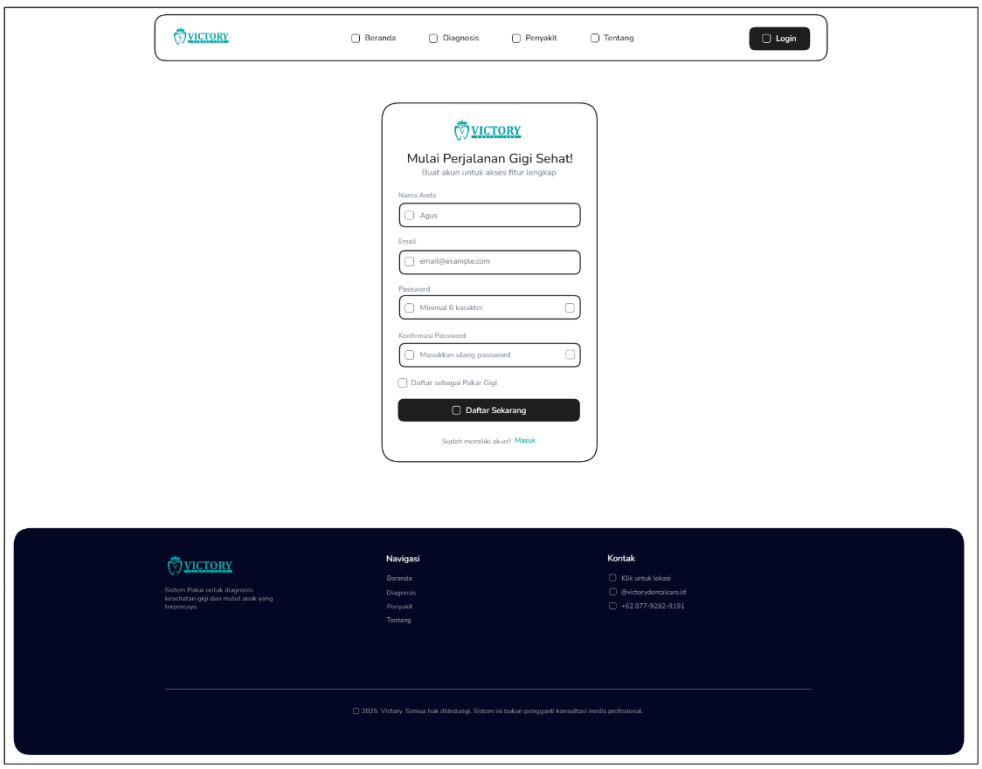


Gambar 30. *Class Diagram* Sistem Pakar Gigi dan Mulut Anak

4.2.2 Perancangan *Mockup* Sistem

Pada bagian ini, disajikan rancangan antarmuka (*mockup*) dari sistem pakar yang akan dikembangkan. *Mockup* ini memberikan gambaran visual mengenai desain dan alur interaksi pengguna dengan sistem. Berikut adalah tampilan-tampilan yang dirancang:

a. Halaman Daftar

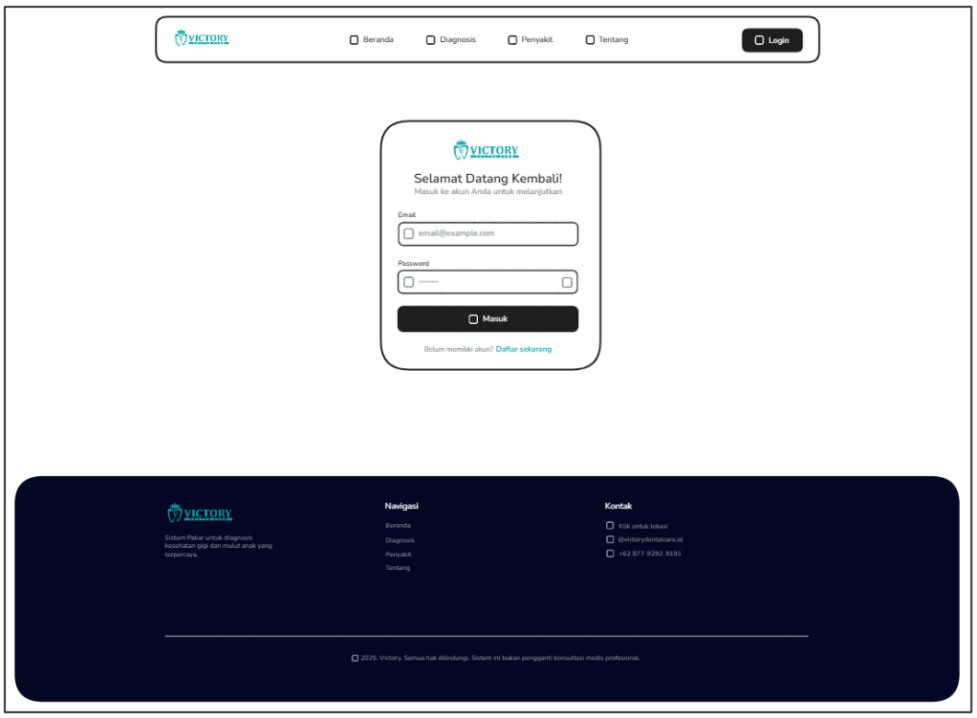


Gambar 31. *Mockup* Halaman Daftar

Sebagaimana terlihat pada Gambar 31, halaman daftar dirancang untuk memfasilitasi pengguna baru dalam pembuatan akun agar dapat mengakses fitur sistem secara penuh. Pada antarmuka ini, pengguna diminta untuk melengkapi data diri yang meliputi nama, email, serta kata sandi dan konfirmasinya. Terdapat juga opsi *checkbox* "Daftar sebagai Pakar Gigi", yang memungkinkan pendaftaran spesifik bagi tenaga medis yang ingin berkontribusi sebagai pakar

dalam sistem, yang nantinya akan memasukkan data nomor STR (Surat Tanda Registrasi) sebelum akhirnya menekan tombol "Daftar Sekarang" untuk memproses registrasi.

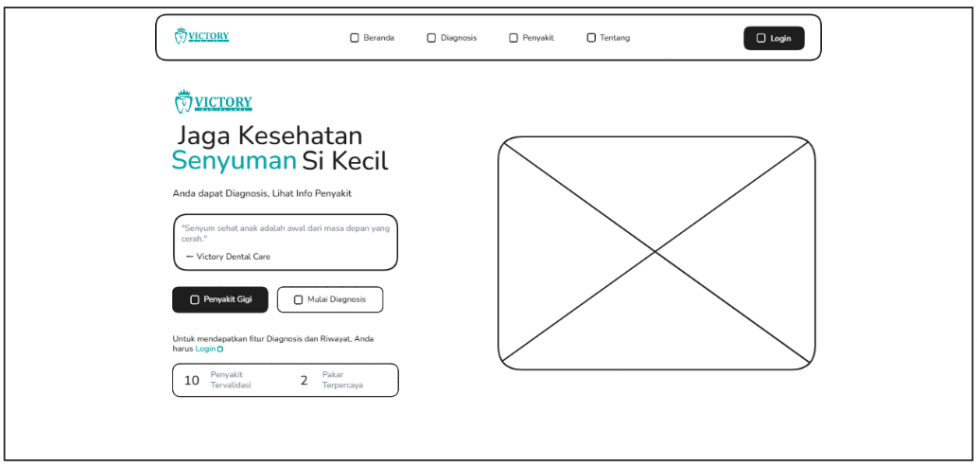
b. Halaman Masuk



Gambar 32. *Mockup* Halaman Masuk

Sebagaimana terlihat pada Gambar 32, halaman masuk berfungsi sebagai gerbang autentikasi bagi pengguna untuk mengakses fitur-fitur yang lebih spesifik, seperti diagnosis, riwayat diagnosis atau *dashboard* admin/pakar. Pengguna yang sudah terdaftar dapat masuk ke sistem dengan memasukkan kredensial berupa email dan *password*. Bagi pengunjung baru, halaman ini juga menyediakan tautan "Daftar Sekarang" yang akan mengarahkan mereka ke formulir pendaftaran untuk membuat akun baru.

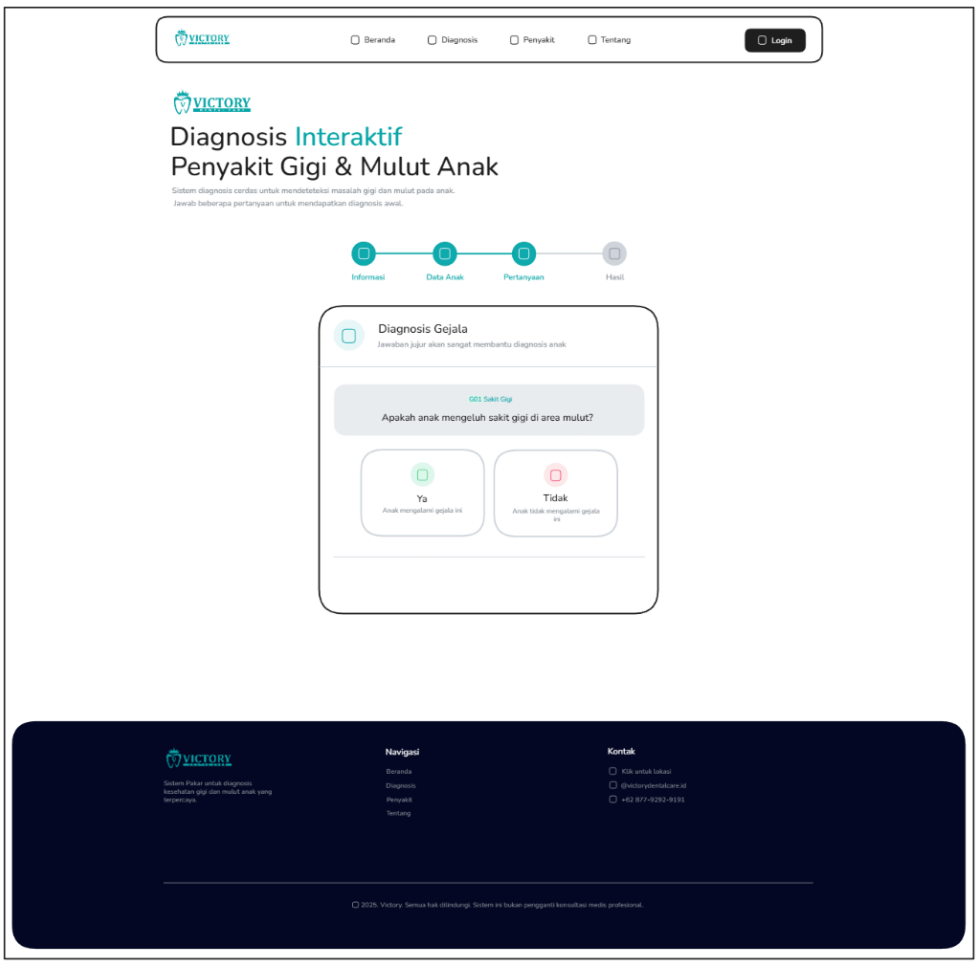
c. Halaman Beranda



Gambar 33. *Mockup* Halaman Beranda

Sebagaimana terlihat pada Gambar 33, halaman beranda merupakan halaman utama yang dirancang untuk menyambut pengguna dan memberikan akses langsung ke fitur-fitur inti sistem. Pada halaman ini, pengguna disajikan dengan judul utama "Jaga Kesehatan Senyuman Si Kecil" serta dua tombol aksi utama: "Mulai Diagnosis" untuk memulai proses diagnosis sistem pakar dan "Penyakit Gigi" untuk mengakses informasi mengenai berbagai penyakit. Selain itu, terdapat menu navigasi untuk halaman lain, informasi singkat untuk membangun kepercayaan pengguna seperti jumlah penyakit yang tervalidasi dan pakar terpercaya, serta tombol *login* untuk mengakses fitur yang lebih lengkap.

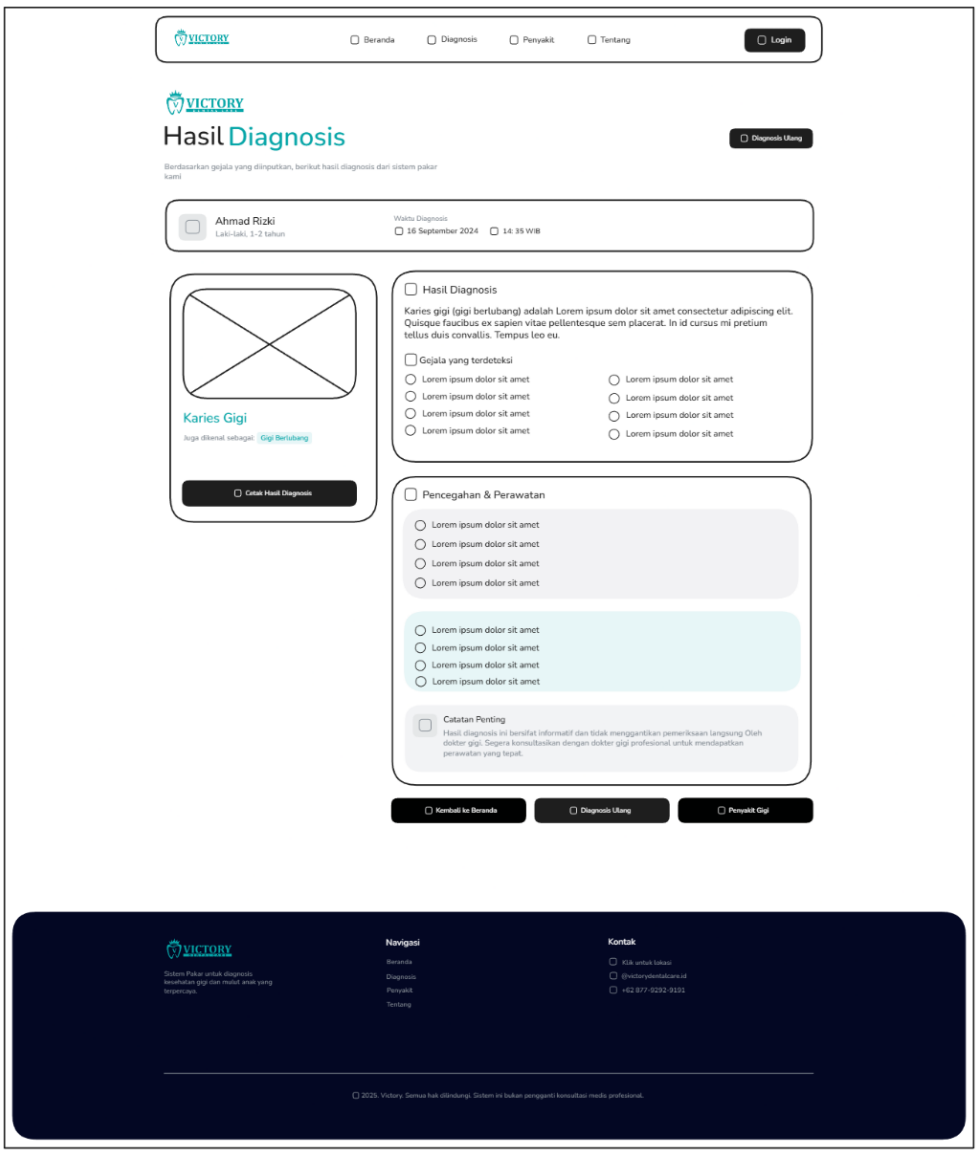
d. Halaman Diagnosis



Gambar 34. *Mockup* Halaman Diagnosis

Sebagaimana terlihat pada Gambar 34, halaman diagnosis merupakan antarmuka inti tempat pengguna berinteraksi dengan sistem pakar untuk melakukan diagnosis. Pada halaman ini, sistem akan secara berurutan menampilkan satu per satu pertanyaan terkait gejala penyakit gigi dan mulut. Pengguna diminta untuk memberikan jawaban "Ya" atau "Tidak" sesuai dengan kondisi yang dialami anak. Setiap jawaban akan menjadi fakta yang digunakan oleh sistem untuk melakukan proses inferensi menggunakan metode *forward chaining* hingga dapat ditarik sebuah kesimpulan diagnosis penyakit.

e. Halaman Hasil Diagnosis

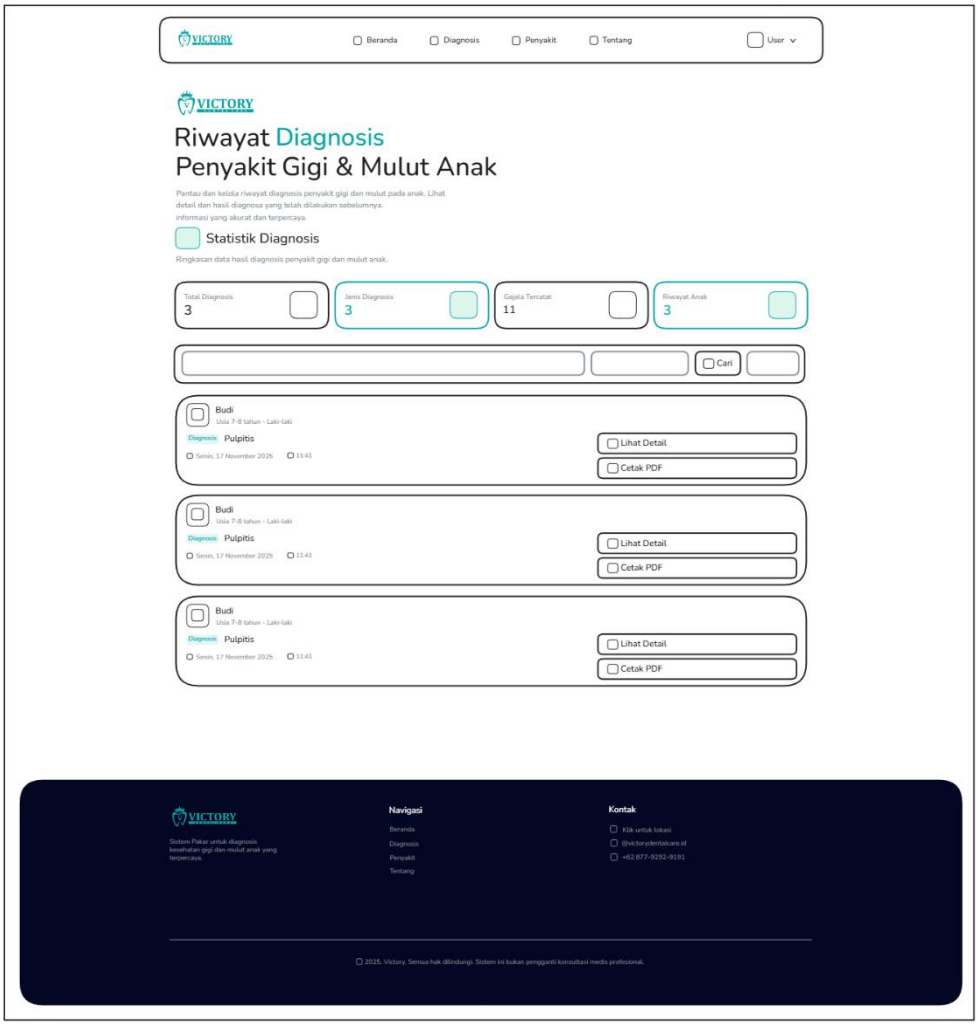


Gambar 35. *Mockup* Halaman Hasil Diagnosis

Sebagaimana terlihat pada Gambar 35, halaman hasil diagnosis adalah tahap akhir dari alur diagnosis yang menampilkan kesimpulan diagnosis berdasarkan jawaban pengguna. Halaman ini menyajikan informasi, meliputi nama penyakit yang teridentifikasi, deskripsi penyakit, dan daftar gejala yang terdeteksi selama proses diagnosis. Untuk melengkapi informasi, sistem juga memberikan rekomendasi langkah pencegahan dan perawatan awal, serta menyediakan fitur untuk mencetak hasil. Terdapat pula catatan penting yang menegaskan bahwa

hasil dari sistem pakar ini bersifat informatif dan tidak menggantikan pemeriksaan langsung oleh tenaga medis profesional.

f. Halaman Riwayat Diagnosis



Gambar 36. *Mockup* Halaman Riwayat Diagnosis

Sebagaimana terlihat pada Gambar 36, halaman riwayat diagnosis dirancang untuk membantu pengguna memantau dan mengelola rekam jejak kesehatan gigi anak secara terorganisir. Halaman ini menyajikan ringkasan statistik diagnosis di bagian atas, diikuti oleh daftar riwayat diagnosis yang memuat informasi esensial seperti nama anak, tanggal pemeriksaan, dan hasil penyakit yang teridentifikasi. Untuk memudahkan akses data, disediakan fitur

pencarian, serta setiap entri riwayat dilengkapi dengan tombol aksi "Lihat Detail" untuk meninjau kembali analisis sistem dan tombol "Cetak PDF" untuk keperluan dokumentasi laporan.

g. Halaman Info Penyakit



Gambar 37. *Mockup* Halaman Info Penyakit

Sebagaimana terlihat pada Gambar 37, halaman info penyakit berfungsi sebagai pusat informasi edukatif yang menyediakan katalog lengkap mengenai berbagai jenis penyakit gigi dan mulut pada anak. Halaman ini menampilkan

daftar penyakit dalam format kartu (*grid*) yang informatif, mencakup ilustrasi, nama penyakit, serta deskripsi singkat. Dilengkapi dengan fitur pencarian di bagian atas, serta menyediakan tombol "Detail" pada setiap kartu untuk mengakses penjelasan lebih mendalam mengenai deskripsi lanjut dan penanganannya.

h. Halaman Detail Info Penyakit

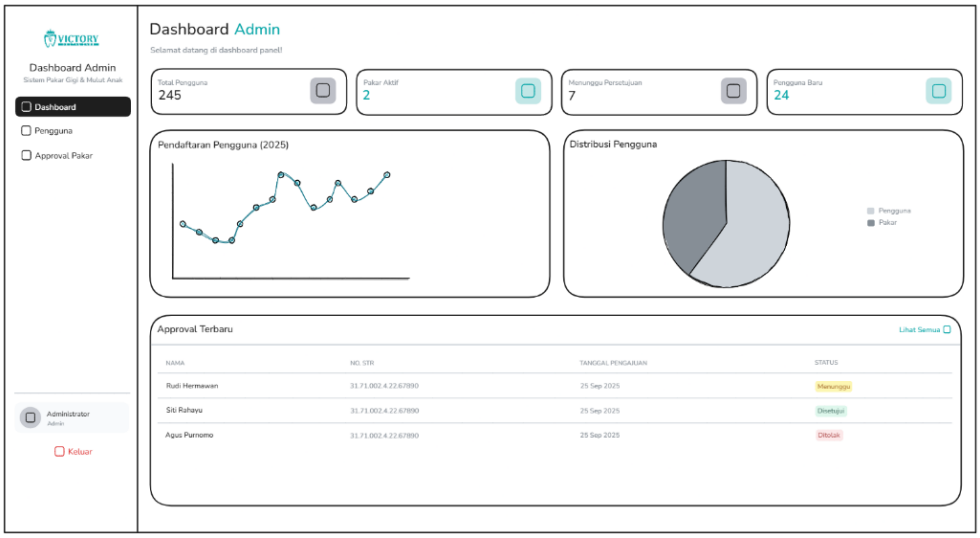


Gambar 38. *Mockup* Halaman Detail Info Penyakit

Sebagaimana terlihat pada Gambar 38, halaman detail info penyakit merupakan antarmuka edukatif yang menyajikan ulasan mengenai satu jenis

penyakit tertentu yang dipilih pengguna. Halaman ini memuat informasi terstruktur mulai dari definisi penyakit, faktor penyebab, hingga solusi penanganan yang mencakup langkah pencegahan dan perawatan. Halaman ini juga dilengkapi dengan tombol pintas "Diagnosis Sekarang" yang mengarahkan langsung ke proses diagnosis sistem pakar.

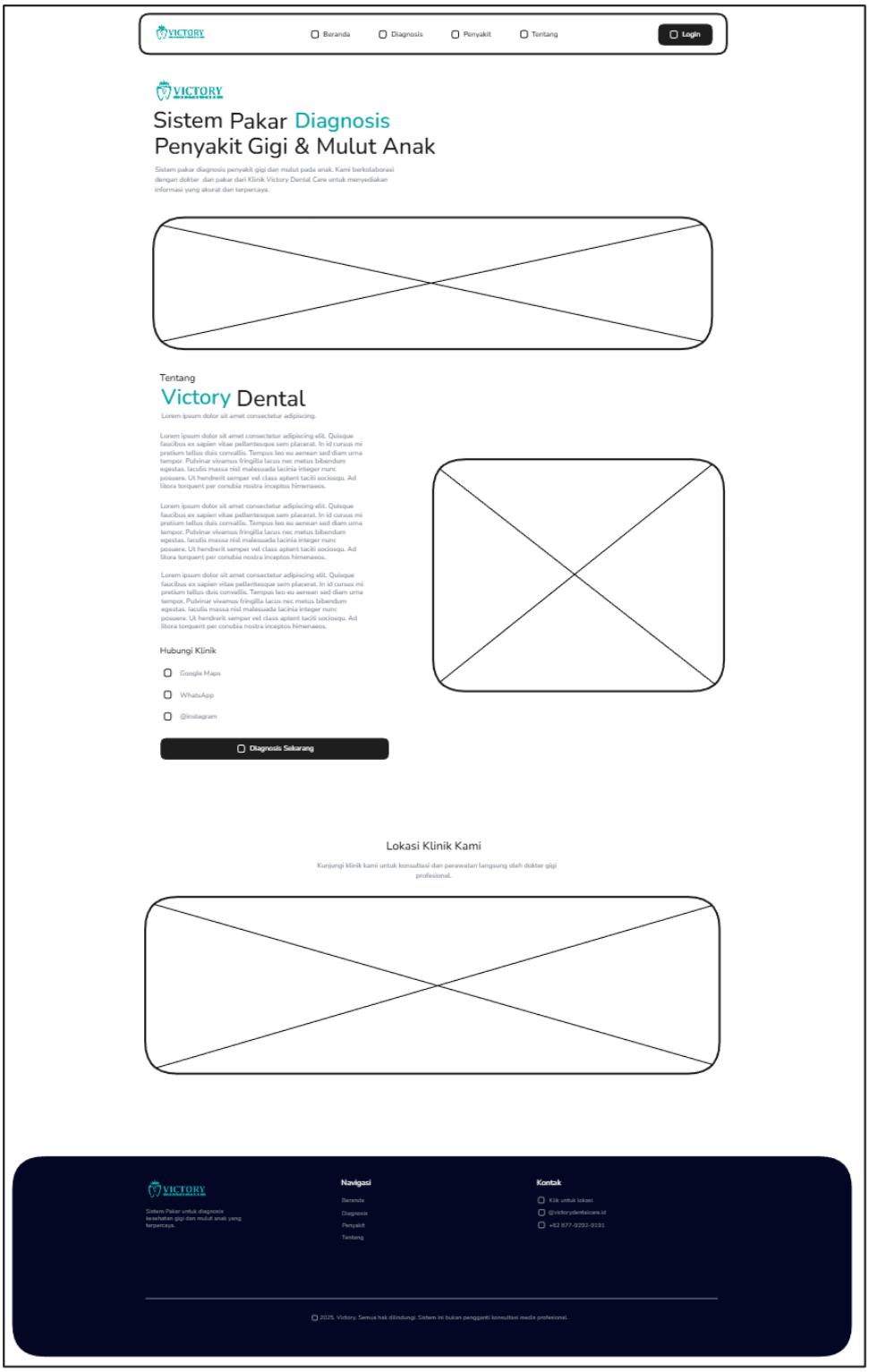
i. *Dashboard* Admin



Gambar 39. *Mockup Dashboard* Admin

Sebagaimana terlihat pada Gambar 39*, dashboard* admin adalah antarmuka khusus yang dirancang bagi pengguna dengan hak akses istimewa (administrator maupun pakar) untuk mengelola dan memantau aktivitas sistem. Halaman ini menyajikan ringkasan data statistik penting secara visual, seperti total pengguna dan tren pendaftaran, yang berguna untuk pemantauan. Selain itu, *dashboard* ini memiliki fitur seperti *panel* "*Approval* Terbaru", yang secara spesifik memungkinkan pengguna dengan *role* administrator untuk melakukan validasi dan persetujuan pendaftaran pakar baru.

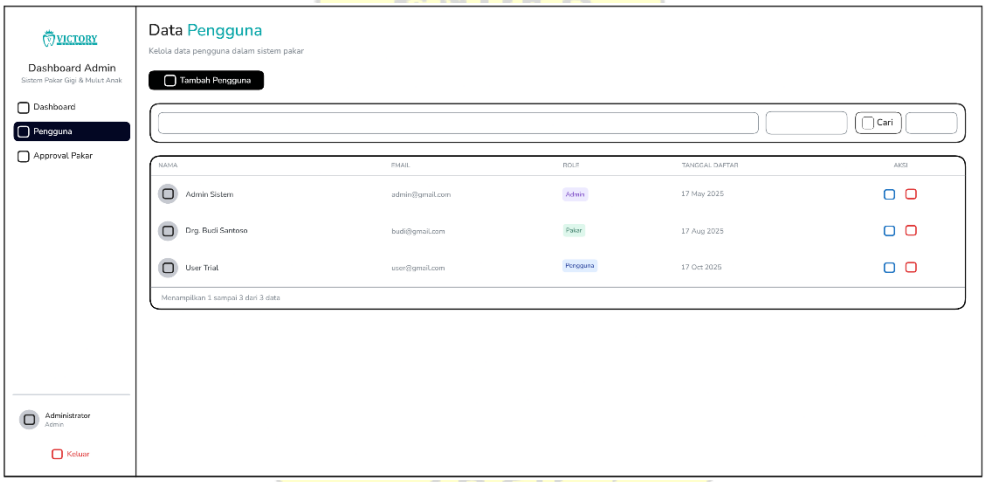
j. Halaman Tentang



Gambar 40. *Mockup* Halaman Tentang

Sebagaimana terlihat pada Gambar 40, halaman tentang berfungsi sebagai sarana informasi profil instansi mitra penelitian, yaitu Klinik Victory Dental Care. Halaman ini memuat deskripsi singkat mengenai latar belakang klinik, saluran kontak resmi yang dapat dihubungi, serta integrasi peta digital yang menampilkan lokasi fisik klinik untuk memudahkan kunjungan. Meskipun berfokus pada informasi profil, antarmuka ini tetap dirancang strategis dengan menyertakan tombol akses cepat "Diagnosis Sekarang" untuk memudahkan pengguna beralih langsung ke fitur utama sistem.

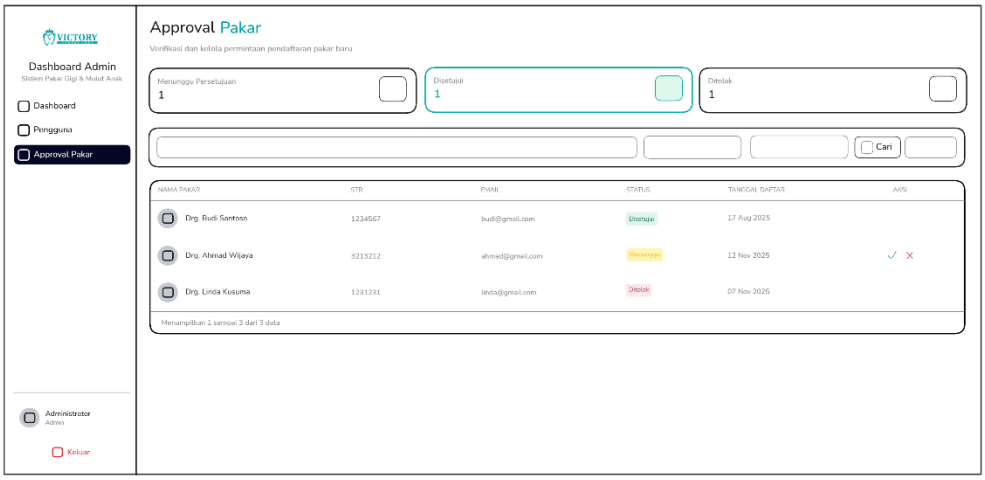
k. Halaman Kelola Pengguna



Gambar 41. *Mockup* Halaman Kelola Pengguna

Sebagaimana terlihat pada Gambar 41, halaman kelola pengguna merupakan fitur administratif yang memberikan kontrol penuh kepada admin untuk mengelola basis data pengguna sistem. Halaman ini menyajikan daftar terstruktur dari seluruh akun yang terdaftar, menampilkan informasi seperti nama, alamat email, serta peran pengguna (*role*). Untuk mendukung efisiensi manajemen, halaman ini dilengkapi dengan fitur pencarian data, tombol "Tambah Pengguna" untuk registrasi manual, serta opsi aksi pada setiap entri untuk keperluan penyuntingan atau penghapusan akun.

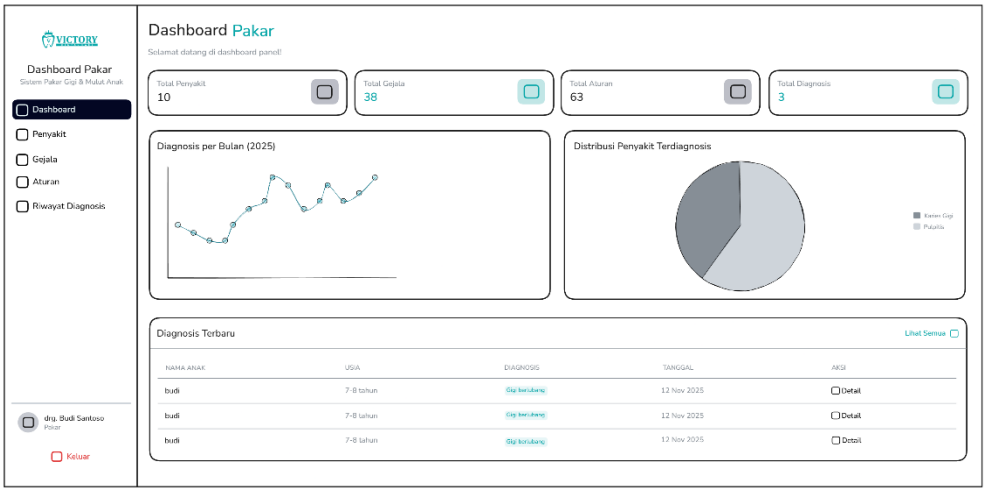
l. Halaman Kelola *Approval* Pakar



Gambar 42. *Mockup* Halaman Kelola *Approval* Pakar

Sebagaimana terlihat pada Gambar 42, halaman kelola *approval* pakar merupakan antarmuka yang memfasilitasi administrator dalam memvalidasi kredibilitas calon pakar yang mendaftar ke dalam sistem. Halaman ini menyajikan ringkasan statistik status pengajuan serta daftar detail pemohon yang mencakup informasi identitas medis penting, khususnya nomor STR (Surat Tanda Registrasi). Fitur utama pada halaman ini terletak pada kolom aksi, di mana administrator memiliki otoritas penuh untuk meninjau dan memberikan keputusan, baik menyetujui (ceklis) maupun menolak (silang) guna memastikan bahwa hanya tenaga medis terverifikasi yang mendapatkan hak akses sebagai pakar.

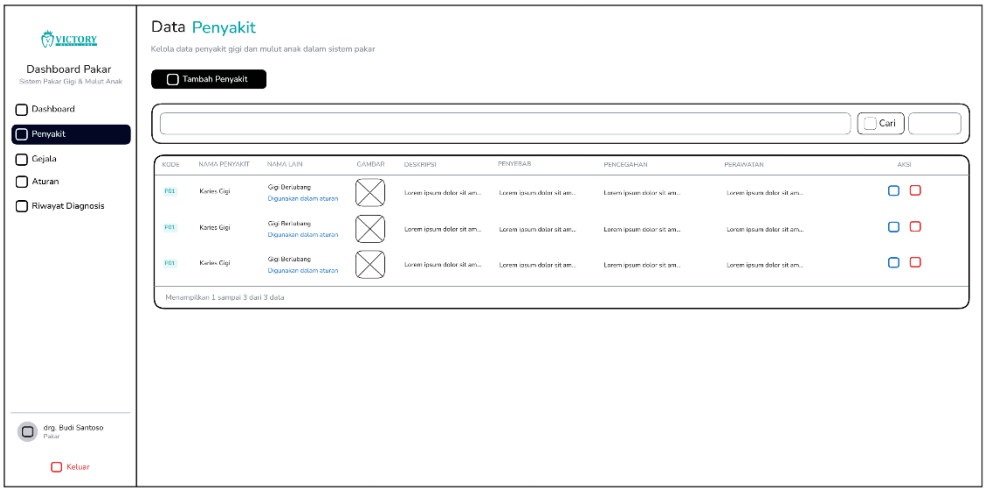
m. *Dashboard* Pakar



Gambar 43. *Mockup Dashboard* Pakar

Sebagaimana terlihat pada Gambar 43, halaman *dashboard* pakar berfungsi sebagai pusat kendali operasional bagi tenaga medis dalam memantau dan mengelola basis pengetahuan sistem. Halaman ini menyajikan ringkasan berupa kartu statistik jumlah penyakit, gejala, dan aturan yang aktif, dilengkapi dengan visualisasi grafik tren diagnosis bulanan serta distribusi penyakit untuk analisis tren kesehatan. Selain fitur *monitoring*, *dashboard* ini juga menampilkan tabel "Diagnosis Terbaru" yang memungkinkan pakar untuk meninjau riwayat diagnosis pasien terkini secara langsung.

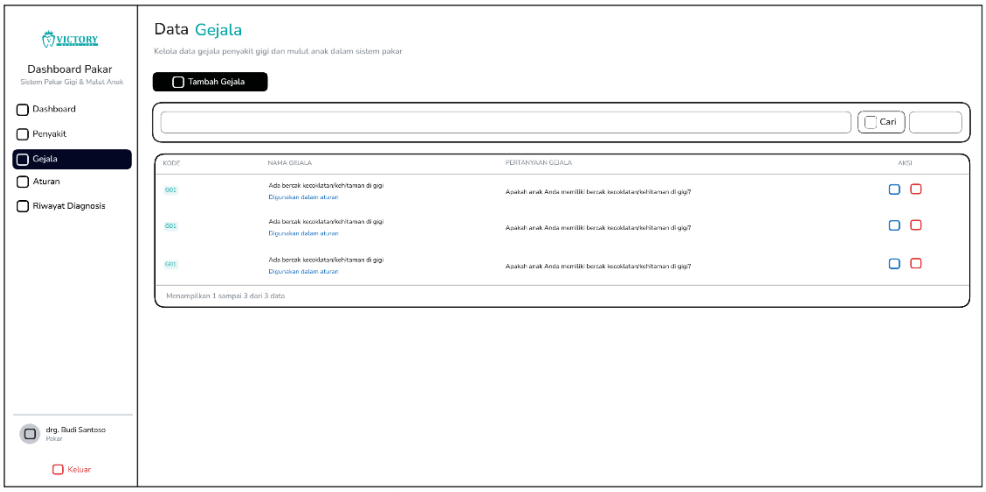
n. Halaman Kelola Penyakit



Gambar 44. *Mockup* Halaman Kelola Penyakit

Sebagaimana terlihat pada Gambar 44, halaman kelola penyakit merupakan antarmuka manajemen basis pengetahuan yang memberikan wewenang kepada pakar untuk mengelola data referensi penyakit dalam sistem. Halaman ini menampilkan tabel data, mencakup kode penyakit, nama, ilustrasi visual, serta informasi detail medis seperti deskripsi, penyebab, pencegahan, dan perawatan. Halaman ini dilengkapi dengan fitur pencarian, tombol "Tambah Penyakit" untuk *input* data baru, serta kolom aksi yang memfasilitasi pakar untuk menyunting atau menghapus data penyakit.

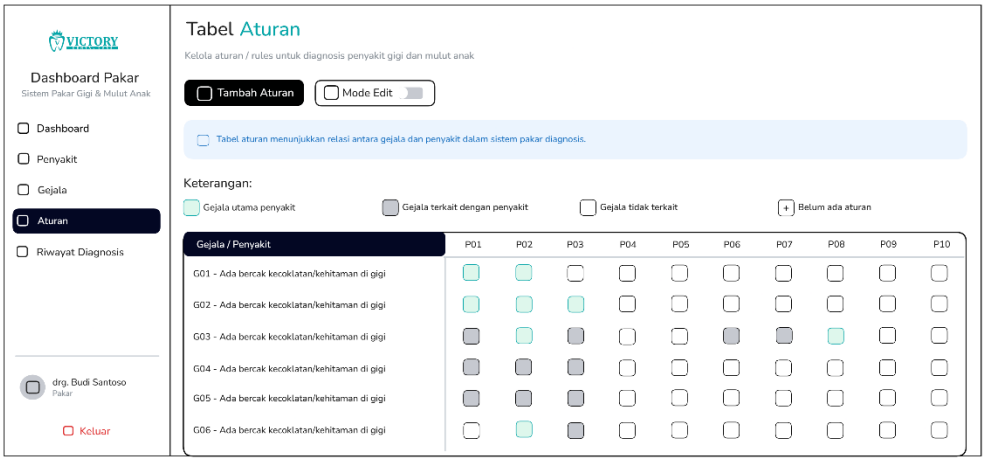
o. Halaman Kelola Gejala



Gambar 45. *Mockup* Halaman Kelola Gejala

Sebagaimana terlihat pada Gambar 45, halaman kelola gejala dirancang sebagai antarmuka bagi pakar untuk mengonfigurasi basis data gejala yang menjadi parameter utama dalam proses diagnosis. Halaman ini menampilkan tabel daftar gejala yang memuat informasi seperti kode, nama gejala, serta pertanyaan spesifik yang akan diajukan kepada pengguna sistem. Halaman ini menyediakan fasilitas manajemen lengkap, meliputi tombol "Tambah Gejala" untuk *input* data baru, fitur pencarian, serta opsi penyuntingan dan penghapusan pada kolom aksi.

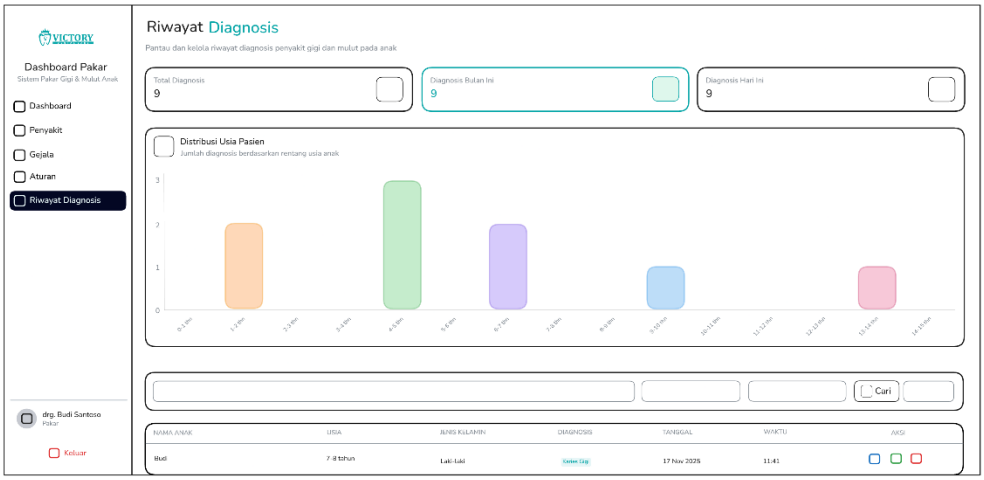
p. Halaman Kelola Aturan



Gambar 46. *Mockup* Halaman Kelola Aturan

Sebagaimana terlihat pada Gambar 46, halaman kelola aturan merupakan antarmuka penting yang memvisualisasikan basis pengetahuan sistem dalam bentuk matriks 2x2 relasi antara gejala dan penyakit. Halaman ini memungkinkan pakar untuk mendefinisikan dan memodifikasi logika diagnosis (*rule base*) dengan mudah melalui mekanisme tabel interaktif, di mana setiap sel merepresentasikan hubungan spesifik, baik sebagai gejala utama maupun gejala terkait yang ditandai dengan indikator warna sesuai untuk memudahkan identifikasi pola diagnosis yang digunakan dalam metode *forward chaining*.

q. Halaman Kelola Riwayat Diagnosis



Gambar 47. *Mockup* Halaman Kelola Riwayat Diagnosis

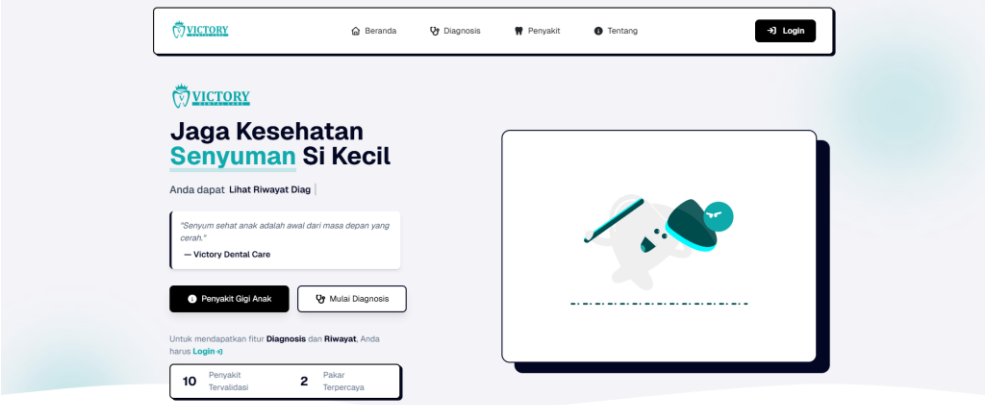
Sebagaimana terlihat pada Gambar 47, halaman kelola riwayat diagnosis berfungsi sebagai pusat data rekam medis digital yang memungkinkan pakar untuk memantau seluruh aktivitas diagnosis yang terjadi di dalam sistem. Halaman ini menyajikan ringkasan statistik kinerja diagnosis, mulai dari total hingga data harian di bagian atas, menampilkan distribusi usia pengguna dengan diagram batang, serta menampilkan tabel rincian data pasien yang mencakup identitas anak, hasil diagnosis, dan waktu pelaksanaan diagnosis. Halaman ini juga dilengkapi dengan fitur pencarian untuk memudahkan penelusuran data spesifik. Selain itu, tersedia tombol aksi pada setiap entri yang memungkinkan pakar untuk melakukan berbagai tindakan manajemen, mulai dari melihat detail diagnosis, mencetak laporan dalam format PDF, hingga menghapus riwayat data jika diperlukan.

4.3 *Implementation*

Tahap *implementation* merupakan tahap realisasi dari hasil perancangan sistem ke dalam bentuk aplikasi yang dapat dijalankan secara fungsional. Pada tahap ini, seluruh komponen sistem pakar diagnosis penyakit gigi dan mulut pada anak dibangun menggunakan *framework* Laravel sebagai kerangka kerja pengembangan aplikasi berbasis web dan MySQL sebagai sistem manajemen basis data untuk penyimpanan dan pengelolaan data. Laravel dipilih karena menyediakan arsitektur MVC (*Model-View-Controller*) yang terstruktur, fitur keamanan, serta ekosistem yang lengkap untuk pengembangan aplikasi web modern. Sementara itu, MySQL digunakan karena kemampuannya dalam menangani relasi data yang kompleks dan performa yang optimal untuk sistem berbasis pengetahuan. Integrasi antara Laravel dan MySQL dilakukan melalui Eloquent ORM (*Object-Relational Mapper*) yang memudahkan operasi *database* dengan pendekatan *object-oriented*.

Hasil implementasi ditampilkan melalui tangkapan layar dari setiap halaman yang ada pada sistem Setiap tampilan halaman disertai dengan potongan kode program yang menunjukkan cara kerja sistem di balik antarmuka yang ditampilkan. Berikut ini dijelaskan hasil implementasi pada masing-masing halaman sistem:

a. Halaman Beranda



Gambar 48. Halaman Beranda

Halaman beranda merupakan halaman utama yang ditampilkan kepada pengguna setelah mengakses sistem pakar diagnosis penyakit gigi dan mulut pada anak. Sebagaimana ditunjukkan pada Gambar 48, halaman ini dirancang dengan tampilan yang informatif dan *user-friendly*, menyajikan gambaran umum mengenai sistem serta memberikan akses cepat ke fitur-fitur utama yang tersedia. Selain itu, terdapat tombol *call-to-action* yang mengarahkan pengguna untuk memulai proses diagnosis atau menjelajahi informasi penyakit gigi dan mulut pada anak yang telah tersedia dalam sistem.

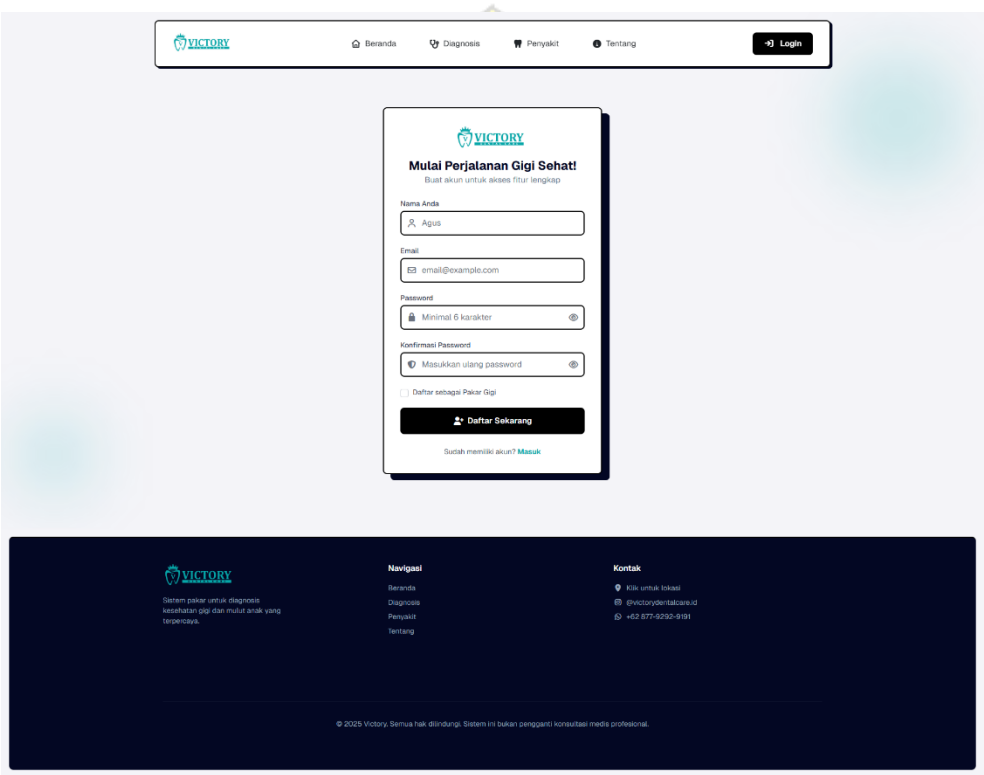
Halaman ini dapat diakses oleh pengguna umum tanpa harus melakukan *login* terlebih dahulu, sehingga memungkinkan siapa saja untuk mendapatkan informasi dasar mengenai sistem. Namun, untuk mengakses fitur diagnosis lengkap dan menyimpan riwayat diagnosis, pengguna diwajibkan untuk melakukan registrasi dan *login*. Bagi pengguna yang telah *login* dengan role admin atau pakar, sistem akan secara otomatis mengarahkan mereka ke *dashboard* masing-masing melalui *middleware* khusus, sehingga halaman beranda hanya ditampilkan untuk pengguna biasa (klien). Navigasi pada

halaman ini juga menyediakan menu untuk mengakses halaman lain seperti halaman tentang sistem dan daftar penyakit yang dapat didiagnosis.

Berikut adalah implementasi *routing* yang menangani akses ke halaman beranda:

```
<?php
Route::middleware('user.only')->group(function () {
    Route::get('/', function () {
        return view('pages.home');
    })->name('home');
});
```

b. Halaman Daftar



Gambar 49. Halaman Daftar

Halaman daftar merupakan halaman awal bagi pengguna baru untuk membuat akun dalam sistem pakar diagnosis penyakit gigi dan mulut pada anak. Sebagaimana ditunjukkan pada Gambar 49, halaman ini menyediakan formulir registrasi yang meminta pengguna untuk mengisi informasi dasar berupa nama lengkap, alamat email, kata sandi, dan konfirmasi kata sandi. Untuk

memastikan keamanan akun, sistem menerapkan validasi pada setiap *field input*, termasuk validasi format email yang benar dan kecocokan antara kata sandi dengan konfirmasi kata sandi.

Salah satu fitur khusus pada halaman ini adalah opsi "Daftar sebagai Pakar Gigi" yang disediakan bagi tenaga medis profesional. Ketika opsi ini dipilih, sistem akan menampilkan *field* tambahan untuk memasukkan nomor STR (Surat Tanda Registrasi) sebagai bukti kredensial profesional. Pengguna yang mendaftar sebagai pakar akan memiliki status "*pending*" secara *default* dan memerlukan persetujuan dari administrator sebelum dapat mengakses fitur-fitur khusus pakar. Setelah seluruh data diisi dengan lengkap dan valid, pengguna dapat menekan tombol "Daftar Sekarang" untuk mengirimkan formulir registrasi. Sistem kemudian akan memproses data registrasi, menyimpannya ke dalam *database*, dan mengarahkan pengguna ke halaman *login* untuk melakukan autentikasi pertama kali.

Berikut adalah implementasi kode pada AuthController yang menangani proses registrasi pengguna:

```
<?php
public function showRegisterForm()
{
    if (Auth::check()) {
        return $this->redirectBasedOnRole();
    }
    return view('pages.register');
}

public function register(Request $request)
{
    $role = $request->has('is_expert') ? 'pakar' : 'user';

    $rules = [
        'nama_lengkap' => 'required|string|max:100',
        'email' =>
'required|string|email|max:100|unique:pengguna,email',
        'password' => 'required|string|min:6|confirmed',
    ];
```

```
    $messages = [
        'nama_lengkap.required' => 'Nama lengkap wajib diisi',
        'nama_lengkap.max' => 'Nama lengkap maksimal 100 karakter',
        'email.required' => 'Email wajib diisi',
        'email.email' => 'Format email tidak valid',
        'email.unique' => 'Email sudah terdaftar',
        'password.required' => 'Password wajib diisi',
        'password.min' => 'Password minimal 6 karakter',
        'password.confirmed' => 'Konfirmasi password tidak cocok',
    ];

    if ($role === 'pakar') {
        $rules['no_str_dokter'] =
'required|string|size:7|unique:pengguna,no_str_dokter';
        $messages['no_str_dokter.required'] = 'Nomor STR wajib
diisi untuk pakar';
        $messages['no_str_dokter.size'] = 'Nomor STR harus 7
digit';
        $messages['no_str_dokter.unique'] = 'Nomor STR sudah
terdaftar';
    }

    $request->validate($rules, $messages);

    $data = [
        'nama_lengkap' => $request->nama_lengkap,
        'email' => strtolower($request->email),
        'password' => Hash::make($request->password),
        'role' => $role,
        'tanggal_daftar' => Carbon::now(),
    ];

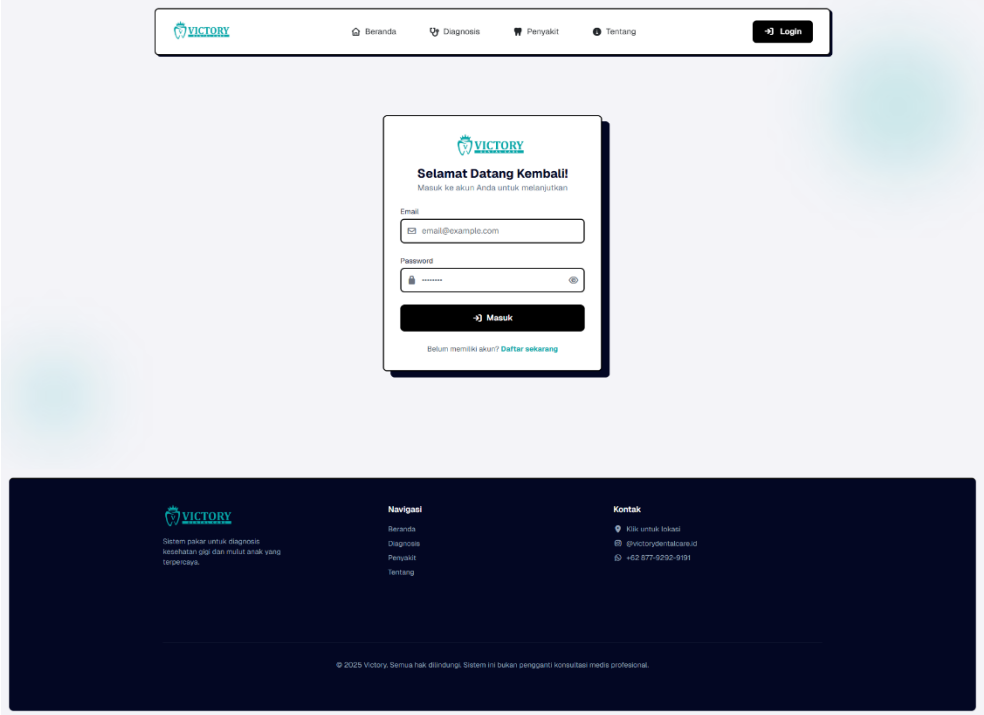
    if ($role === 'pakar') {
        $data['no_str_dokter'] = $request->no_str_dokter;
        $data['status_approval'] = 'pending';
    } else {
        $data['status_approval'] = 'approved';
    }

    Pengguna::create($data);

    if ($role === 'pakar') {
        return redirect()->route('login')->with('success',
'Registrasi berhasil! Akun pakar Anda menunggu persetujuan admin.
Silakan login setelah di-approve.');
    }

    return redirect()->route('login')->with('success', 'Registrasi
berhasil! Silakan login dengan akun Anda.');
}
```

c. Halaman Masuk



Gambar 50. Halaman Masuk

Halaman masuk merupakan gerbang autentikasi bagi pengguna yang telah memiliki akun untuk mengakses sistem pakar diagnosis penyakit gigi dan mulut pada anak. Sebagaimana ditunjukkan pada Gambar 50, halaman ini menyediakan formulir *login* yang sederhana namun fungsional, dengan dua *field input* utama yaitu email dan kata sandi. Sistem menerapkan validasi autentikasi yang ketat untuk memastikan keamanan akses, di mana hanya pengguna dengan kredensial yang valid dan akun dengan status aktif yang dapat masuk ke dalam sistem.

Setelah proses autentikasi berhasil, sistem akan secara otomatis mengarahkan pengguna ke *dashboard* yang sesuai berdasarkan peran (*role*) masing-masing. Pengguna dengan *role* "admin" akan diarahkan ke *dashboard* administrator untuk mengelola pengguna dan persetujuan pakar. Pengguna

dengan *role* "pakar" yang telah disetujui akan diarahkan ke *dashboard* pakar untuk mengelola basis pengetahuan sistem, sedangkan pengguna biasa (klien) akan diarahkan ke halaman beranda untuk memulai proses diagnosis. Khusus untuk akun pakar dengan status "*pending*" atau "*rejected*", sistem akan menampilkan notifikasi khusus dan mencegah akses ke dalam sistem hingga mendapat persetujuan dari administrator. Bagi pengguna yang belum memiliki akun, tersedia tautan "Daftar Sekarang" yang mengarahkan ke halaman registrasi.

Berikut adalah implementasi kode pada AuthController yang menangani proses autentikasi pengguna:

```
<?php
public function showLoginForm()
{
    if (Auth::check()) {
        return $this->redirectBasedOnRole();
    }
    return view('pages.login');
}

public function login(Request $request)
{
    $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ], [
        'email.required' => 'Email wajib diisi',
        'email.email' => 'Format email tidak valid',
        'password.required' => 'Password wajib diisi',
    ]);

    $pengguna = Pengguna::where('email', strtolower($request-
>email))->first();

    if (!$pengguna || !Hash::check($request->password, $pengguna-
>password)) {
        return back()->withErrors(['email' => 'Email atau password
salah'])->withInput();
    }

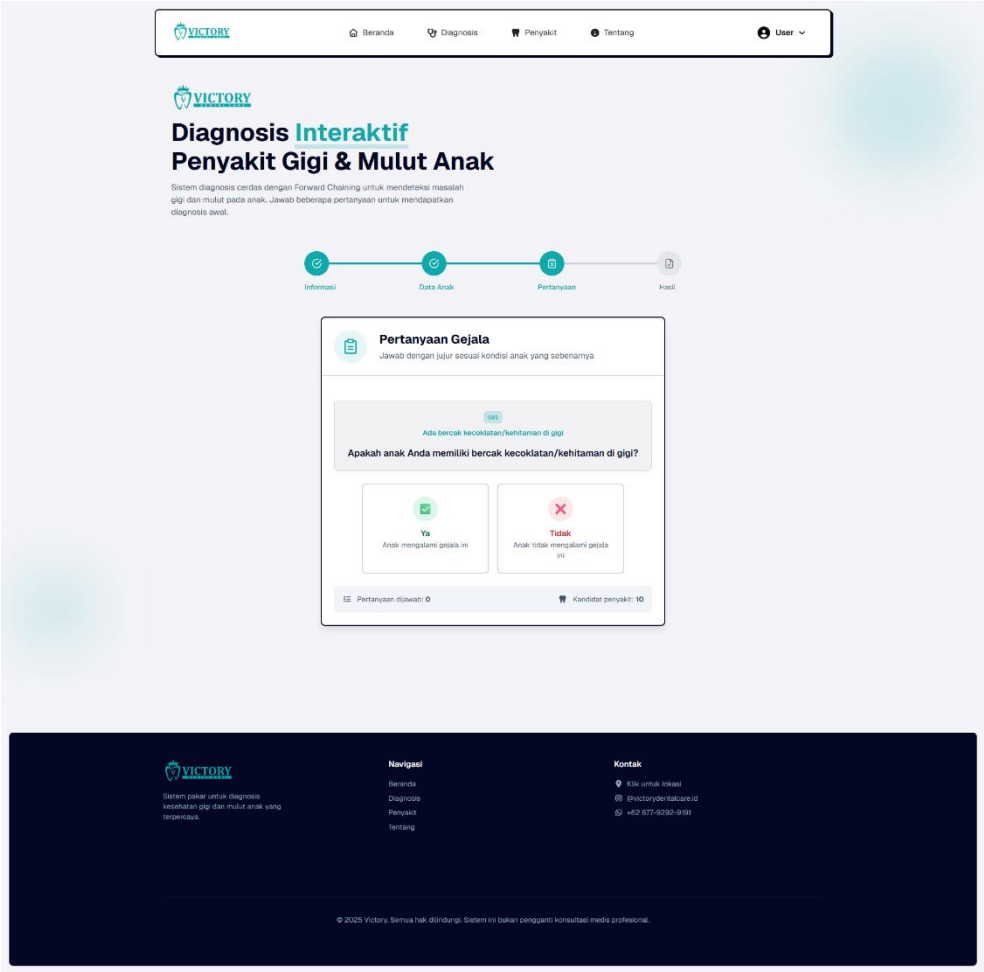
    if ($pengguna->role === 'pakar' && $pengguna->status_approval
!== 'approved') {
        $message = $pengguna->status_approval === 'pending'
            ? 'Akun Anda masih menunggu approval admin'
```

```
            : 'Akun Anda ditolak oleh admin';
        return back()->withErrors(['email' => $message])-
>withInput();
    }

    Auth::login($pengguna);
    $request->session()->regenerate();

    return $this->redirectBasedOnRole();
}
```

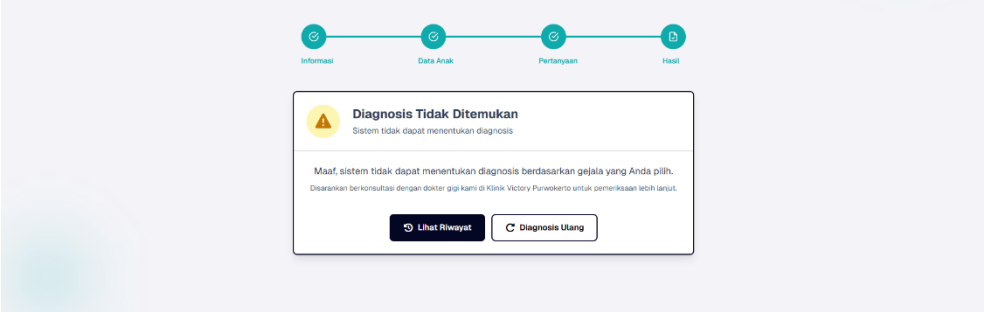
d. Halaman Diagnosis



Gambar 51. Halaman Diagnosis

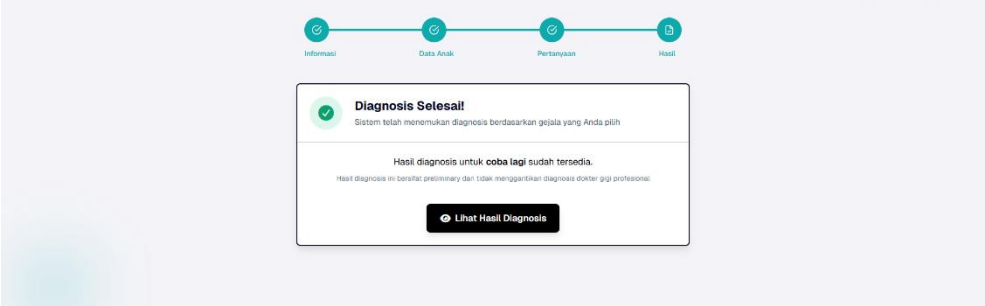
Halaman diagnosis merupakan halaman inti dari sistem pakar yang memfasilitasi proses identifikasi penyakit gigi dan mulut pada anak berdasarkan gejala-gejala yang dialami. Sebagaimana ditunjukkan pada Gambar 51, halaman ini menampilkan serangkaian pertanyaan mengenai gejala secara

interaktif dan bertahap menggunakan metode *forward chaining*. Pengguna akan menjawab setiap pertanyaan dengan opsi "Ya" atau "Tidak", dan sistem akan secara dinamis menentukan pertanyaan selanjutnya berdasarkan jawaban yang diberikan serta menerapkan teknik *pruning* untuk mengeliminasi kandidat penyakit yang tidak relevan. Proses ini berlangsung hingga sistem dapat menghasilkan kesimpulan diagnosis atau hingga seluruh pertanyaan relevan selesai ditanyakan. Halaman ini hanya dapat diakses oleh pengguna yang telah *login*, sehingga memastikan setiap sesi diagnosis dapat disimpan dalam riwayat pengguna.



Gambar 52. Halaman Diagnosis Tidak Ditemukan

Setelah seluruh pertanyaan selesai dijawab, sistem akan menampilkan hasil diagnosis dalam dua kemungkinan kondisi. Gambar 52 menunjukkan tampilan ketika sistem tidak dapat menemukan diagnosis yang sesuai dengan kombinasi gejala yang dipilih pengguna, di mana sistem akan memberikan pesan informatif bahwa tidak ada penyakit yang cocok dan menyarankan pengguna untuk berkonsultasi langsung dengan dokter gigi di Klinik Victory Purwokerto untuk pemeriksaan lebih lanjut. Sebaliknya, Gambar 53 menunjukkan tampilan ketika diagnosis berhasil ditemukan.



Gambar 53. Halaman Diagnosis Ditemukan

Berikut adalah implementasi *controller* yang menangani proses diagnosis menggunakan metode *forward chaining*:

```
<?php
public function startDiagnosis(Request $request)
{
    $request->validate([
        'nama_anak' => 'required|string|max:255',
        'usia_anak' => 'required|string|max:50',
        'jenis_kelamin' => 'required|in:Laki-laki,Perempuan'
    ]);

    $diagnosis = Diagnosis::create([
        'id_pengguna' => Auth::id(),
        'nama_anak' => $request->nama_anak,
        'usia_anak' => $request->usia_anak,
        'jenis_kelamin' => $request->jenis_kelamin,
        'tanggal_diagnosis' => now()->toDateString(),
        'waktu_diagnosis' => now()->toTimeString(),
    ]);

    return response()->json([
        'success' => true,
        'id_diagnosis' => $diagnosis->id_diagnosis,
        'message' => 'Diagnosis dimulai'
    ]);
}

public function saveAnswer(Request $request)
{
    $request->validate([
        'id_diagnosis' => 'required|exists:diagnosis,id_diagnosis',
        'id_gejala' => 'required|exists:gejala,id_gejala',
        'jawaban_user' => 'required|boolean'
    ]);

    DetailDiagnosis::create([
        'id_diagnosis' => $request->id_diagnosis,
        'id_gejala' => $request->id_gejala,
        'jawaban_user' => $request->jawaban_user
    ]);
```

```
    return response()->json([
        'success' => true,
        'message' => 'Jawaban disimpan'
    ]);
}

public function getNextQuestion(Request $request)
{
    $idDiagnosis = $request->id_diagnosis;

    $answeredSymptoms = DetailDiagnosis::where('id_diagnosis',
$idDiagnosis)->get();
    $yesAnswers = $answeredSymptoms->where('jawaban_user', true)-
>pluck('id_gejala')->toArray();
    $noAnswers = $answeredSymptoms->where('jawaban_user', false)-
>pluck('id_gejala')->toArray();

    $candidateDiseases = $this->getCandidateDiseases($yesAnswers,
$noAnswers);

    if (empty($candidateDiseases)) {
        return response()->json([
            'completed' => true,
            'has_result' => false,
            'message' => 'Tidak ada penyakit yang cocok'
        ]);
    }

    $confirmedDisease = $this-
>checkConfirmedDisease($candidateDiseases, $yesAnswers);

    if ($confirmedDisease) {
        $this->saveDiagnosisResult($idDiagnosis,
$confirmedDisease);

        return response()->json([
            'completed' => true,
            'has_result' => true,
            'id_penyakit' => $confirmedDisease
        ]);
    }

    $nextSymptom = $this->getNextSymptom($candidateDiseases,
$answeredIds);

    if (!$nextSymptom) {
        return response()->json([
            'completed' => true,
            'has_result' => false,
            'message' => 'Tidak dapat menentukan diagnosis'
        ]);
    }

    return response()->json([
        'completed' => false,
        'gejala' => [
            'id_gejala' => $nextSymptom->id_gejala,
```

```
            'pertanyaan_gejala' => $nextSymptom->pertanyaan_gejala
        ]
    ]);
}

private function getCandidateDiseases($yesAnswers, $noAnswers)
{
    $allDiseases = Penyakit::all();
    $candidates = [];

    foreach ($allDiseases as $disease) {
        $rules = Aturan::where('id_penyakit', $disease-
>id_penyakit)->get();
        $diseaseSymptoms = $rules->pluck('id_gejala')->toArray();

        $invalidYesAnswers = array_diff($yesAnswers,
$diseaseSymptoms);
        if (!empty($invalidYesAnswers)) {
            continue;
        }

        $grouped = $rules->groupBy('group_id');
        $isValid = true;

        foreach ($grouped as $groupRules) {
            $symptomIds = $groupRules->pluck('id_gejala')-
>toArray();

            if ($groupRules->count() === 1) {
                if (in_array($symptomIds[0], $noAnswers)) {
                    $isValid = false;
                    break;
                }
            } else {
                $allNo = count(array_intersect($symptomIds,
$noAnswers)) === count($symptomIds);
                if ($allNo) {
                    $isValid = false;
                    break;
                }
            }
        }

        if ($isValid) {
            $candidates[] = [
                'id_penyakit' => $disease->id_penyakit,
                'rules' => $rules
            ];
        }
    }

    return $candidates;
}

private function checkConfirmedDisease($candidates, $yesAnswers)
{
    $fullyMatched = [];
```

```
    foreach ($candidates as $candidate) {
        $grouped = $candidate['rules']->groupBy('group_id');
        $allConditionsMet = true;

        foreach ($grouped as $groupRules) {
            $symptomIds = $groupRules->pluck('id_gejala')-
>toArray();

            if ($groupRules->count() === 1) {
                if (!in_array($symptomIds[0], $yesAnswers)) {
                    $allConditionsMet = false;
                    break;
                }
            } else {
                if (empty(array_intersect($symptomIds,
$yesAnswers))) {
                    $allConditionsMet = false;
                    break;
                }
            }
        }

        if ($allConditionsMet) {
            $fullyMatched[] = $candidate;
        }
    }

    if (count($fullyMatched) === 1) {
        return $fullyMatched[0]['id_penyakit'];
    }

    return null;
}

private function getNextSymptom($candidates, $answeredIds)
{
    $relevantSymptoms = [];

    foreach ($candidates as $candidate) {
        $symptomIds = $candidate['rules']->pluck('id_gejala')-
>toArray();
        $relevantSymptoms = array_merge($relevantSymptoms,
$symptomIds);
    }

    $unansweredSymptoms =
array_diff(array_unique($relevantSymptoms), $answeredIds);

    if (empty($unansweredSymptoms)) {
        return null;
    }

    foreach ($candidates as $candidate) {
        $grouped = $candidate['rules']->groupBy('group_id');

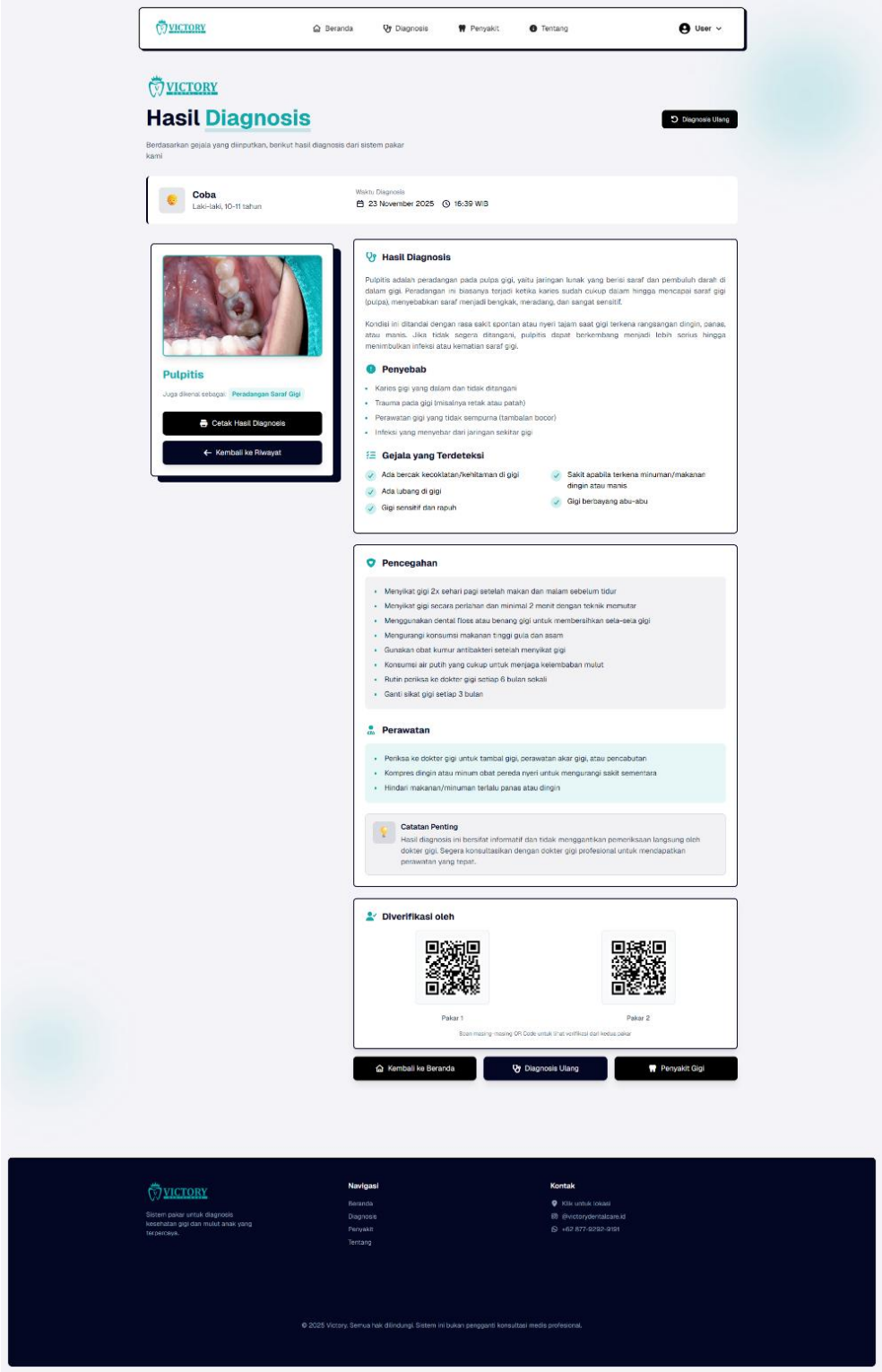
        foreach ($grouped as $groupRules) {
```

```
            if ($groupRules->count() === 1) {
                $symptomId = $groupRules->first()->id_gejala;
                if (in_array($symptomId, $unansweredSymptoms)) {
                    return Gejala::find($symptomId);
                }
            }
        }
    }

    return Gejala::find(array_values($unansweredSymptoms)[0]);
}

private function saveDiagnosisResult($idDiagnosis, $idPenyakit)
{
    HasilDiagnosis::updateOrCreate(
        ['id_diagnosis' => $idDiagnosis],
        ['id_penyakit' => $idPenyakit]
    );
}
```

e. Halaman Hasil Diagnosis



Gambar 54. Halaman Hasil Diagnosis

Halaman hasil diagnosis merupakan halaman yang menampilkan kesimpulan akhir dari proses diagnosis yang telah dilakukan oleh pengguna. Sebagaimana ditunjukkan pada Gambar 54, halaman ini menyajikan informasi lengkap mengenai penyakit yang teridentifikasi berdasarkan gejala-gejala yang dialami anak. Informasi yang ditampilkan mencakup identitas anak (nama, usia, jenis kelamin), tanggal dan waktu diagnosis, daftar gejala yang dialami, serta detail lengkap mengenai penyakit yang terdiagnosis meliputi deskripsi penyakit, penyebab, dan solusi penanganan yang direkomendasikan.

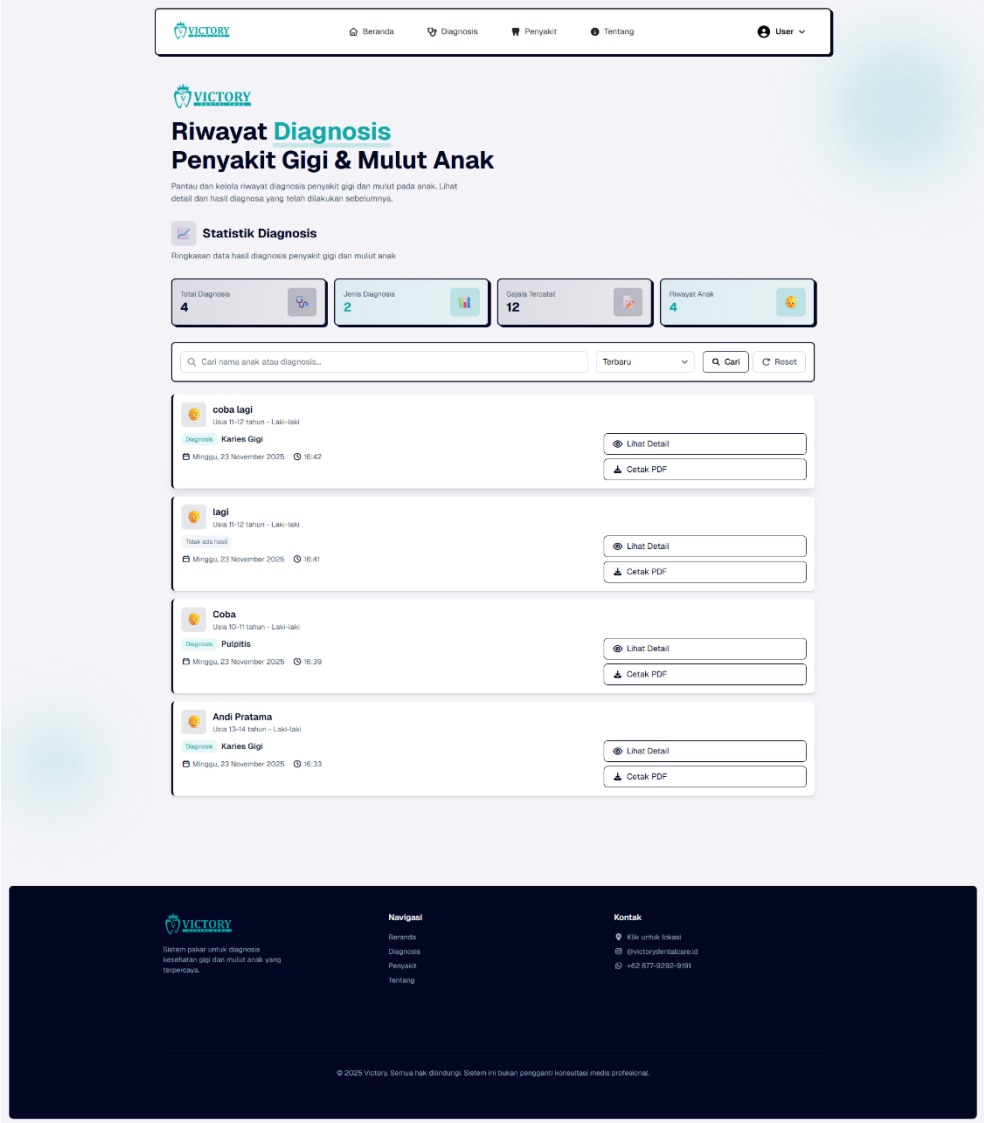
Berikut adalah implementasi kode pada *controller* untuk halaman hasil diagnosis:

```
<?php
public function hasil($id)
{
    $user = Auth::user();

    $diagnosis = Diagnosis::with([
        'hasilDiagnosis.penyakit',
        'detailDiagnosis' => function($query) {
            $query->where('jawaban_user', true)->with('gejala');
        }
    ])
    ->where('id_pengguna', $user->id_pengguna)
    ->where('id_diagnosis', $id)
    ->firstOrFail();

    return view('pages.hasil', compact('diagnosis'));
}
```

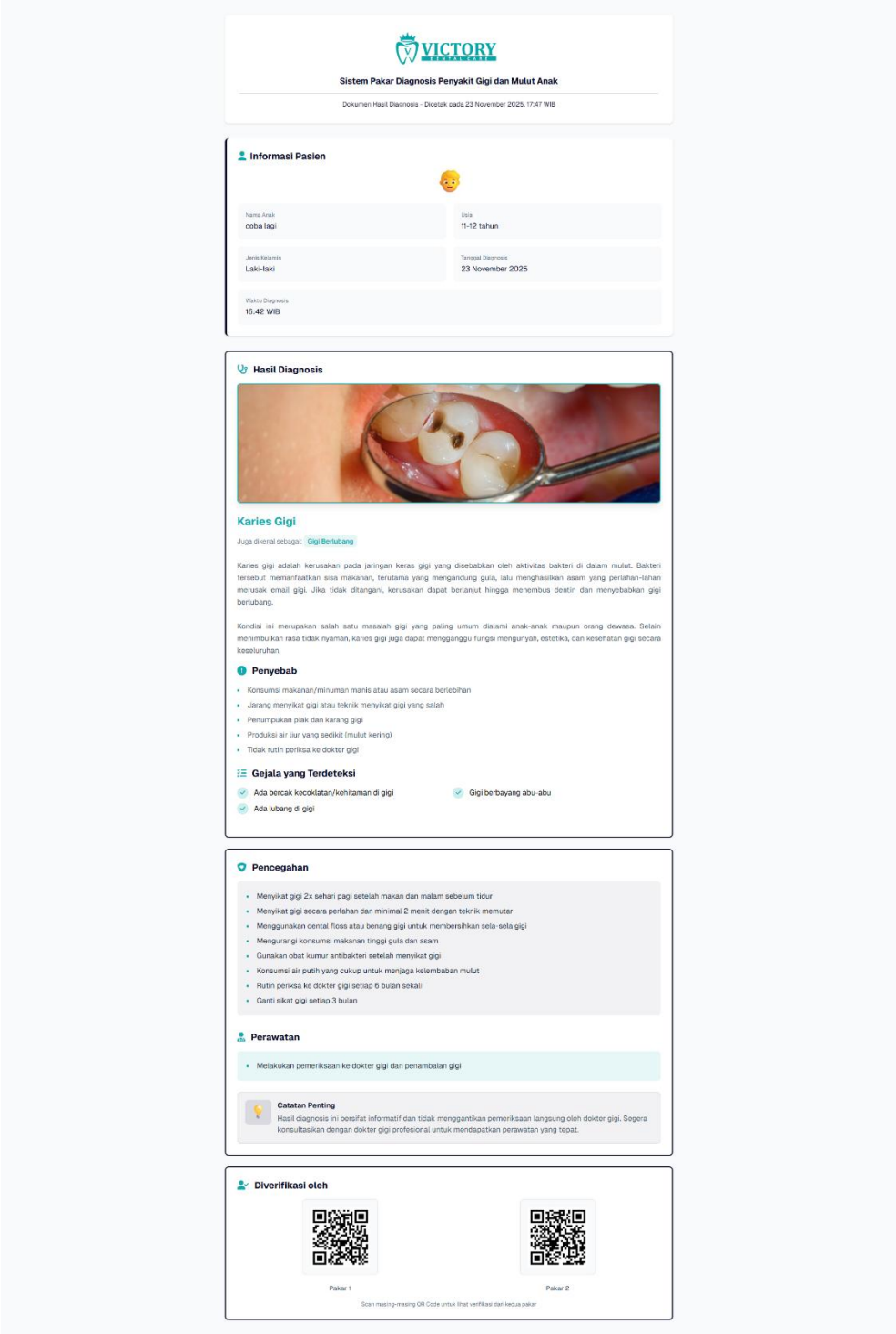
f. Halaman Riwayat Diagnosis



Gambar 55. Halaman Riwayat Diagnosis

Halaman riwayat diagnosis merupakan halaman yang menampilkan daftar seluruh diagnosis yang pernah dilakukan oleh pengguna. Sebagaimana ditunjukkan pada Gambar 55, halaman ini menyajikan informasi riwayat diagnosis dalam bentuk tabel yang terstruktur, mencakup data nama anak, usia, jenis kelamin, tanggal dan waktu diagnosis, serta hasil diagnosis yang diperoleh. Setiap baris data dilengkapi dengan tombol aksi yang memungkinkan

pengguna untuk melihat detail hasil diagnosis dan mengunduh hasil dalam format PDF.



Gambar 56. Cetak PDF

Fitur yang tersedia pada halaman riwayat diagnosis meliputi tombol "Lihat Detail" yang mengarahkan pengguna ke halaman hasil diagnosis lengkap seperti yang ditampilkan pada Gambar 54, serta tombol "Cetak PDF" yang memungkinkan pengguna untuk mengunduh hasil diagnosis dalam format PDF sebagaimana ditunjukkan pada Gambar 56. Halaman ini hanya dapat diakses oleh pengguna yang telah *login* dan hanya menampilkan riwayat diagnosis milik pengguna tersebut untuk menjaga privasi data.

Berikut adalah implementasi kode pada *controller* untuk halaman riwayat diagnosis:

```
<?php
public function riwayat(Request $request)
{
    $user = Auth::user();

    $query = Diagnosis::with(['hasilDiagnosis.penyakit',
'detailDiagnosis.gejala'])
        ->where('id_pengguna', $user->id_pengguna);

    if ($request->filled('search')) {
        $search = $request->search;
        $query->where(function($q) use ($search) {
            $q->where('nama_anak', 'like', "%{$search}%")
                ->orWhereHas('hasilDiagnosis.penyakit',
function($q2) use ($search) {
                    $q2->where('nama_penyakit', 'like',
"%{$search}%");
                });
        });
    }

    $sort = $request->get('sort', 'newest');
    if ($sort === 'oldest') {
        $query->orderBy('tanggal_diagnosis', 'asc')
                ->orderBy('waktu_diagnosis', 'asc');
    } else {
        $query->orderBy('tanggal_diagnosis', 'desc')
                ->orderBy('waktu_diagnosis', 'desc');
    }

    $riwayatList = $query->get();

    $stats = [
        'total_diagnosis' => Diagnosis::where('id_pengguna',
$user->id_pengguna)->count(),
```

```
        'jenis_diagnosis' => HasilDiagnosis::whereHas('diagnosis',
function($q) use ($user) {
            $q->where('id_pengguna', $user->id_pengguna);
        })->distinct('id_penyakit')->count('id_penyakit'),
        'total_gejala' => DetailDiagnosis::whereHas('diagnosis',
function($q) use ($user) {
            $q->where('id_pengguna', $user->id_pengguna);
        })->where('jawaban_user', true)->count(),
        'total_anak' => Diagnosis::where('id_pengguna', $user-
>id_pengguna)
            ->distinct('nama_anak')->count('nama_anak')
    ];

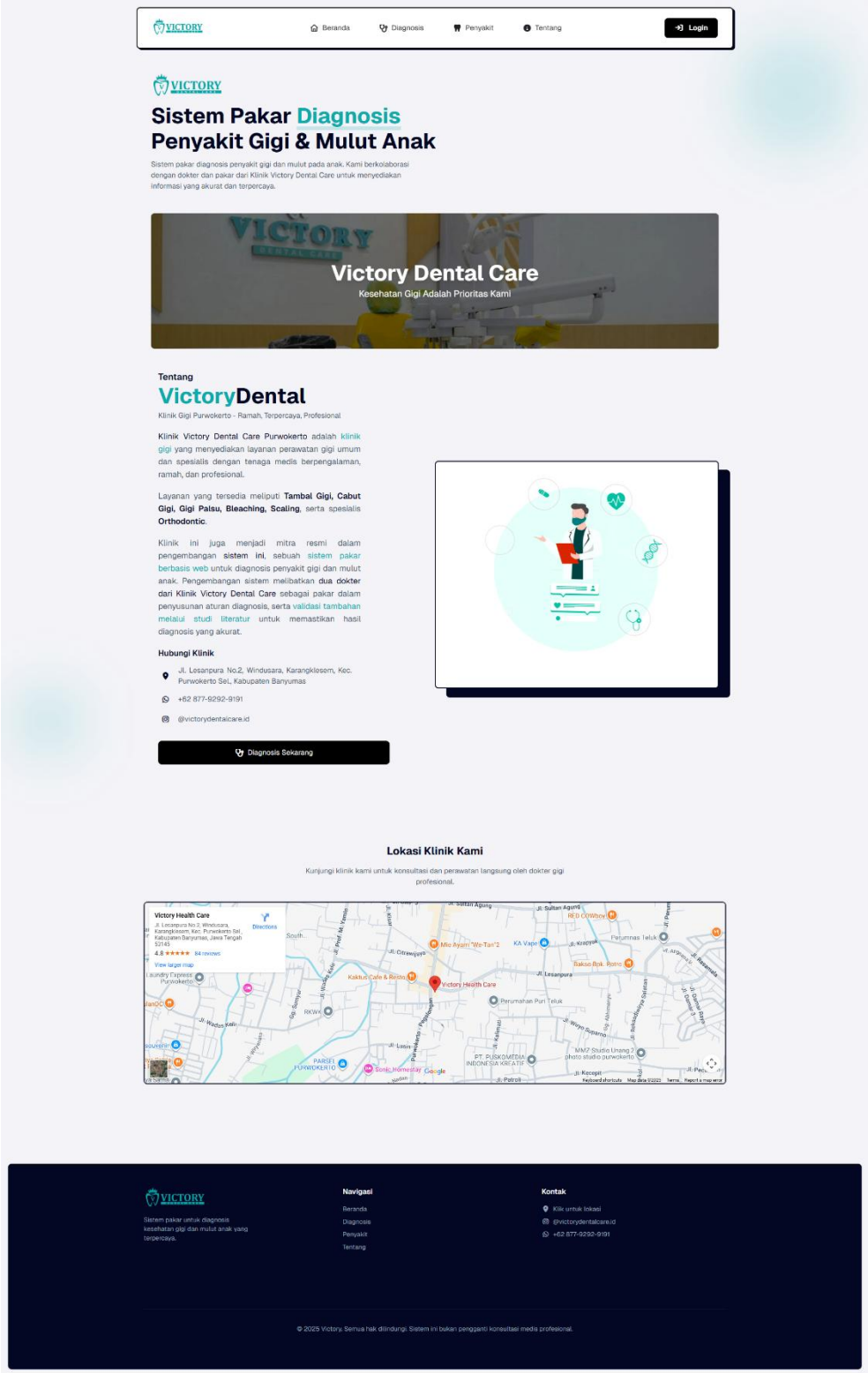
    return view('pages.riwayat', compact('riwayatList', 'stats'));
}

public function generatePDF($id)
{
    $user = Auth::user();

    $diagnosis = Diagnosis::with([
        'hasilDiagnosis.penyakit',
        'detailDiagnosis' => function($query) {
            $query->where('jawaban_user', true)->with('gejala');
        }
    ])
    ->where('id_pengguna', $user->id_pengguna)
    ->where('id_diagnosis', $id)
    ->firstOrFail();

    return view('pages.hasil-pdf', compact('diagnosis'));
}
```

g. Halaman Tentang

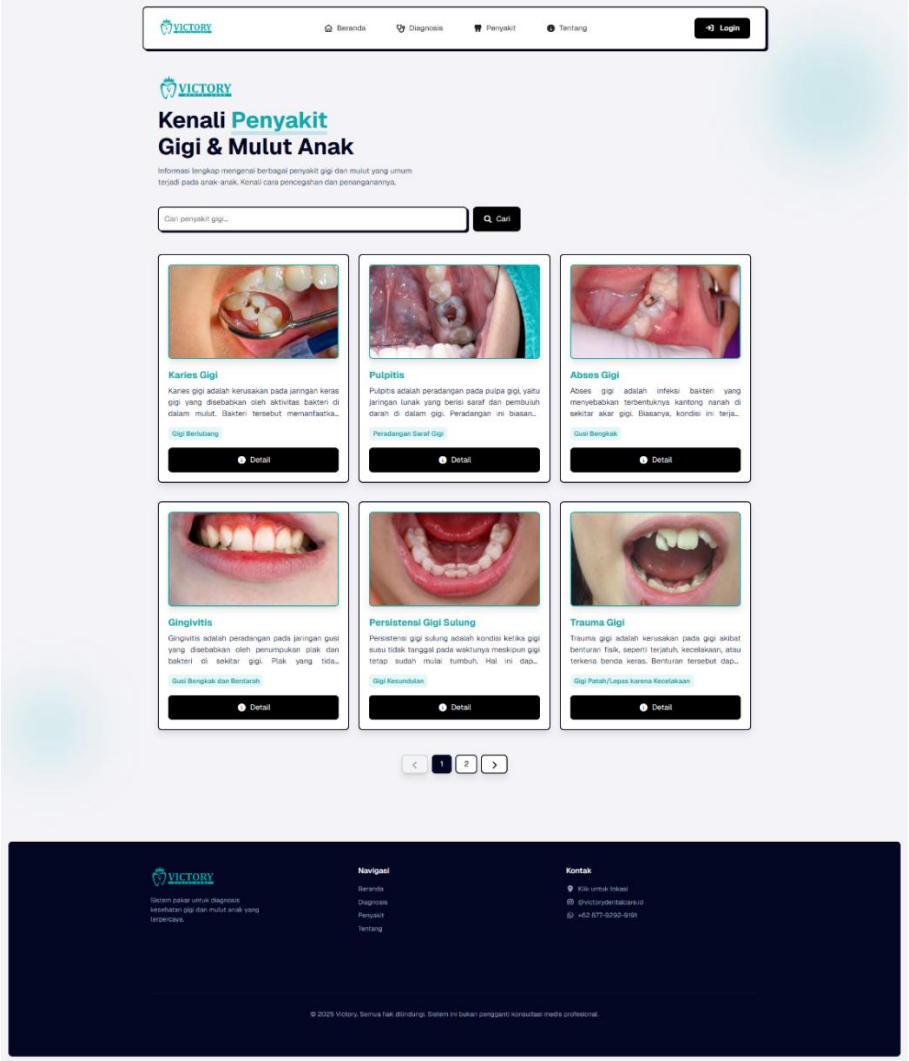


Gambar 57. Halaman Tentang

Halaman tentang merupakan halaman yang menyajikan informasi umum mengenai Klinik Victory Purwokerto. Sebagaimana ditunjukkan pada Gambar 57, halaman ini juga dapat menampilkan informasi kontak, sosial media, dan lokasi Klinik Victory Purwokerto. Berikut adalah implementasi *routing* untuk halaman tentang:

```
<?php
Route::middleware('user.only')->group(function () {
    Route::get('/tentang', function () {
        return view('pages.tentang');
    })->name('tentang');
});
```

h. Halaman Info Penyakit



Gambar 58. Halaman Info Penyakit

Halaman info penyakit merupakan halaman yang menyediakan informasi lengkap mengenai berbagai jenis penyakit gigi dan mulut pada anak yang dapat didiagnosis oleh sistem. Sebagaimana ditunjukkan pada Gambar 58, halaman ini menampilkan daftar penyakit dalam bentuk *card* yang menarik dan informatif, di mana setiap *card* menampilkan nama penyakit dan deskripsi singkat mengenai kondisi penyakit tersebut.

Setiap *card* penyakit dilengkapi dengan tombol "Lihat Detail" yang mengarahkan pengguna ke halaman detail penyakit. Halaman detail akan menampilkan informasi yang lebih mengenai penyakit tersebut, termasuk gejala-gejala yang umum dialami, penyebab terjadinya penyakit, serta solusi penanganan dan pencegahan yang direkomendasikan. Halaman info penyakit ini dapat diakses oleh semua pengguna, baik yang sudah *login* maupun belum, sehingga berfungsi sebagai sumber informasi publik yang bermanfaat untuk edukasi kesehatan gigi dan mulut anak.

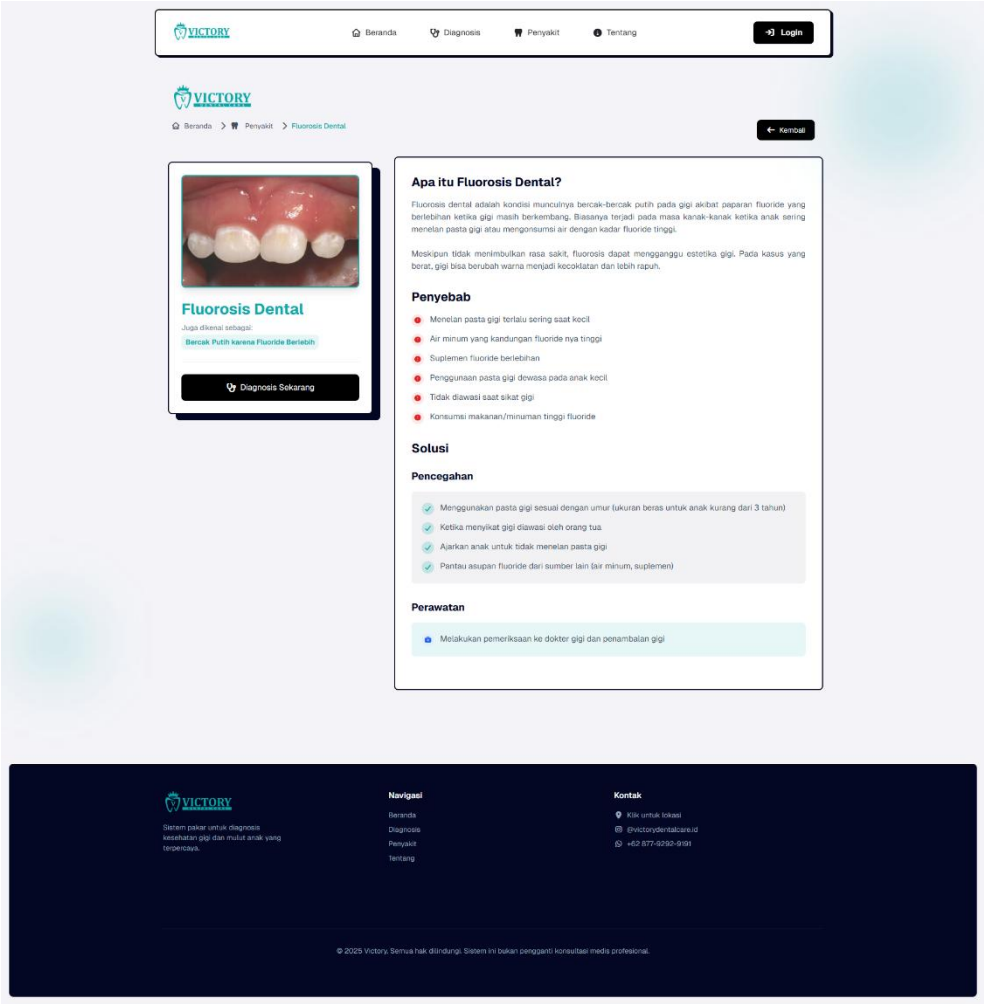
Berikut adalah implementasi kode pada *controller* untuk halaman info penyakit:

```
<?php
public function index(Request $request)
{
    $search = $request->input('search');

    $penyakits = Penyakit::query()
        ->when($search, function ($query, $search) {
            return $query->where('nama_penyakit', 'like',
"%{$search}%")
                         ->orWhere('nama_lain', 'like',
"%{$search}%")
                         ->orWhere('kode_penyakit', 'like',
"%{$search}%")
                         ->orWhere('deskripsi', 'like',
"%{$search}%");
        })
        ->orderBy('kode_penyakit', 'asc')
        ->paginate(6)
        ->withQueryString();
```

```
    return view('pages.penyakit', compact('penyakits', 'search'));
}
```

i. Halaman Detail Info Penyakit



Gambar 59. Halaman Detail Info Penyakit

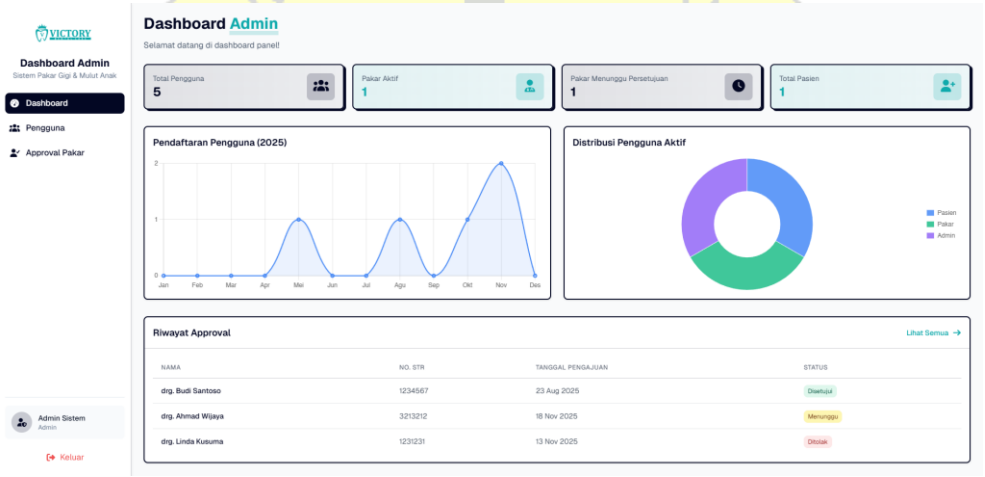
Halaman detail info penyakit merupakan halaman yang menampilkan informasi lengkap mengenai satu jenis penyakit gigi dan mulut pada anak secara spesifik. Sebagaimana ditunjukkan pada Gambar 59, halaman ini menyajikan informasi detail yang mencakup kode penyakit, nama penyakit beserta nama lainnya, deskripsi lengkap mengenai kondisi penyakit, penyebab terjadinya penyakit, serta solusi penanganan dan pencegahan yang direkomendasikan. Halaman ini dirancang dengan tampilan yang terstruktur dan mudah dipahami,

sehingga pengguna dapat memperoleh pemahaman menyeluruh tentang suatu penyakit tertentu tanpa harus melalui proses diagnosis terlebih dahulu. Informasi yang disajikan didasarkan pada pengetahuan medis yang telah divalidasi oleh pakar, sehingga dapat dijadikan referensi untuk edukasi kesehatan gigi dan mulut anak.

Berikut adalah implementasi kode pada *controller* untuk halaman detail info penyakit:

```
<?php
public function show($id)
{
    $penyakit = Penyakit::findOrFail($id);
    return view('pages.detailpenyakit', compact('penyakit'));
}
```

j. *Dashboard* Admin



Gambar 60. *Dashboard* Admin

Halaman *dashboard* admin merupakan halaman utama yang ditampilkan kepada administrator setelah melakukan *login* ke dalam sistem. Sebagaimana ditunjukkan pada Gambar 60, halaman ini menyajikan ringkasan statistik dan informasi penting mengenai aktivitas sistem secara keseluruhan dalam bentuk visualisasi yang informatif dan mudah dipahami. *Dashboard* menampilkan berbagai metrik kunci seperti total pengguna yang terdaftar dalam sistem,

jumlah pakar aktif yang telah disetujui, jumlah pakar yang masih menunggu persetujuan (*pending*), serta total pasien atau pengguna biasa. Selain itu, *dashboard* juga dilengkapi dengan grafik pendaftaran pengguna per bulan dalam 12 bulan terakhir yang memvisualisasikan tren pertumbuhan pengguna, serta diagram distribusi pengguna berdasarkan *role* (pasien, pakar, admin) untuk memberikan gambaran komposisi pengguna dalam sistem.

Bagian bawah *dashboard* menampilkan tabel riwayat *approval* terbaru yang memuat 5 pengajuan pakar terbaru beserta status persetujuannya (*pending, approved,* atau *rejected*). Informasi ini membantu administrator dalam memantau aktivitas pendaftaran pakar dan memudahkan proses persetujuan yang masih *pending*.

Berikut ini adalah implementasi kode pada *controller* untuk *dashboard* admin:

```
<?php
public function dashboard()
{
    $totalPengguna = Pengguna::count();

    $pakarAktif = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'approved')
        ->count();

    $pakarPending = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'pending')
        ->count();

    $totalPasien = Pengguna::where('role', 'user')->count();

    $pendaftaranPerBulan = Pengguna::select(
            DB::raw('MONTH(tanggal_daftar) as bulan'),
            DB::raw('YEAR(tanggal_daftar) as tahun'),
            DB::raw('COUNT(*) as total')
        )
        ->whereYear('tanggal_daftar', date('Y'))
        ->groupBy('tahun', 'bulan')
        ->orderBy('bulan')
        ->get()
        ->mapWithKeys(function ($item) {
            $namaBulan = [
```

```
                1 => 'Jan', 2 => 'Feb', 3 => 'Mar', 4 => 'Apr',
                5 => 'Mei', 6 => 'Jun', 7 => 'Jul', 8 => 'Agu',
                9 => 'Sep', 10 => 'Okt', 11 => 'Nov', 12 => 'Des'
            ];
            return [$namaBulan[$item->bulan] => $item->total];
        });

    $bulanLengkap = ['Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun',
'Jul', 'Agu', 'Sep', 'Okt', 'Nov', 'Des'];
    $grafikPendaftaran = [];
    foreach ($bulanLengkap as $bulan) {
        $grafikPendaftaran[$bulan] = $pendaftaranPerBulan-
>get($bulan, 0);
    }

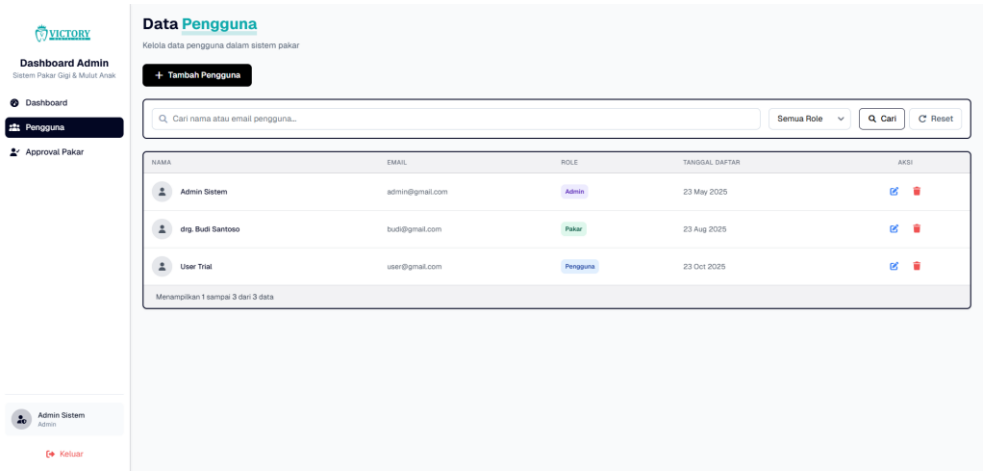
    $totalUser = Pengguna::where('role', 'user')->count();
    $totalPakarApproved = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'approved')
        ->count();
    $totalAdmin = Pengguna::where('role', 'admin')->count();

    $distribusiPengguna = [
        'Pasien' => $totalUser,
        'Pakar' => $totalPakarApproved,
        'Admin' => $totalAdmin
    ];

    $approvalTerbaru = Pengguna::where('role', 'pakar')
        ->orderBy('created_at', 'desc')
        ->limit(5)
        ->get();

    return view('admin.dashboard', compact(
        'totalPengguna',
        'pakarAktif',
        'pakarPending',
        'totalPasien',
        'grafikPendaftaran',
        'distribusiPengguna',
        'approvalTerbaru'
    ));
}
```

k. Halaman Daftar Data Pengguna



Gambar 61. Halaman Daftar Data Pengguna

Halaman daftar data pengguna merupakan halaman yang menyediakan fitur manajemen pengguna bagi administrator untuk mengelola seluruh akun yang terdaftar dalam sistem. Sebagaimana ditunjukkan pada Gambar 61, halaman ini menampilkan tabel daftar pengguna yang mencakup informasi nama lengkap, email, *role* (*user*/pakar/admin), dan tanggal pendaftaran. Halaman ini dilengkapi dengan berbagai fitur untuk memudahkan administrator dalam mengelola data pengguna, termasuk fitur pencarian berdasarkan nama atau email, *filter* berdasarkan *role*, serta tombol aksi untuk menambah pengguna baru, mengedit data pengguna, dan menghapus pengguna. Data ditampilkan dengan sistem *pagination* yang menampilkan 5 pengguna per halaman untuk menjaga keterbacaan dan performa halaman.

Tombol "Tambah Pengguna" memungkinkan administrator untuk menambahkan akun pengguna baru dengan *role user*, pakar, atau admin secara langsung tanpa melalui proses registrasi biasa. Setiap baris data pengguna memiliki tombol "Edit" untuk mengubah informasi pengguna seperti nama lengkap, email, dan *password*, serta tombol "Hapus" untuk menghapus akun

pengguna dari sistem. Sistem memiliki validasi khusus yang mencegah penghapusan admin terakhir untuk memastikan selalu ada minimal satu administrator yang dapat mengelola sistem.

Berikut adalah implementasi pada kode *controller* untuk halaman daftar data pengguna:

```
<?php
public function usersIndex(Request $request)
{
    $query = Pengguna::query();

    $query->where(function($q) {
        $q->where('role', '!=', 'pakar')
          ->orWhere(function($q2) {
              $q2->where('role', 'pakar')
                 ->where('status_approval', 'approved');
          });
    });

    if ($request->filled('search')) {
        $search = $request->search;
        $query->where(function($q) use ($search) {
            $q->where('nama_lengkap', 'like', '%' . $search . '%')
              ->orWhere('email', 'like', '%' . $search . '%');
        });
    }

    if ($request->filled('role')) {
        $query->where('role', $request->role);
    }

    $query->orderBy('created_at', 'desc');

    $users = $query->paginate(5)->withQueryString();

    return view('admin.users.index', compact('users'));
}

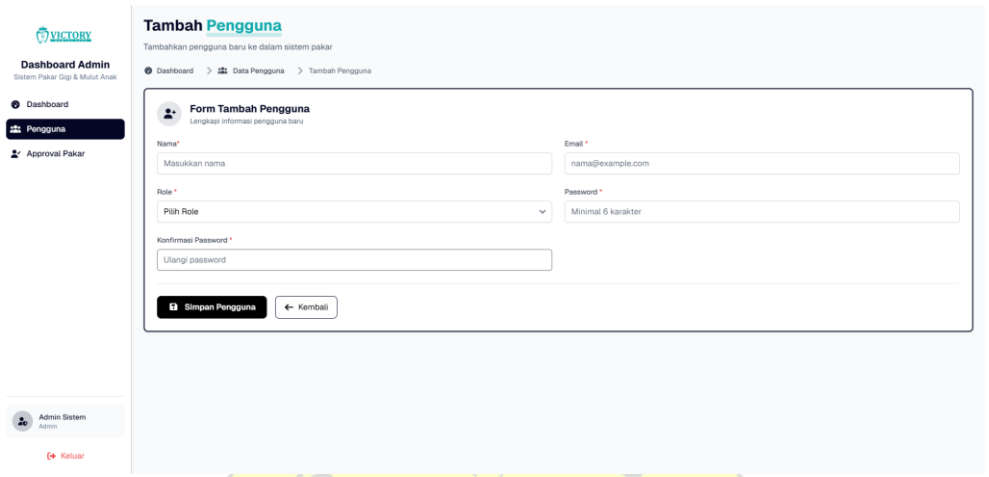
public function usersDestroy($id)
{
    $user = Pengguna::findOrFail($id);

    if ($user->role === 'admin') {
        $adminCount = Pengguna::where('role', 'admin')->count();
        if ($adminCount <= 1) {
            return redirect()->back()
                ->with('error', 'Tidak dapat menghapus admin
terakhir! Sistem harus memiliki minimal 1 admin.');
        }
    }
```

```
    $user->delete();

    return redirect()->route('admin.users.index')
        ->with('success', 'Pengguna berhasil dihapus!');
}
```

l. Halaman Tambah Data Pengguna



Gambar 62. Halaman Tambah Data Pengguna

Halaman tambah data pengguna merupakan halaman yang menyediakan formulir bagi administrator untuk menambahkan akun pengguna baru secara manual ke dalam sistem. Sebagaimana ditunjukkan pada Gambar 62, halaman ini menampilkan *form input* yang mencakup *field* nama lengkap, email, *password* dan konfirmasi *password*, serta *dropdown* untuk memilih *role* pengguna (*user*, pakar, atau admin). Fitur khusus pada halaman ini adalah *field* nomor STR (Surat Tanda Registrasi) yang akan muncul secara dinamis ketika administrator memilih *role* "pakar", memastikan setiap pakar yang ditambahkan memiliki kredensial profesional yang valid. Validasi dilakukan secara ketat pada setiap *field*, termasuk pengecekan keunikan email dan nomor STR untuk mencegah duplikasi data, serta validasi format dan panjang karakter sesuai standar yang ditetapkan.

Berikut adalah implementasi pada kode *controller* untuk halaman tambah data pengguna:

```
<?php
public function usersCreate()
{
    return view('admin.users.create');
}

public function usersStore(Request $request)
{
    $rules = [
        'nama_lengkap' => 'required|string|max:100',
        'email' => 'required|email|unique:pengguna,email|max:100',
        'password' => 'required|string|min:6|confirmed',
        'role' => 'required|in:user,pakar,admin',
    ];

    if ($request->role === 'pakar') {
        $rules['no_str_dokter'] =
'required|string|size:7|unique:pengguna,no_str_dokter';
    }

    $validated = $request->validate($rules);

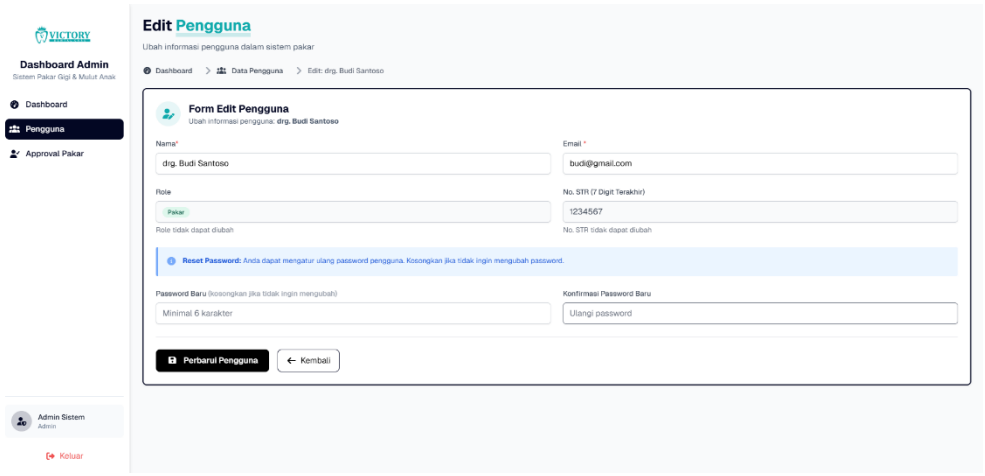
    $data = [
        'nama_lengkap' => $validated['nama_lengkap'],
        'email' => strtolower($validated['email']),
        'password' => bcrypt($validated['password']),
        'role' => $validated['role'],
        'tanggal_daftar' => now()->format('Y-m-d'),
    ];

    if ($validated['role'] === 'pakar') {
        $data['no_str_dokter'] = $validated['no_str_dokter'];
        $data['status_approval'] = 'pending';
    } else {
        $data['status_approval'] = 'approved';
    }

    Pengguna::create($data);

    return redirect()->route('admin.users.index')
        ->with('success', 'Pengguna berhasil ditambahkan!');
}
```

m. Halaman Edit Data Pengguna



Gambar 63. Halaman Edit Data Pengguna

Halaman edit data pengguna merupakan halaman yang menyediakan formulir bagi administrator untuk mengubah informasi akun pengguna yang sudah terdaftar dalam sistem. Sebagaimana ditunjukkan pada Gambar 63, halaman ini menampilkan *form* yang telah terisi dengan data pengguna saat ini, mencakup *field* nama lengkap, email, *password* baru, dan konfirmasi *password* baru. Berbeda dengan halaman tambah pengguna, pada halaman edit ini *field role* ditampilkan dalam bentuk *read-only* atau *disabled*, yang berarti administrator tidak dapat mengubah role pengguna yang sudah terdaftar untuk menjaga konsistensi hak akses dalam sistem. *Field password* bersifat opsional, sehingga administrator hanya perlu mengisi *field password* jika ingin mengubah *password* pengguna.

Berikut adalah implementasi kode pada *controller* untuk halaman edit data pengguna:

```php
<?php
public function usersEdit($id)
{
    $user = Pengguna::findOrFail($id);
    return view('admin.users.edit', compact('user'));
}

public function usersUpdate(Request $request, $id)
{
    $user = Pengguna::findOrFail($id);

    $rules = [
        'nama_lengkap' => 'required|string|max:100',
        'email' => 'required|email|max:100|unique:pengguna,email,'
. $id . ',id_pengguna',
    ];

    if ($request->filled('password')) {
        $rules['password'] = 'required|string|min:6|confirmed';
    }

    $validated = $request->validate($rules);

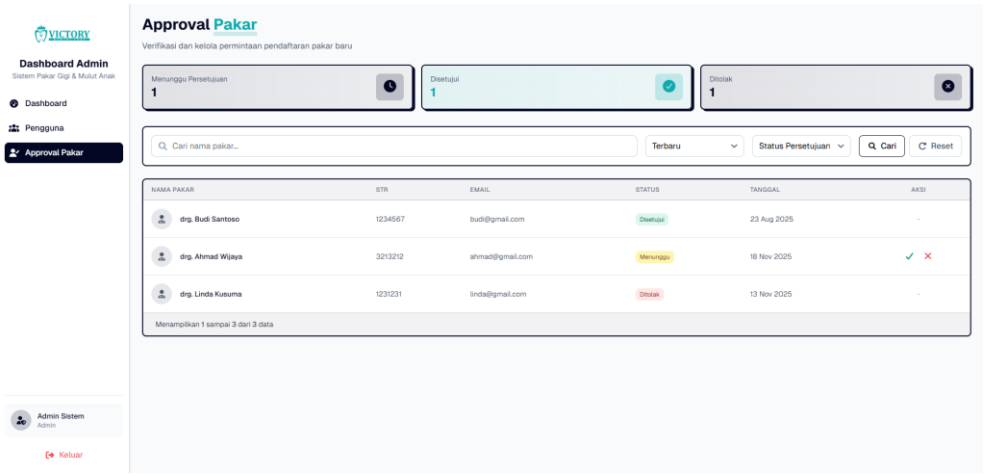
    $data = [
        'nama_lengkap' => $validated['nama_lengkap'],
        'email' => strtolower($validated['email']),
    ];

    if ($request->filled('password')) {
        $data['password'] = bcrypt($validated['password']);
    }

    $user->update($data);

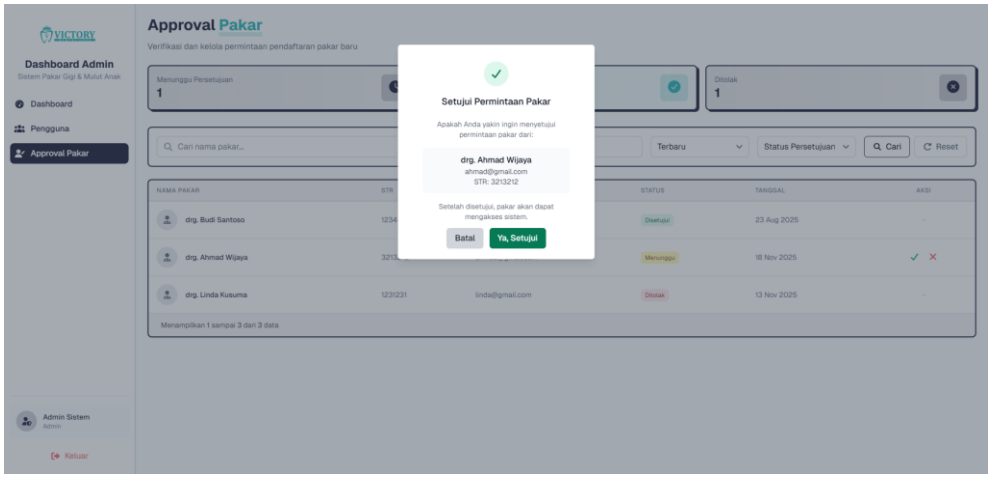
    return redirect()->route('admin.users.index')
        ->with('success', 'Pengguna berhasil diperbarui!');
}
```

n. Halaman Daftar Data *Approval* Pakar



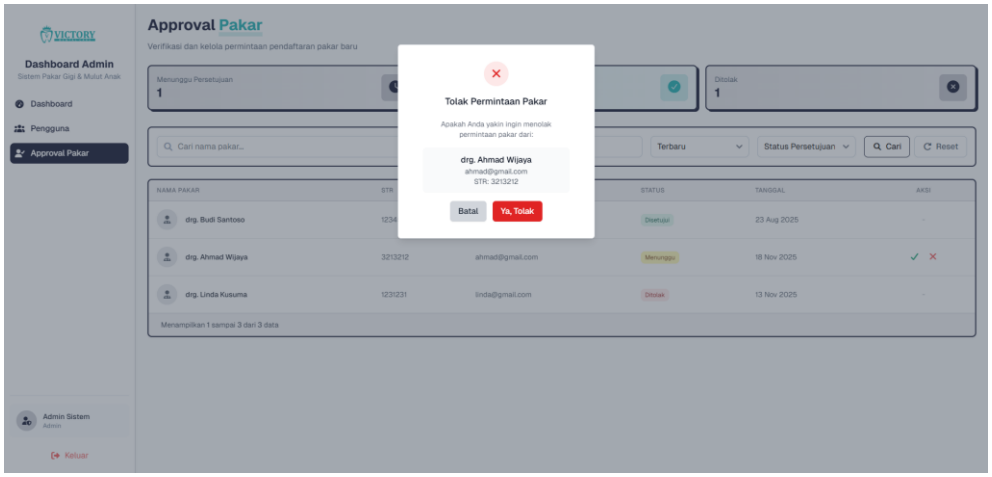
Gambar 64. Halaman Daftar Data *Approval* Pakar

Halaman daftar data *approval* pakar merupakan halaman khusus bagi administrator untuk mengelola persetujuan pendaftaran akun pakar yang masuk ke dalam sistem. Sebagaimana ditunjukkan pada Gambar 64, halaman ini menampilkan statistik ringkas berupa total pakar dengan status *pending* (menunggu persetujuan), *approved* (disetujui), dan *rejected* (ditolak) dalam bentuk *card* di bagian atas halaman. Bagian utama halaman menampilkan tabel daftar pakar yang mencakup informasi nama lengkap, email, nomor STR (Surat Tanda Registrasi), status *approval*, tanggal pendaftaran, serta tombol aksi untuk menyetujui atau menolak pendaftaran pakar. Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama, email, atau nomor STR, *filter* berdasarkan status *approval* (*pending, approved, rejected*), serta opsi pengurutan data berdasarkan tanggal pendaftaran terbaru atau terlama. Data ditampilkan dengan sistem *pagination* 5 *item* per halaman.



Gambar 65. Setujui Pakar

Administrator dapat melakukan dua aksi utama pada halaman ini. Pertama, tombol "Setujui" berwarna hijau yang ditampilkan pada pakar dengan status *pending*, sebagaimana ditunjukkan pada Gambar 65, yang akan mengubah status *approval* menjadi "*approved*" dan memberikan hak akses penuh kepada pakar untuk mengelola basis pengetahuan sistem. Kedua, tombol "Tolak" berwarna merah yang juga tersedia untuk pakar dengan status *pending*, sebagaimana ditunjukkan pada Gambar 66, yang akan mengubah status *approval* menjadi "*rejected*" sehingga pakar tidak dapat *login* dan mengakses sistem. Sistem menampilkan pesan konfirmasi sebelum melakukan aksi setujui atau tolak untuk mencegah kesalahan operasional, dan memberikan *feedback* berupa notifikasi sukses setelah aksi berhasil dilakukan.



Gambar 66. Tolak Pakar

Berikut adalah implementasi kode pada *controller* untuk halaman *approval* pakar:

```
<?php
public function approval(Request $request)
{
    $totalPending = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'pending')
        ->count();

    $totalApproved = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'approved')
        ->count();

    $totalRejected = Pengguna::where('role', 'pakar')
        ->where('status_approval', 'rejected')
        ->count();

    $query = Pengguna::where('role', 'pakar');

    if ($request->filled('search')) {
        $search = $request->search;
        $query->where(function($q) use ($search) {
            $q->where('nama_lengkap', 'like', '%' . $search . '%')
              ->orWhere('email', 'like', '%' . $search . '%')
              ->orWhere('no_str_dokter', 'like', '%' . $search .
'%');
        });
    }

    if ($request->filled('status')) {
        $query->where('status_approval', $request->status);
    }

    $sortOrder = $request->get('sort', 'newest') === 'oldest' ?
'asc' : 'desc';
    $query->orderBy('created_at', $sortOrder);
```

```
    $pakarList = $query->paginate(5)->withQueryString();

    return view('admin.approval', compact(
        'pakarList',
        'totalPending',
        'totalApproved',
        'totalRejected'
    ));
}

public function approvePakar($id)
{
    $pakar = Pengguna::where('role', 'pakar')->findOrFail($id);

    $pakar->update([
        'status_approval' => 'approved'
    ]);

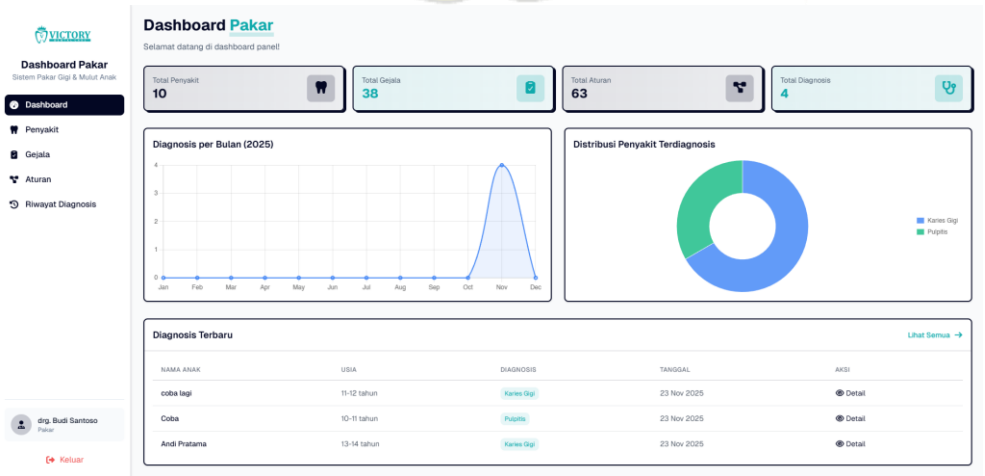
    return redirect()->route('admin.approval')
        ->with('success', 'Pakar ' . $pakar->nama_lengkap . '
berhasil disetujui!');
}

public function rejectPakar($id)
{
    $pakar = Pengguna::where('role', 'pakar')->findOrFail($id);

    $pakar->update([
        'status_approval' => 'rejected'
    ]);

    return redirect()->route('admin.approval')
        ->with('success', 'Pakar ' . $pakar->nama_lengkap . '
telah ditolak.');
}
```

o. *Dashboard* Pakar



Gambar 67. *Dashboard* Pakar

Halaman *dashboard* pakar merupakan halaman utama yang ditampilkan kepada pakar setelah melakukan *login* ke dalam sistem. Sebagaimana ditunjukkan pada Gambar 67, halaman ini menyajikan ringkasan statistik dan informasi penting mengenai basis pengetahuan sistem serta aktivitas diagnosis dalam bentuk visualisasi yang informatif dan mudah dipahami. *Dashboard* menampilkan berbagai metrik kunci seperti total penyakit yang terdaftar dalam sistem, total gejala yang telah diidentifikasi, total aturan diagnosis yang telah dibuat, serta total diagnosis yang telah dilakukan oleh pengguna.

Bagian tengah *dashboard* menampilkan grafik diagnosis per bulan dalam 12 bulan terakhir tahun berjalan yang memvisualisasikan tren aktivitas penggunaan sistem diagnosis oleh pengguna. Grafik ini membantu pakar dalam memahami pola penggunaan sistem dan mengidentifikasi periode dengan aktivitas tinggi atau rendah. Di samping itu, terdapat diagram distribusi penyakit terdiagnosis yang menampilkan 10 penyakit yang paling sering teridentifikasi oleh sistem, memberikan *insight* kepada pakar mengenai prevalensi berbagai jenis penyakit gigi dan mulut pada anak berdasarkan data diagnosis yang terkumpul. Bagian bawah *dashboard* menampilkan tabel diagnosis terbaru yang memuat 3 diagnosis terakhir beserta hasil penyakit yang teridentifikasi, memungkinkan pakar untuk memantau aktivitas diagnosis terkini dan melakukan *review* jika diperlukan.

Berikut ini adalah implementasi kode pada *controller* untuk *dashboard* pakar:

```php
<?php
public function dashboard()
{
    $totalPenyakit = Penyakit::count();

    $totalGejala = Gejala::count();

    $totalAturan = Aturan::count();

    $totalDiagnosis = Diagnosis::count();

    $diagnosisPerBulan = Diagnosis::select(
            DB::raw('MONTH(created_at) as bulan'),
            DB::raw('YEAR(created_at) as tahun'),
            DB::raw('COUNT(*) as total')
        )
        ->whereYear('created_at', date('Y'))
        ->groupBy('tahun', 'bulan')
        ->orderBy('bulan')
        ->get()
        ->mapWithKeys(function ($item) {
            $namaBulan = [
                1 => 'Jan', 2 => 'Feb', 3 => 'Mar', 4 => 'Apr',
                5 => 'May', 6 => 'Jun', 7 => 'Jul', 8 => 'Aug',
                9 => 'Sep', 10 => 'Oct', 11 => 'Nov', 12 => 'Dec'
            ];
            return [$namaBulan[$item->bulan] => $item->total];
        });

    $bulanLengkap = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
    $grafikDiagnosis = [];
    foreach ($bulanLengkap as $bulan) {
        $grafikDiagnosis[$bulan] = $diagnosisPerBulan->get($bulan,
0);
    }

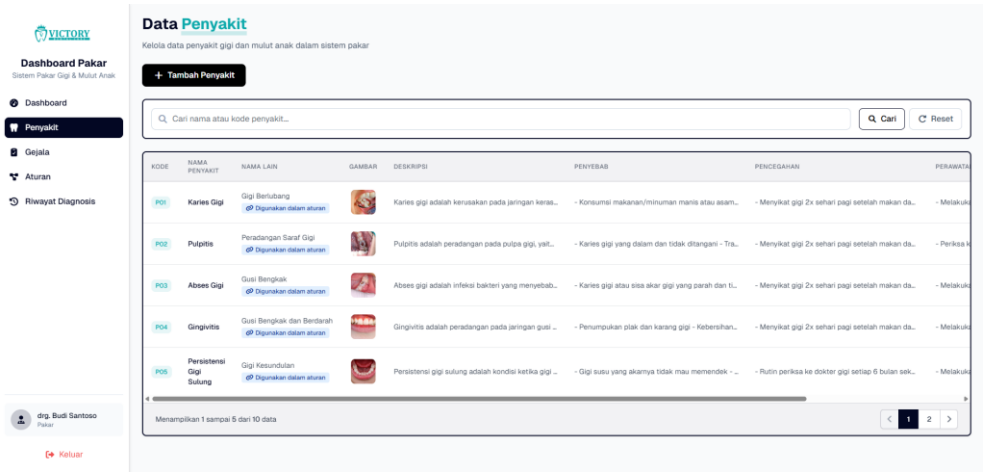
    $distribusiPenyakit =
HasilDiagnosis::select('penyakit.nama_penyakit', DB::raw('COUNT(*)
as total'))
        ->join('penyakit', 'hasil_diagnosis.id_penyakit', '=',
'penyakit.id_penyakit')
        ->whereNotNull('hasil_diagnosis.id_penyakit')
        ->groupBy('hasil_diagnosis.id_penyakit',
'penyakit.nama_penyakit')
        ->orderBy('total', 'desc')
        ->limit(10)
        ->get()
        ->mapWithKeys(function ($item) {
            return [$item->nama_penyakit => $item->total];
        });

    $diagnosisTerbaru =
Diagnosis::with(['hasilDiagnosis.penyakit'])
        ->whereHas('hasilDiagnosis')
        ->orderBy('created_at', 'desc')
        ->limit(5)
```

```
        ->get();

    return view('pakar.dashboard', compact(
        'totalPenyakit',
        'totalGejala',
        'totalAturan',
        'totalDiagnosis',
        'grafikDiagnosis',
        'distribusiPenyakit',
        'diagnosisTerbaru'
    ));
}
```

p. Halaman Daftar Data Penyakit



Gambar 68. Halaman Daftar Data Penyakit

Halaman daftar data penyakit merupakan halaman yang menyediakan fitur manajemen basis pengetahuan penyakit bagi pakar untuk mengelola seluruh data penyakit gigi dan mulut pada anak yang ada dalam sistem. Sebagaimana ditunjukkan pada Gambar 68, halaman ini menampilkan tabel daftar penyakit yang mencakup informasi kode penyakit, nama penyakit, nama lain, dan deskripsi penyakit. Halaman ini dilengkapi dengan berbagai fitur untuk memudahkan pakar dalam mengelola basis pengetahuan, termasuk fitur pencarian berdasarkan kode atau nama penyakit, tombol aksi untuk menambah penyakit baru, mengedit data penyakit yang sudah ada, dan menghapus penyakit

dari sistem. Data ditampilkan dengan sistem *pagination* yang menampilkan 5 penyakit per halaman.

Tombol "Tambah Penyakit" memungkinkan pakar untuk menambahkan jenis penyakit baru ke dalam basis pengetahuan sistem dengan mengisi informasi lengkap meliputi kode penyakit, nama penyakit, nama lain, deskripsi, penyebab, dan solusi penanganan. Setiap baris data penyakit memiliki tombol "Edit" untuk mengubah informasi penyakit yang sudah ada, serta tombol "Hapus" untuk menghapus penyakit dari sistem. Sistem memiliki validasi yang mencegah penghapusan penyakit yang masih memiliki relasi dengan tabel aturan diagnosis atau riwayat diagnosis, untuk menjaga integritas data dan konsistensi basis pengetahuan.

Berikut adalah implementasi kode pada *controller* untuk halaman daftar data penyakit:

```
<?php
public function index(Request $request)
{
    $search = $request->get('search');

    $penyakits = Penyakit::query()
        ->when($search, function ($query, $search) {
            return $query->where('nama_penyakit', 'like',
"%{$search}%")
                ->orWhere('kode_penyakit', 'like', "%{$search}%");
        })
        ->orderBy('created_at', 'asc')
        ->paginate(5);

    return view('pakar.penyakit.index', compact('penyakits'));
}

public function destroy($id)
{
    $penyakit = Penyakit::findOrFail($id);

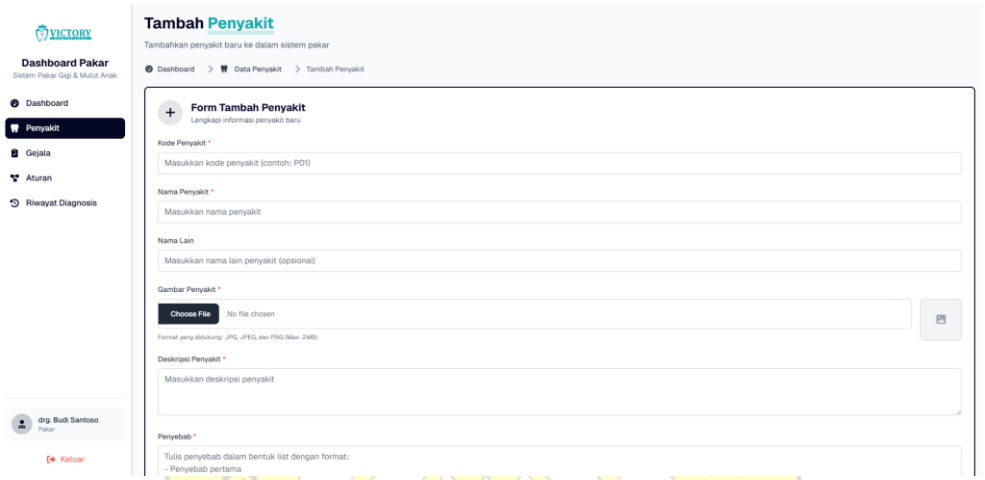
    if ($penyakit->aturan()->exists()) {
        return redirect()->route('pakar.penyakit.index')
            ->with('error', 'Penyakit tidak dapat dihapus karena
sudah digunakan dalam aturan diagnosis');
    }
```

```
    if ($penyakit->gambar && Storage::disk('public')-
>exists($penyakit->gambar)) {
        Storage::disk('public')->delete($penyakit->gambar);
    }

    $penyakit->delete();

    return redirect()->route('pakar.penyakit.index')
        ->with('success', 'Penyakit berhasil dihapus');
}
```

q. Halaman Tambah Data Penyakit



Gambar 69. Halaman Tambah Data Penyakit

Halaman tambah data penyakit merupakan halaman yang menyediakan formulir bagi pakar untuk menambahkan jenis penyakit baru ke dalam basis pengetahuan sistem. Sebagaimana ditunjukkan pada Gambar 69, halaman ini menampilkan *form input* yang mencakup *field* kode penyakit, nama penyakit, nama lain, *upload* gambar penyakit, deskripsi, penyebab, pencegahan, dan perawatan. *Field* kode penyakit harus unik dan otomatis dikonversi ke huruf kapital (*uppercase*) untuk menjaga konsistensi format penamaan dalam sistem. *Field* gambar bersifat wajib dan hanya menerima file dengan format JPEG, JPG, atau PNG dengan ukuran maksimal 2MB, di mana gambar akan disimpan dengan nama file sesuai kode penyakit untuk memudahkan identifikasi.

Berikut adalah implementasi kode pada *controller* untuk halaman tambah data penyakit:

```
<?php
public function create()
{
    return view('pakar.penyakit.create');
}

public function store(Request $request)
{
    $request->validate([
        'kode_penyakit' =>
'required|string|max:10|unique:penyakit,kode_penyakit',
        'nama_penyakit' => 'required|string|max:255',
        'nama_lain' => 'nullable|string|max:255',
        'gambar' => 'required|image|mimes:jpeg,jpg,png|max:2048',
        'deskripsi' => 'required|string',
        'penyebab' => 'required|string',
        'pencegahan' => 'required|string',
        'perawatan' => 'required|string'
    ], [
        'kode_penyakit.required' => 'Kode penyakit harus diisi',
        'kode_penyakit.unique' => 'Kode penyakit sudah digunakan',
        'nama_penyakit.required' => 'Nama penyakit harus diisi',
        'gambar.required' => 'Gambar penyakit harus diupload',
        'gambar.image' => 'File harus berupa gambar',
        'gambar.mimes' => 'Gambar harus berformat jpeg, jpg, atau
png',
        'gambar.max' => 'Ukuran gambar maksimal 2MB',
        'deskripsi.required' => 'Deskripsi harus diisi',
        'penyebab.required' => 'Penyebab harus diisi',
        'pencegahan.required' => 'Pencegahan harus diisi',
        'perawatan.required' => 'Perawatan harus diisi'
    ]);

    $kodePenyakit = strtoupper($request->kode_penyakit);

    $gambarPath = null;
    if ($request->hasFile('gambar')) {
        $gambar = $request->file('gambar');
        $fileName = $kodePenyakit . '.' . $gambar-
>getClientOriginalExtension();

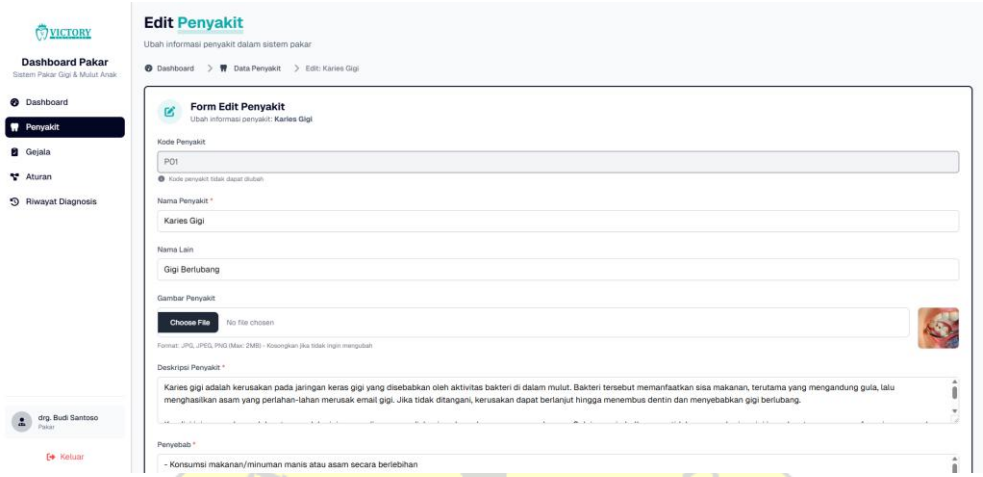
        $path = $gambar->storeAs('penyakit', $fileName, 'public');
        $gambarPath = $path;
    }

    Penyakit::create([
        'kode_penyakit' => $kodePenyakit,
        'nama_penyakit' => $request->nama_penyakit,
        'nama_lain' => $request->nama_lain,
        'gambar' => $gambarPath,
        'deskripsi' => $request->deskripsi,
        'penyebab' => $request->penyebab,
```

```
        'pencegahan' => $request->pencegahan,
        'perawatan' => $request->perawatan
    ]);

    return redirect()->route('pakar.penyakit.index')
        ->with('success', 'Penyakit berhasil ditambahkan');
}
```

r. Halaman Edit Data Penyakit



Gambar 70. Halaman Edit Data Penyakit

Halaman edit data penyakit merupakan halaman yang menyediakan formulir bagi pakar untuk mengubah informasi penyakit yang sudah terdaftar dalam basis pengetahuan sistem. Sebagaimana ditunjukkan pada Gambar 70, halaman ini menampilkan *form* yang telah terisi dengan data penyakit saat ini, mencakup *field* kode penyakit (*read-only/disabled*), nama penyakit, nama lain, gambar penyakit saat ini dengan *preview*, upload gambar baru (opsional), deskripsi, penyebab, pencegahan, dan perawatan.

Berikut adalah implementasi kode pada *controller* untuk halaman edit data penyakit:

```
<?php
public function edit($id)
{
    $penyakit = Penyakit::findOrFail($id);
    return view('pakar.penyakit.edit', compact('penyakit'));
}
```

```
public function update(Request $request, $id)
{
    $penyakit = Penyakit::findOrFail($id);

    $request->validate([
        'nama_penyakit' => 'required|string|max:255',
        'nama_lain' => 'nullable|string|max:255',
        'gambar' => 'nullable|image|mimes:jpeg,jpg,png|max:2048',
        'deskripsi' => 'required|string',
        'penyebab' => 'required|string',
        'pencegahan' => 'required|string',
        'perawatan' => 'required|string'
    ], [
        'nama_penyakit.required' => 'Nama penyakit harus diisi',
        'gambar.image' => 'File harus berupa gambar',
        'gambar.mimes' => 'Gambar harus berformat jpeg, jpg, atau
png',
        'gambar.max' => 'Ukuran gambar maksimal 2MB',
        'deskripsi.required' => 'Deskripsi harus diisi',
        'penyebab.required' => 'Penyebab harus diisi',
        'pencegahan.required' => 'Pencegahan harus diisi',
        'perawatan.required' => 'Perawatan harus diisi'
    ]);

    $updateData = [
        'nama_penyakit' => $request->nama_penyakit,
        'nama_lain' => $request->nama_lain,
        'deskripsi' => $request->deskripsi,
        'penyebab' => $request->penyebab,
        'pencegahan' => $request->pencegahan,
        'perawatan' => $request->perawatan
    ];

    if ($request->hasFile('gambar')) {
        if ($penyakit->gambar && Storage::disk('public')-
>exists($penyakit->gambar)) {
            Storage::disk('public')->delete($penyakit->gambar);
        }

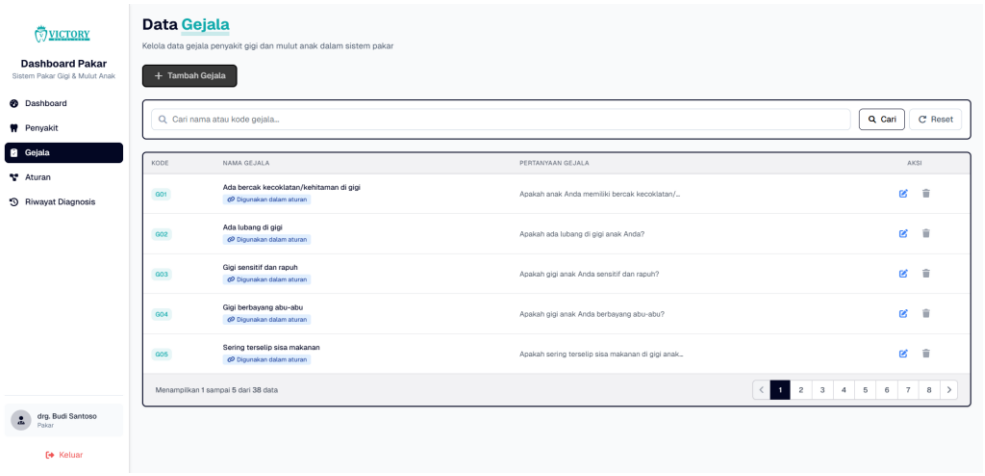
        $gambar = $request->file('gambar');
        $fileName = $penyakit->kode_penyakit . '.' . $gambar-
>getClientOriginalExtension();

        $path = $gambar->storeAs('penyakit', $fileName, 'public');
        $updateData['gambar'] = $path;
    }

    $penyakit->update($updateData);

    return redirect()->route('pakar.penyakit.index')
        ->with('success', 'Penyakit berhasil diperbarui');
}
```

s. Halaman Daftar Data Gejala



Gambar 71. Halaman Daftar Data Gejala

Halaman daftar data gejala merupakan halaman yang menyediakan fitur manajemen basis pengetahuan gejala bagi pakar untuk mengelola seluruh data gejala penyakit gigi dan mulut pada anak yang ada dalam sistem. Sebagaimana ditunjukkan pada Gambar 71, halaman ini menampilkan tabel daftar gejala yang mencakup informasi kode gejala, nama gejala, dan pertanyaan gejala yang akan ditampilkan kepada pengguna saat proses diagnosis. Halaman ini dilengkapi dengan berbagai fitur untuk memudahkan pakar dalam mengelola basis pengetahuan, termasuk fitur pencarian berdasarkan kode atau nama gejala, tombol aksi untuk menambah gejala baru, mengedit data gejala yang sudah ada, dan menghapus gejala dari sistem. Data ditampilkan dengan sistem *pagination* yang menampilkan 5 gejala per halaman.

Tombol "Tambah Gejala" memungkinkan pakar untuk menambahkan jenis gejala baru ke dalam basis pengetahuan sistem dengan mengisi informasi meliputi kode gejala, nama gejala, dan pertanyaan gejala yang akan diajukan kepada pengguna. Setiap baris data gejala memiliki tombol "Edit" untuk mengubah informasi gejala yang sudah ada, serta tombol "Hapus" untuk

menghapus gejala dari sistem. Sistem memiliki validasi yang mencegah penghapusan gejala yang masih memiliki relasi dengan tabel aturan diagnosis, untuk menjaga integritas data dan konsistensi basis pengetahuan.

Berikut adalah implementasi kode pada *controller* untuk halaman daftar data gejala:

```
<?php
public function index(Request $request)
{
    $search = $request->get('search');

    $gejalas = Gejala::query()
        ->when($search, function ($query, $search) {
            return $query->where('nama_gejala', 'like',
"%{$search}%")
                ->orWhere('kode_gejala', 'like', "%{$search}%");
        })
        ->orderBy('created_at', 'asc')
        ->paginate(5);

    return view('pakar.gejala.index', compact('gejalas'));
}

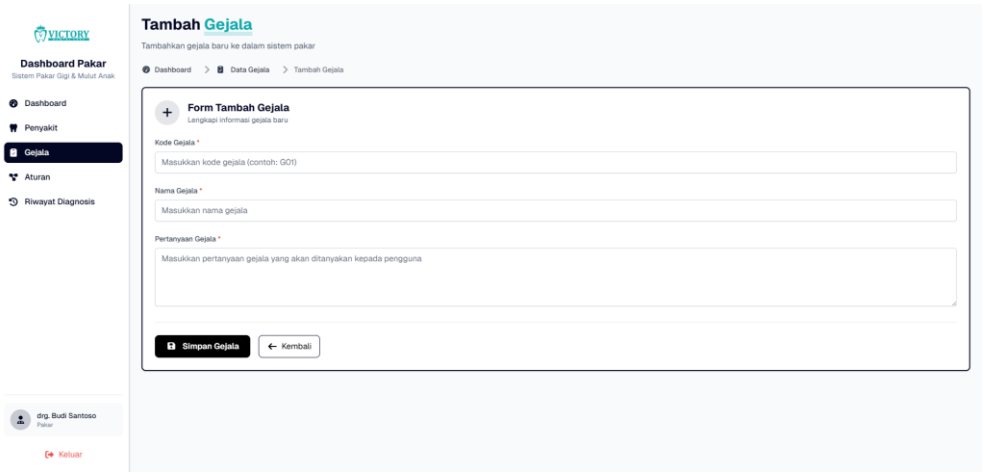
public function destroy($id)
{
    $gejala = Gejala::findOrFail($id);

    if ($gejala->aturan()->exists()) {
        return redirect()->route('pakar.gejala.index')
            ->with('error', 'Gejala tidak dapat dihapus karena
sudah digunakan dalam aturan diagnosis');
    }

    $gejala->delete();

    return redirect()->route('pakar.gejala.index')
        ->with('success', 'Gejala berhasil dihapus');
}
```

t. Halaman Tambah Data Gejala



Gambar 72. Halaman Tambah Data Gejala

Halaman tambah data gejala merupakan halaman yang menyediakan formulir bagi pakar untuk menambahkan jenis gejala baru ke dalam basis pengetahuan sistem. Sebagaimana ditunjukkan pada Gambar 72, halaman ini menampilkan *form input* yang mencakup tiga *field* utama yaitu kode gejala, nama gejala, dan pertanyaan gejala. *Field* kode gejala harus unik dan akan otomatis dikonversi ke huruf kapital (*uppercase*) untuk menjaga konsistensi format penamaan dalam sistem, mengikuti pola penamaan seperti G01, G02, dan seterusnya. *Field* nama gejala berisi deskripsi singkat dari gejala yang akan ditambahkan, sedangkan *field* pertanyaan gejala berisi kalimat tanya yang akan ditampilkan kepada pengguna saat proses diagnosis berlangsung.

Berikut adalah implementasi kode pada *controller* untuk halaman tambah data gejala:

```
<?php
public function create()
{
    return view('pakar.gejala.create');
}

public function store(Request $request)
{
```

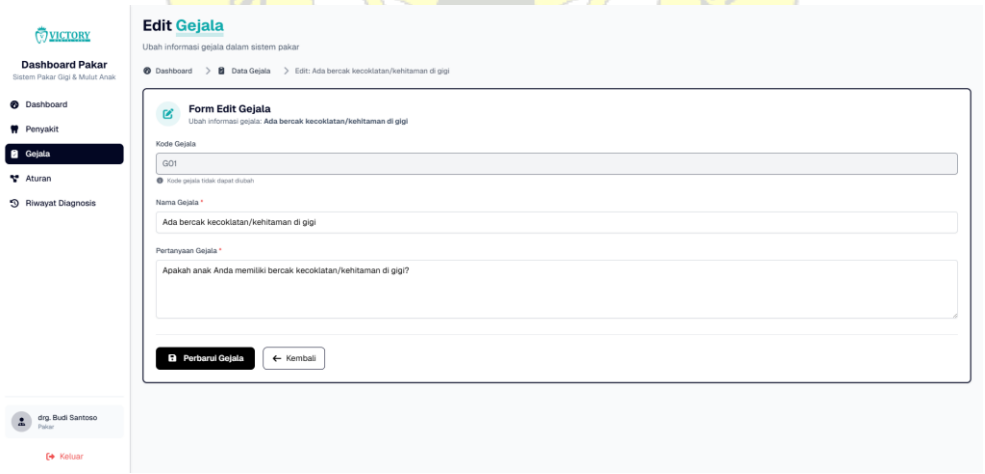
```
    $request->validate([
        'kode_gejala' =>
'required|string|max:10|unique:gejala,kode_gejala',
        'nama_gejala' => 'required|string|max:255',
        'pertanyaan_gejala' => 'required|string'
    ], [
        'kode_gejala.required' => 'Kode gejala harus diisi',
        'kode_gejala.unique' => 'Kode gejala sudah digunakan',
        'nama_gejala.required' => 'Nama gejala harus diisi',
        'pertanyaan_gejala.required' => 'Pertanyaan gejala harus
diisi'
    ]);

    $kodeGejala = strtoupper($request->kode_gejala);

    Gejala::create([
        'kode_gejala' => $kodeGejala,
        'nama_gejala' => $request->nama_gejala,
        'pertanyaan_gejala' => $request->pertanyaan_gejala
    ]);

    return redirect()->route('pakar.gejala.index')
        ->with('success', 'Gejala berhasil ditambahkan');
}
```

u. Halaman Edit Data Gejala



Gambar 73. Halaman Edit Data Gejala

Halaman edit data gejala merupakan halaman yang menyediakan formulir bagi pakar untuk mengubah informasi gejala yang sudah terdaftar dalam basis pengetahuan sistem. Sebagaimana ditunjukkan pada Gambar 73, halaman ini menampilkan *form* yang telah terisi dengan data gejala saat ini, mencakup *field* kode gejala (*read-only/disabled*), nama gejala, dan pertanyaan gejala. Berbeda

dengan halaman tambah gejala, pada halaman edit ini *field* kode gejala tidak dapat diubah untuk menjaga konsistensi referensi data dalam sistem, terutama karena kode gejala digunakan sebagai *identifier* dalam tabel aturan diagnosis. *Field* yang dapat diubah hanya nama gejala dan pertanyaan gejala, memberikan fleksibilitas kepada pakar untuk memperbaiki deskripsi atau meningkatkan kualitas pertanyaan yang diajukan kepada pengguna tanpa mengganggu struktur relasi data yang sudah ada.

Berikut adalah implementasi kode pada *controller* untuk halaman edit data gejala:

```
<?php
public function edit($id)
{
    $gejala = Gejala::findOrFail($id);
    return view('pakar.gejala.edit', compact('gejala'));
}

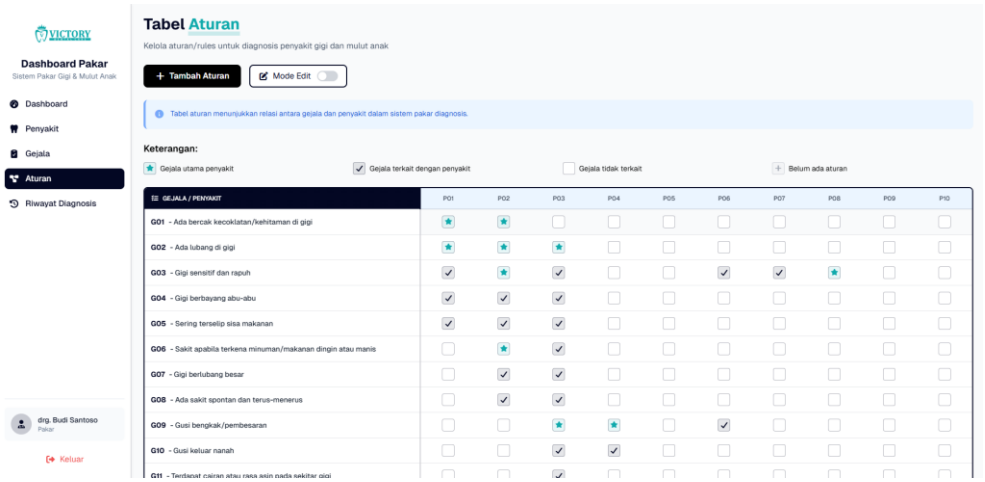
public function update(Request $request, $id)
{
    $gejala = Gejala::findOrFail($id);

    $request->validate([
        'nama_gejala' => 'required|string|max:255',
        'pertanyaan_gejala' => 'required|string'
    ], [
        'nama_gejala.required' => 'Nama gejala harus diisi',
        'pertanyaan_gejala.required' => 'Pertanyaan gejala harus
diisi'
    ]);

pertanyaan_gejala
    $gejala->update([
        'nama_gejala' => $request->nama_gejala,
        'pertanyaan_gejala' => $request->pertanyaan_gejala
    ]);

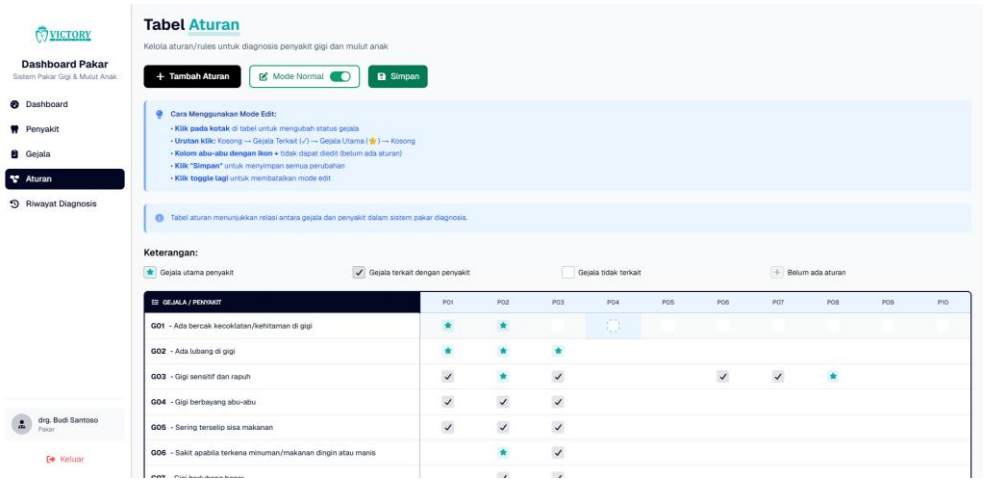
    return redirect()->route('pakar.gejala.index')
        ->with('success', 'Gejala berhasil diperbarui');
}
```

v. Halaman Daftar Data Aturan



Gambar 74. Halaman Daftar Data Aturan

Halaman daftar data aturan merupakan halaman yang menyediakan fitur manajemen basis pengetahuan aturan diagnosis bagi pakar untuk mengelola hubungan antara penyakit dan gejala-gejalanya dalam bentuk matriks 2x2. Sebagaimana ditunjukkan pada Gambar 74, halaman ini menampilkan tabel matriks yang menghubungkan penyakit (kolom) dengan gejala (baris), di mana setiap sel menunjukkan apakah suatu gejala terkait dengan penyakit tertentu atau tidak. Matriks ini menggunakan sistem penandaan visual dengan bintang berwarna hijau untuk gejala utama (*AND logic*) dan centang untuk gejala terkait (*OR logic*). Gejala utama adalah gejala yang harus ada (*mandatory*) untuk mendiagnosis suatu penyakit, sedangkan gejala terkait adalah gejala opsional yang dapat membantu memperkuat diagnosis namun tidak wajib semua dialami. Halaman ini dilengkapi dengan tombol "Tambah Aturan" untuk menambahkan aturan baru bagi penyakit yang belum memiliki aturan, serta tombol "Mode Edit" yang mengaktifkan mode *editing inline* untuk mengubah aturan yang sudah ada.



Gambar 75. Mode Edit Data Aturan

Sebagaimana ditunjukkan pada Gambar 75, ketika mode edit diaktifkan, setiap sel pada matriks berubah menjadi *checkbox* yang memungkinkan pakar untuk mengubah status hubungan antara penyakit dan gejala. Pakar dapat mengubah gejala dari tidak terkait menjadi gejala utama (*AND*), menjadi gejala terkait (*OR*), atau sebaliknya. Perubahan dapat dilakukan pada *multiple* penyakit sekaligus, dan sistem akan menampilkan indikator visual untuk sel yang telah dimodifikasi. Setelah semua perubahan dilakukan, pakar dapat menekan tombol "Simpan" untuk menyimpan perubahan, atau "Mode Normal" untuk membatalkan seluruh perubahan dan kembali ke mode *view*. Sistem ini menggunakan pendekatan *delete-and-insert* untuk update aturan, di mana aturan lama dihapus dan aturan baru dibuat dengan ID yang berbeda. Pendekatan ini aman karena ID aturan tidak direferensi oleh tabel lain, dan yang penting adalah konsistensi relasi antara id_penyakit, id_gejala, dan group_id.

Berikut adalah implementasi kode pada *controller* untuk halaman daftar data aturan:

```
<?php
public function index()
{
    $penyakits = Penyakit::orderBy('kode_penyakit')->get();
    $gejalas = Gejala::orderBy('kode_gejala')->get();

    $aturans = Aturan::with(['penyakit', 'gejala'])
        ->orderBy('id_penyakit')
        ->orderBy('group_id')
        ->get();

    $matrixData = [];
    foreach ($penyakits as $penyakit) {
        $penyakitAturans = $aturans->where('id_penyakit',
$penyakit->id_penyakit);

        $groups = $penyakitAturans->groupBy('group_id');

        $andGejalaIds = [];
        $orGejalaIds = [];

        foreach ($groups as $groupId => $groupAturans) {
            if ($groupAturans->count() == 1) {
                $andGejalaIds[] = $groupAturans->first()-
>id_gejala;
            } else {
                foreach ($groupAturans as $aturan) {
                    $orGejalaIds[] = $aturan->id_gejala;
                }
            }
        }

        $matrixData[$penyakit->id_penyakit] = [
            'and' => $andGejalaIds,
            'or' => $orGejalaIds,
            'has_aturan' => $penyakitAturans->isNotEmpty()
        ];
    }

    return view('pakar.aturan.index', compact('penyakits',
'gejalas', 'matrixData'));
}

public function bulkUpdate(Request $request)
{
    $request->validate([
        'changes' => 'required|array',
        'changes.*.penyakit_id' =>
'required|exists:penyakit,id_penyakit',
        'changes.*.gejala_utama' => 'array',
        'changes.*.gejala_terkait' => 'array'
    ]);

    try {
        DB::beginTransaction();

        foreach ($request->changes as $change) {
```

```
            $penyakitId = $change['penyakit_id'];
            $gejalaUtama = $change['gejala_utama'] ?? [];
            $gejalaTerkait = $change['gejala_terkait'] ?? [];

            Aturan::where('id_penyakit', $penyakitId)->delete();

            $groupId = 1;

            foreach ($gejalaUtama as $gejalaId) {
                Aturan::create([
                    'id_penyakit' => $penyakitId,
                    'id_gejala' => $gejalaId,
                    'group_id' => $groupId
                ]);
                $groupId++;
            }

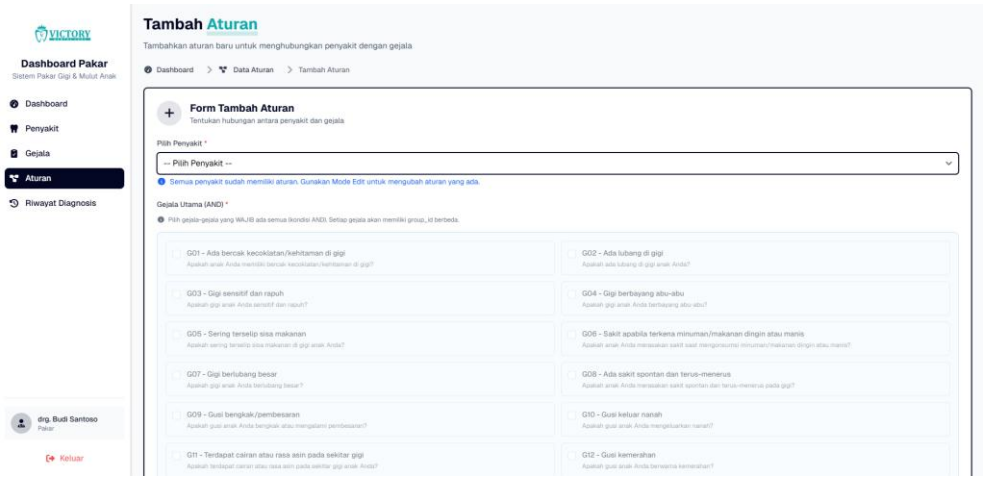
            if (!empty($gejalaTerkait)) {
                foreach ($gejalaTerkait as $gejalaId) {
                    Aturan::create([
                        'id_penyakit' => $penyakitId,
                        'id_gejala' => $gejalaId,
                        'group_id' => $groupId
                    ]);
                }
            }
        }

        DB::commit();

        return response()->json([
            'success' => true,
            'message' => 'Semua perubahan berhasil disimpan!'
        ]);

    } catch (\Exception $e) {
        DB::rollBack();
        return response()->json([
            'success' => false,
            'message' => 'Gagal menyimpan perubahan: ' . $e-
>getMessage()
        ], 500);
    }
}
```

w. Halaman Tambah Data Aturan



Gambar 76. Halaman Tambah Data Aturan

Halaman tambah data aturan merupakan halaman yang menyediakan formulir bagi pakar untuk menambahkan aturan diagnosis baru yang menghubungkan suatu penyakit dengan gejala-gejalanya. Sebagaimana ditunjukkan pada Gambar 76, halaman ini menampilkan *form* yang terdiri dari *dropdown* pemilihan penyakit yang belum memiliki aturan, serta dua bagian pemilihan gejala yaitu gejala utama (*AND logic*) dan gejala terkait (*OR logic*). Gejala utama adalah gejala-gejala yang harus ada atau bersifat *mandatory* untuk mendiagnosis suatu penyakit, di mana jika salah satu gejala utama tidak dialami maka penyakit tersebut tidak akan terdiagnosis. Gejala terkait adalah gejala-gejala opsional yang dapat membantu memperkuat diagnosis, di mana minimal salah satu dari gejala terkait harus dialami namun tidak harus semuanya.

Dropdown pemilihan penyakit hanya menampilkan penyakit yang belum memiliki aturan diagnosis. Hal ini mencegah duplikasi aturan untuk satu penyakit, karena setiap penyakit hanya boleh memiliki satu set aturan yang lengkap. Bagian pemilihan gejala menggunakan *checkbox* dengan *multiple select*, memungkinkan pakar untuk memilih beberapa gejala sekaligus untuk

kategori *AND* dan *OR*. Minimal satu gejala utama (*AND*) harus dipilih, sedangkan gejala terkait (*OR*) bersifat opsional.

Berikut adalah implementasi kode pada *controller* untuk halaman tambah data aturan:

```
<?php
public function create()
{
    $penyakitWithAturan = Aturan::distinct()-
>pluck('id_penyakit');
    $penyakits = Penyakit::whereNotIn('id_penyakit',
$penyakitWithAturan)
        ->orderBy('kode_penyakit')
        ->get();

    $gejalas = Gejala::orderBy('kode_gejala')->get();

    return view('pakar.aturan.create', compact('penyakits',
'gejalas'));
}

public function store(Request $request)
{
    $request->validate([
        'id_penyakit' => 'required|exists:penyakit,id_penyakit',
        'gejala_and' => 'required|array|min:1',
        'gejala_and.*' => 'exists:gejala,id_gejala',
        'gejala_or' => 'nullable|array',
        'gejala_or.*' => 'exists:gejala,id_gejala'
    ], [
        'id_penyakit.required' => 'Penyakit harus dipilih',
        'id_penyakit.exists' => 'Penyakit tidak valid',
        'gejala_and.required' => 'Minimal satu gejala utama (AND)
harus dipilih',
        'gejala_and.min' => 'Minimal satu gejala utama (AND) harus
dipilih',
        'gejala_and.*.exists' => 'Gejala tidak valid',
        'gejala_or.*.exists' => 'Gejala tidak valid'
    ]);

    try {
        DB::beginTransaction();

        $groupId = 1;

        foreach ($request->gejala_and as $idGejala) {
            Aturan::create([
                'id_penyakit' => $request->id_penyakit,
                'id_gejala' => $idGejala,
                'group_id' => $groupId
            ]);
            $groupId++;
```

```
        }

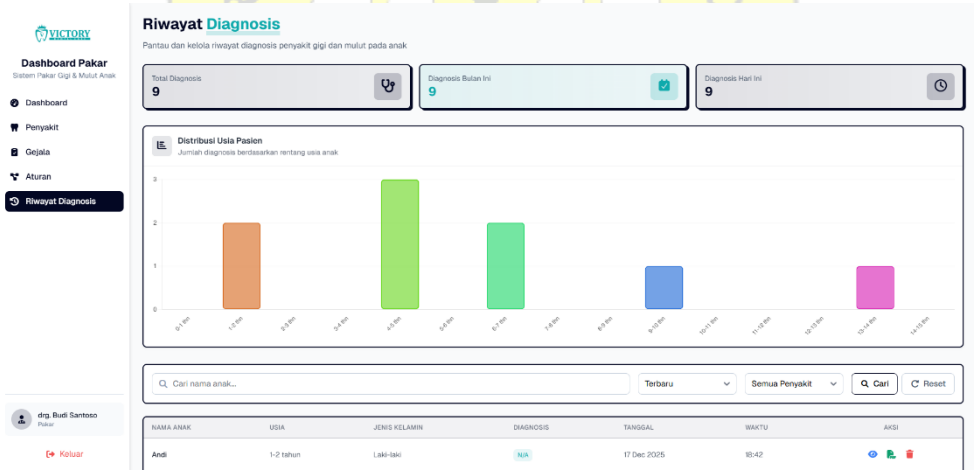
        if (!empty($request->gejala_or)) {
            foreach ($request->gejala_or as $idGejala) {
                Aturan::create([
                    'id_penyakit' => $request->id_penyakit,
                    'id_gejala' => $idGejala,
                    'group_id' => $groupId
                ]);
            }
        }

        DB::commit();

        return redirect()->route('pakar.aturan.index')
            ->with('success', 'Aturan berhasil ditambahkan');

    } catch (\Exception $e) {
        DB::rollBack();
        return redirect()->back()
            ->withInput()
            ->with('error', 'Gagal menambahkan aturan: ' . $e-
>getMessage());
    }
}
```

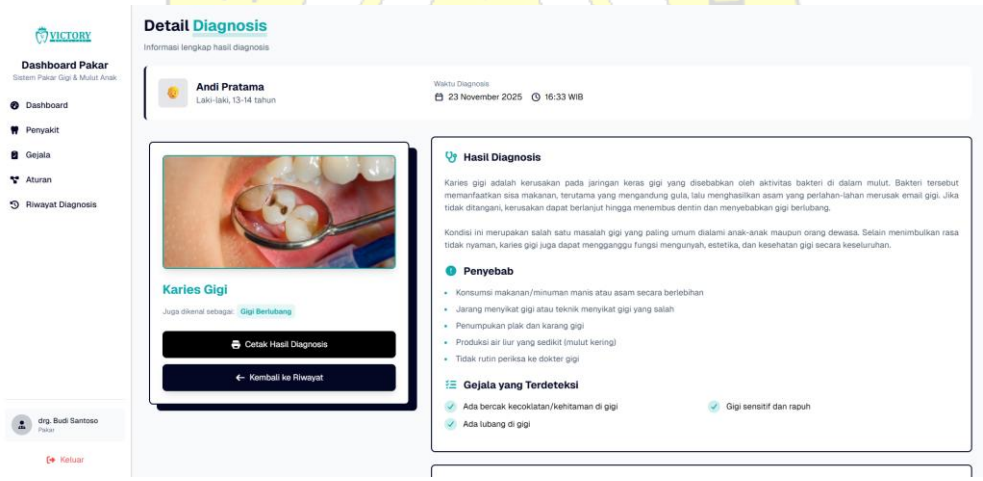
x. Halaman Daftar Data Riwayat Diagnosis



Gambar 77. Halaman Daftar Data Riwayat Diagnosis

Halaman daftar data riwayat diagnosis merupakan halaman yang menyediakan fitur *monitoring* dan manajemen riwayat diagnosis bagi pakar untuk memantau aktivitas penggunaan sistem oleh pengguna. Sebagaimana ditunjukkan pada Gambar 77, halaman ini menampilkan statistik ringkas berupa total diagnosis yang pernah dilakukan, jumlah diagnosis bulan berjalan, dan

jumlah diagnosis hari ini dalam bentuk *card* di bagian atas halaman. Bagian utama halaman menampilkan distribusi usia pengguna untuk menampilkan jumlah diagnosis berdasarkan rentang usia dengan diagram batang. Halaman ini juga menampilkan tabel daftar riwayat diagnosis yang mencakup informasi nama anak, usia, jenis kelamin, tanggal diagnosis, hasil penyakit yang terdiagnosis, serta tombol aksi untuk melihat detail, mengunduh PDF, dan menghapus riwayat diagnosis. Halaman ini dilengkapi dengan berbagai fitur untuk memudahkan pakar dalam menemukan dan menganalisis data diagnosis, termasuk fitur pencarian berdasarkan nama anak, *filter* berdasarkan penyakit yang terdiagnosis, serta opsi pengurutan data berdasarkan tanggal diagnosis terbaru atau terlama. Data ditampilkan dengan sistem *pagination* 5 *item* per halaman.



Gambar 78. Detail Data Riwayat Diagnosis

Tombol aksi "Lihat Detail" pada setiap baris mengarahkan pakar ke halaman detail riwayat diagnosis sebagaimana ditunjukkan pada Gambar 78, yang menampilkan informasi lengkap meliputi data anak, tanggal dan waktu diagnosis, daftar gejala yang dialami, serta detail penyakit yang terdiagnosis

beserta deskripsi, penyebab, pencegahan, dan perawatan. Tombol aksi "Cetak PDF" memungkinkan pakar untuk mengunduh hasil diagnosis dalam format PDF dengan *layout* yang sama seperti pada Gambar 56. Tombol aksi "Hapus" dilengkapi dengan konfirmasi untuk mencegah penghapusan data secara tidak sengaja, dan akan menghapus *record* diagnosis beserta relasi detail diagnosis dan hasil diagnosis.

Berikut adalah implementasi kode pada *controller* untuk halaman riwayat diagnosis:

```
<?php
public function riwayat(Request $request)
{
    $totalDiagnosis = Diagnosis::count();

    $diagnosisBulanIni = Diagnosis::whereYear('created_at',
date('Y'))
        ->whereMonth('created_at', date('m'))
        ->count();

    $diagnosisHariIni = Diagnosis::whereDate('created_at',
date('Y-m-d'))->count();

    $rangeUsiaData = [
        '0-1' => Diagnosis::where('usia_anak', '0-1')->count(),
        '1-2' => Diagnosis::where('usia_anak', '1-2')->count(),
        '2-3' => Diagnosis::where('usia_anak', '2-3')->count(),
        '3-4' => Diagnosis::where('usia_anak', '3-4')->count(),
        '4-5' => Diagnosis::where('usia_anak', '4-5')->count(),
        '5-6' => Diagnosis::where('usia_anak', '5-6')->count(),
        '6-7' => Diagnosis::where('usia_anak', '6-7')->count(),
        '7-8' => Diagnosis::where('usia_anak', '7-8')->count(),
        '8-9' => Diagnosis::where('usia_anak', '8-9')->count(),
        '9-10' => Diagnosis::where('usia_anak', '9-10')->count(),
        '10-11' => Diagnosis::where('usia_anak', '10-11')-
>count(),
        '11-12' => Diagnosis::where('usia_anak', '11-12')-
>count(),
        '12-13' => Diagnosis::where('usia_anak', '12-13')-
>count(),
        '13-14' => Diagnosis::where('usia_anak', '13-14')-
>count(),
        '14-15' => Diagnosis::where('usia_anak', '14-15')-
>count(),
    ];

    $query = Diagnosis::with(['hasilDiagnosis.penyakit']);
```

```
    if ($request->filled('search')) {
        $search = $request->search;
        $query->where('nama_anak', 'like', "%{$search}%");
    }

    if ($request->filled('penyakit') && $request->penyakit !=
'all') {
        $query->whereHas('hasilDiagnosis.penyakit', function ($q)
use ($request) {
            $q->where('nama_penyakit', $request->penyakit);
        });
    }

    $sortBy = $request->get('sort', 'newest');
    if ($sortBy == 'oldest') {
        $query->orderBy('created_at', 'asc');
    } else {
        $query->orderBy('created_at', 'desc');
    }

    $riwayatList = $query->paginate(5);

    $penyakitList = Penyakit::orderBy('nama_penyakit')->get();

    return view('pakar.riwayat', compact(
        'totalDiagnosis',
        'diagnosisBulanIni',
        'diagnosisHariIni',
        'riwayatList',
        'penyakitList',
        'rangeUsiaData'
    ));
}

public function hasilDetail($id)
{
    $diagnosis = Diagnosis::with([
        'hasilDiagnosis.penyakit',
        'detailDiagnosis' => function($query) {
            $query->where('jawaban_user', true)->with('gejala');
        }
    ])->findOrFail($id);

    return view('pakar.hasil', compact('diagnosis'));
}

public function generatePDF($id)
{
    $diagnosis = Diagnosis::with([
        'hasilDiagnosis.penyakit',
        'detailDiagnosis' => function($query) {
            $query->where('jawaban_user', true)->with('gejala');
        }
    ])->findOrFail($id);

    return view('pakar.hasil-pdf', compact('diagnosis'));
}
```

```
public function destroy($id)
{
    try {
        $diagnosis = Diagnosis::findOrFail($id);

        $diagnosis->detailDiagnosis()->delete();
        $diagnosis->hasilDiagnosis()->delete();

        $diagnosis->delete();

        return redirect()->route('pakar.riwayat')
            ->with('success', 'Riwayat diagnosis berhasil
dihapus');
    } catch (\Exception $e) {
        return redirect()->route('pakar.riwayat')
            ->with('error', 'Gagal menghapus riwayat diagnosis');
    }
}
```

4.4 *Testing*

Tahap *testing* merupakan tahap pengujian sistem untuk memastikan bahwa seluruh fungsi dan fitur yang telah diimplementasikan berjalan sesuai dengan kebutuhan yang telah ditetapkan pada tahap *requirements analysis*. Pengujian dilakukan secara sistematis untuk mengidentifikasi dan memperbaiki kesalahan atau *bug* yang mungkin terjadi, serta memvalidasi bahwa sistem dapat beroperasi dengan baik dalam berbagai skenario penggunaan.

Metode pengujian yang digunakan dalam penelitian ini adalah *black box testing*, yaitu metode pengujian yang berfokus pada fungsionalitas sistem tanpa memperhatikan struktur internal kode program. Pendekatan ini dipilih karena lebih menekankan pada validasi bahwa setiap fitur menghasilkan *output* yang benar sesuai dengan *input* yang diberikan, serta memastikan sistem memenuhi *requirement specification* yang telah didefinisikan.

Proses pengujian dilakukan dengan membuat *test case* untuk setiap kebutuhan fungsional pengguna yang telah diidentifikasi pada Tabel 9, mencakup seluruh fitur yang tersedia untuk administrator, pakar, dan klien.

Setiap *test case* dirancang dengan skenario pengujian yang spesifik, mencakup data *input* yang akan digunakan, serta *expected result* yang harus dihasilkan oleh sistem. Pengujian dilakukan secara iteratif pada setiap modul yang telah selesai diimplementasikan untuk memastikan integrasi antar modul berjalan dengan baik dan tidak menimbulkan konflik fungsionalitas.

Berikut adalah rincian hasil pengujian yang telah dilakukan pada sistem pakar diagnosis penyakit gigi dan mulut pada anak:

a. Pengujian Fitur Administrator

Pengujian fitur administrator dilakukan untuk memvalidasi fungsionalitas pengelolaan sistem yang mencakup autentikasi admin, manajemen data pengguna, serta proses persetujuan akun pakar. Pengujian ini memastikan bahwa admin dapat menjalankan tugasnya dalam mengontrol akses pengguna dan menjaga integritas data sistem. Hasil pengujian fitur administrator dapat dilihat pada Tabel 10.

Tabel 10. Pengujian Fitur Administrator

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-A-01] *Login* Admin dengan Kredensial Valid	Memasukkan email dan *password* admin yang terdaftar dan valid pada *form login*	Sistem melakukan validasi kredensial dan mengarahkan admin ke halaman *dashboard* admin	Sesuai
[URS-A-01] *Login* Admin dengan Kredensial Tidak Valid	Memasukkan email admin yang benar tetapi *password* yang salah, atau kombinasi keduanya yang salah	Sistem menampilkan pesan *error* "Email atau *password* salah" dan tetap berada di halaman *login*	Sesuai
[URS-A-02] Tambah Data Pengguna Baru	Mengisi *form* tambah pengguna dengan nama lengkap, email yang belum terdaftar,	Sistem menyimpan data pengguna baru ke *database* dan menampilkan pesan	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
	password, konfirmasi *password*, dan memilih *role user*/pakar/admin	"Pengguna berhasil ditambahkan" serta mengarahkan ke halaman daftar pengguna	
[URS-A-02] Edit Data Pengguna yang Ada	Memilih pengguna dari daftar, mengubah nama lengkap atau email pada *form* edit, dan menyimpan perubahan	Sistem memperbarui data pengguna di *database* dan menampilkan pesan "Pengguna berhasil diperbarui"	Sesuai
[URS-A-02] Hapus Data Pengguna	Memilih pengguna dengan *role* user/pakar/admin dari daftar, menekan tombol hapus, dan mengkonfirmasi penghapusan	Sistem menghapus data pengguna dari *database* beserta relasinya dan menampilkan pesan "Pengguna berhasil dihapus"	Sesuai
[URS-A-02] Hapus Data Pengguna dengan *Role* Admin Terakhir	Memilih pengguna dengan *role* admin yang merupakan satu-satunya admin dalam sistem dari daftar, menekan tombol hapus, dan mengkonfirmasi penghapusan	Sistem menolak penghapusan untuk mencegah sistem kehilangan administrator, menampilkan pesan *error* "Tidak dapat menghapus admin terakhir dalam sistem" dan tetap berada di halaman daftar pengguna	Sesuai
[URS-A-02] Pencarian Data Pengguna Berdasarkan Kata Kunci	Memasukkan kata kunci pada kolom pencarian untuk mencari nama atau email pengguna tertentu	Sistem menampilkan daftar pengguna yang nama lengkap atau emailnya mengandung kata kunci yang dimasukkan	Sesuai
[URS-A-02] Filter Pengguna Berdasarkan Role	Memilih salah satu *role* dari *dropdown* filter (*user*, pakar, atau admin)	Sistem menampilkan hanya pengguna dengan *role* yang dipilih pada filter	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-A-03] Menyetujui Pendaftaran Pakar	Memilih pakar dengan status *pending* dari daftar *approval*, kemudian menekan tombol "Setujui"	Sistem mengubah status *approval* pakar menjadi "*approved*" dan menampilkan pesan konfirmasi keberhasilan persetujuan	Sesuai
[URS-A-03] Menolak Pendaftaran Pakar	Memilih pakar dengan status *pending* dari daftar *approval*, kemudian menekan tombol "Tolak"	Sistem mengubah status *approval* pakar menjadi "*rejected*" dan menampilkan pesan konfirmasi penolakan	Sesuai
[URS-A-03] Filter Pakar Berdasarkan Status *Approval*	Memilih salah satu status dari *dropdown filter* (*pending, approved,* atau *rejected*)	Sistem menampilkan hanya pakar dengan status *approval* yang sesuai dengan pilihan *filter*	Sesuai
[URS-A-04] *Logout* dari Sistem	Menekan tombol *logout* pada *navigation bar* admin	Sistem menghapus sesi *login* dan mengarahkan ke halaman *home*	Sesuai

b. Pengujian Fitur Pakar

Pengujian fitur pakar difokuskan pada validasi kemampuan sistem dalam mengelola basis pengetahuan yang mencakup data penyakit, gejala, dan aturan diagnosis, serta *monitoring* riwayat diagnosis pengguna. Pengujian ini memastikan bahwa pakar dapat melakukan CRUD *operation* pada seluruh entitas basis pengetahuan dengan validasi yang tepat. Rincian skenario pengujian fitur pakar disajikan pada Tabel 11.

Tabel 11. Pengujian Fitur Pakar

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-P-01] Registrasi sebagai Pakar	Mengisi *form* registrasi dengan nama lengkap, email yang belum	Sistem menyimpan data pakar dengan status "*pending*", menampilkan	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
	terdaftar, *password*, konfirmasi *password*, mencentang opsi "Daftar sebagai Pakar Gigi", dan memasukkan nomor STR (Surat Tanda Registrasi) yang valid	pesan sukses pendaftaran, dan mengarahkan ke halaman *login* untuk menunggu *approval* admin	
[URS-P-01] Registrasi Pakar dengan Email atau Nomor STR (Surat Tanda Registrasi) Duplikat	Mengisi form registrasi pakar dengan email/nomor STR yang sudah terdaftar dalam sistem	Sistem menampilkan pesan *error* "Email atau Nomor STR sudah digunakan" dan tidak menyimpan data ke *database*	Sesuai
[URS-P-01] Daftar dengan *Password* Tidak Sama	Formulir diisi dengan data valid, tetapi *field password* dan konfirmasi *password* tidak sama	Muncul *error* "Konfirmasi *password* tidak cocok"	Sesuai
[URS-P-01] Daftar Sebagai Pakar Tanpa Mengisi No. STR	*Checkbox* "Daftar sebagai Pakar" dicentang, *field* No. STR dikosongkan, *field* lainnya valid	Muncul *error* "Nomor STR wajib diisi untuk pakar"	Sesuai
[URS-P-01] Daftar dengan No. STR Tidak 7 Digit	*Checkbox* "Daftar sebagai Pakar" dicentang, *field* No. STR diisi dengan angka kurang dari atau lebih dari 7 digit, *field* lainnya valid	Muncul *error* "Nomor STR harus 7 digit"	Sesuai
[URS-P-01] Daftar dengan *Password* Kurang Dari 6 Karakter	Formulir diisi dengan *password* kurang dari 6 karakter, *field* lainnya valid	Muncul *error* "*Password* minimal 6 karakter"	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-P-02] *Login* Pakar yang Telah Disetujui	Memasukkan email dan *password* pakar yang sudah memiliki status *approval* "*approved*" pada *form login*	Sistem melakukan validasi dan mengarahkan pakar ke halaman *dashboard* pakar	Sesuai
[URS-P-02] *Login* Pakar yang Belum/Tidak Disetujui	Memasukkan email dan *password* pakar yang memiliki status "*pending*" atau "*rejected*" pada *form login*	Sistem menampilkan pesan "Akun Anda belum/tidak disetujui oleh administrator" dan tidak mengizinkan akses ke sistem	Sesuai
[URS-P-02] *Login* dengan Email Atau *Password* yang Salah	Formulir *login* diisi dengan email atau *password* yang tidak sesuai dengan data di *database*	*Login* gagal, muncul *error* "Email atau *password* salah", tetap di halaman *login*	Sesuai
[URS-P-02] *Login* dengan *Field* Kosong	Salah satu atau kedua *field* (email/*password*) dikosongkan	Muncul *error* validasi "Email wajib diisi" atau "*Password* wajib diisi", tetap di halaman *login*	Sesuai
[URS-P-03] Tambah Data Penyakit dengan Informasi Lengkap	Mengisi *form* tambah penyakit dengan kode penyakit unik, nama penyakit, nama lain, *upload* gambar dalam format yang diizinkan, serta mengisi deskripsi, penyebab, pencegahan, dan perawatan secara lengkap	Sistem menyimpan data penyakit ke *database*, meng-*upload* gambar ke *storage* dengan nama sesuai kode penyakit, dan menampilkan pesan "Penyakit berhasil ditambahkan"	Sesuai
[URS-P-03] Tambah Penyakit dengan Kode yang Sudah Ada	*Form* tambah penyakit diisi dengan kode penyakit yang sudah terdaftar di *database*, *field* lainnya valid	Gagal menyimpan, muncul *error* "Kode penyakit sudah terdaftar", tidak ada data baru yang tersimpan	Sesuai
[URS-P-03] Tambah Penyakit	Salah satu atau beberapa *field* wajib	Gagal menyimpan, muncul pesan validasi sesuai *field*	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
dengan *Field* Wajib Kosong	(kode, nama, atau deskripsi) dikosongkan	yang kosong, tetap di halaman *form* tambah	
[URS-P-03] Edit Data Penyakit dan Ganti Gambar	Memilih penyakit dari daftar, mengubah informasi pada *form* edit, dan *upload* gambar baru untuk mengganti gambar lama	Sistem memperbarui data penyakit di *database*, menghapus file gambar lama dari *storage*, menyimpan gambar baru, dan menampilkan pesan "Penyakit berhasil diperbarui"	Sesuai
[URS-P-03] Hapus Data Penyakit yang Tidak Berelasi	Memilih penyakit yang tidak memiliki aturan diagnosis dari daftar, kemudian menekan tombol hapus dan mengkonfirmasi	Sistem menghapus data penyakit beserta file gambar dari *storage* dan menampilkan pesan "Penyakit berhasil dihapus"	Sesuai
[URS-P-03] Hapus Penyakit yang Sudah Memiliki Aturan	Memilih penyakit yang sudah digunakan dalam aturan diagnosis, kemudian menekan tombol hapus	Sistem menolak penghapusan dan tombol hapus *disabled*	Sesuai
[URS-P-03] Pencarian Penyakit Berdasarkan Kata Kunci	Memasukkan kata kunci pada kolom pencarian untuk mencari penyakit tertentu	Sistem menampilkan penyakit yang kode, nama, nama lain, atau deskripsinya mengandung kata kunci tersebut	Sesuai
[URS-P-04] Tambah Data Gejala Baru	Mengisi *form* tambah gejala dengan kode gejala unik, nama gejala, dan pertanyaan gejala yang akan ditampilkan pada proses diagnosis	Sistem menyimpan data gejala ke *database* dengan kode yang dikonversi ke *uppercase* dan menampilkan pesan "Gejala berhasil ditambahkan"	Sesuai
[URS-P-04] Tambah Gejala	*Form* tambah gejala diisi dengan kode gejala yang sudah	Gagal menyimpan, muncul *error* "Kode gejala sudah	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
dengan Kode yang Sudah Ada	terdaftar di *database*, *field* lainnya valid	terdaftar", tidak ada data baru yang tersimpan	
[URS-P-04] Tambah Gejala dengan *Field* Wajib Kosong	Salah satu atau beberapa *field* wajib (kode, nama, atau pertanyaan) dikosongkan	Gagal menyimpan, muncul pesan validasi sesuai *field* yang kosong, tetap di halaman *form* tambah	Sesuai
[URS-P-04] Edit Data Gejala yang Ada	Memilih gejala dari daftar, mengubah nama gejala atau pertanyaan gejala pada *form* edit	Sistem memperbarui data gejala di *database* dan menampilkan pesan "Gejala berhasil diperbarui"	Sesuai
[URS-P-04] Hapus Data Gejala yang Tidak Berelasi	Memilih gejala yang tidak digunakan dalam aturan diagnosis dari daftar, kemudian menekan tombol hapus	Sistem menghapus data gejala dari *database* dan menampilkan pesan "Gejala berhasil dihapus"	Sesuai
[URS-P-04] Hapus Gejala yang Sudah Memiliki Aturan	Memilih gejala yang sudah digunakan dalam aturan diagnosis, kemudian menekan tombol hapus	Sistem menolak penghapusan dan tombol hapus *disabled*	Sesuai
[URS-P-04] Pencarian Gejala Berdasarkan Kata Kunci	Memasukkan kata kunci pada kolom pencarian untuk mencari gejala tertentu	Sistem menampilkan gejala yang kode atau namanya mengandung kata kunci yang dimasukkan	Sesuai
[URS-P-05] Tambah Aturan Diagnosis Baru	Memilih penyakit yang belum memiliki aturan dari *dropdown*, memilih beberapa gejala sebagai gejala utama (*AND logic*), dan memilih gejala terkait (*OR logic*) jika ada, kemudian menyimpan	Sistem menyimpan aturan dengan group_id yang tepat (*AND*: *unique* per gejala, *OR*: *shared* group_id) dan menampilkan pesan "Aturan berhasil ditambahkan"	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-P-05] Tambah Aturan Tanpa Memilih Gejala	*Form* tambah aturan diisi dengan memilih penyakit tetapi tidak memilih satupun gejala utama maupun terkait	Gagal menyimpan, muncul pesan validasi "Minimal satu gejala harus dipilih", tetap di halaman *form* tambah	Sesuai
[URS-P-05] Lihat Matriks Aturan dengan Status Gejala yang Berbeda	Akses halaman daftar aturan untuk melihat matriks 2x2 penyakit dan gejala	Matriks menampilkan simbol bintang untuk gejala utama, simbol centang untuk gejala terkait, kotak kosong untuk tidak terkait, dan ikon *plus* untuk penyakit belum ada aturan	Sesuai
[URS-P-05] Edit Aturan Melalui Mode Edit Matriks	Mengaktifkan mode edit pada halaman aturan, mengubah beberapa relasi antara penyakit dan gejala dengan mengubah status *AND/OR* atau menghapus/menambah relasi, kemudian menyimpan perubahan *batch*	Sistem menghapus aturan lama untuk penyakit yang diubah, membuat aturan baru sesuai dengan modifikasi yang dilakukan, dan menampilkan pesan "Semua perubahan berhasil disimpan"	Sesuai
[URS-P-05] Simpan Perubahan Aturan dari Mode Edit	Setelah melakukan perubahan pada beberapa sel dalam mode edit, klik tombol “Simpan”	Semua perubahan berhasil disimpan ke *database* secara *bulk update*, muncul notifikasi sukses, matriks menampilkan data ter-*update*	Sesuai
[URS-P-05] Membatalkan Mode Edit Tanpa Menyimpan	Aktifkan mode edit, lakukan beberapa perubahan pada matriks, kemudian klik *toggle* mode edit lagi tanpa menyimpan	Halaman di-*refresh* dan semua perubahan yang belum disimpan hilang, matriks kembali ke data asli dari *database*	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-P-06] Melihat Riwayat Diagnosis dengan Statistik	Mengakses halaman riwayat diagnosis pakar	Sistem menampilkan statistik total diagnosis, diagnosis bulan berjalan, dan diagnosis hari ini, serta tabel daftar riwayat diagnosis dengan *pagination*	Sesuai
[URS-P-06] *Filter* Riwayat Berdasarkan Penyakit	Memilih nama penyakit tertentu dari *dropdown filter* pada halaman riwayat diagnosis	Sistem menampilkan hanya riwayat diagnosis yang hasil diagnosisnya sesuai dengan penyakit yang dipilih	Sesuai
[URS-P-06] Pencarian Riwayat Berdasarkan Nama Anak	Memasukkan nama anak pada kolom pencarian di halaman riwayat diagnosis	Sistem menampilkan riwayat diagnosis yang nama anaknya mengandung kata kunci yang dimasukkan	Sesuai
[URS-P-06] Lihat Detail Riwayat Diagnosis Lengkap	Memilih salah satu riwayat diagnosis dari daftar dan menekan tombol "Lihat Detail"	Sistem menampilkan halaman detail diagnosis lengkap dengan data anak, tanggal diagnosis, gejala yang dialami, dan hasil penyakit dengan informasi lengkap	Sesuai
[URS-P-06] Cetak riwayat diagnosis ke PDF	Klik tombol cetak atau ikon print pada salah satu riwayat diagnosis	Sistem men-*generate* file PDF berisi detail diagnosis lengkap dengan format yang rapi dan dapat diunduh atau dibuka di tab baru	Sesuai
[URS-P-06] Hapus Riwayat Diagnosis	Memilih riwayat diagnosis dari daftar, menekan tombol hapus, dan mengkonfirmasi penghapusan	Sistem menghapus *record* diagnosis beserta detail diagnosis dan hasil diagnosis dari *database*, menampilkan pesan "Riwayat diagnosis berhasil dihapus"	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-P-07] *Logout* dari Sistem	Menekan tombol *logout* pada *navigation bar* pakar	Sistem menghapus sesi *login* dan mengarahkan ke halaman *home*	Sesuai

c. Pengujian Fitur Klien

Pengujian fitur klien dilakukan untuk memverifikasi fungsionalitas yang digunakan oleh pengguna umum, mulai dari registrasi, akses informasi penyakit, proses diagnosis menggunakan *forward chaining*, hingga pengelolaan riwayat diagnosis pribadi. Pengujian ini memastikan bahwa *user experience* berjalan lancar dan menghasilkan *output* diagnosis yang akurat. Skenario pengujian lengkap untuk fitur klien dapat dilihat pada Tabel 12.

Tabel 12. Pengujian Fitur Klien

Skenario	*Input*	*Output* yang Diharapkan	Status
[URS-K-01] Registrasi sebagai Klien/*User*	Mengisi form registrasi dengan nama lengkap, email yang belum terdaftar, *password*, dan konfirmasi *password* tanpa mencentang opsi pakar	Sistem menyimpan data *user* dengan *role* "*user*" dan status "*approved*" secara otomatis, menampilkan pesan sukses, dan mengarahkan ke halaman *login*	Sesuai
[URS-K-01] Registrasi dengan Email yang Sudah Terdaftar	Mengisi *form* registrasi dengan email yang sudah ada dalam *database*	Sistem menampilkan pesan *error* "Email sudah terdaftar" dan tidak menyimpan data duplikat	Sesuai
[URS-K-02] *Login* Klien dengan Kredensial Valid	Memasukkan email dan *password user* yang terdaftar dan valid pada *form login*	Sistem melakukan validasi dan mengarahkan *user* ke halaman *home*	Sesuai
[URS-K-03] Mengakses Info Penyakit Tanpa *Login*	Mengakses halaman daftar penyakit tanpa melakukan *login* terlebih dahulu	Sistem menampilkan halaman daftar penyakit dalam bentuk *card grid* dengan informasi dasar	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
		tanpa memerlukan autentikasi	
[URS-K-03] Melihat Detail Informasi Penyakit	Memilih salah satu penyakit dari daftar dan menekan tombol "Lihat Detail" tanpa *login*	Sistem menampilkan halaman detail penyakit lengkap dengan deskripsi, penyebab, pencegahan, dan perawatan tanpa memerlukan autentikasi	Sesuai
[URS-K-03] Pencarian Penyakit Berdasarkan Kata Kunci	Memasukkan kata kunci pada kolom pencarian di halaman info penyakit	Sistem menampilkan penyakit yang nama, nama lain, kode, atau deskripsinya mengandung kata kunci tersebut	Sesuai
[URS-K-04] Memulai Proses Diagnosis Tanpa *Login*	Mencoba mengakses halaman diagnosis atau URL `/diagnosis` tanpa melakukan *login* terlebih dahulu	Sistem mendeteksi tidak ada sesi autentikasi aktif, dan *redirect* ke halaman *login*	Sesuai
[URS-K-04] Memulai Proses Diagnosis dengan Data Anak	Setelah *login*, mengakses halaman diagnosis, menekan tombol "Mulai Diagnosis", dan mengisi data anak berupa nama, usia, dan jenis kelamin	Sistem menyimpan data anak, membuat *record* diagnosis baru di *database*, dan menampilkan pertanyaan gejala pertama dari sistem	Sesuai
[URS-K-04] Menjawab Pertanyaan Gejala Secara Berurutan	Menjawab "Ya" atau "Tidak" pada setiap pertanyaan gejala yang ditampilkan oleh sistem secara berurutan	Sistem menyimpan setiap jawaban ke detail_diagnosis, menjalankan algoritma *forward chaining*, dan menampilkan pertanyaan berikutnya atau hasil jika sudah mencukupi	Sesuai
[URS-K-04] Mendapatkan Hasil	Menjawab pertanyaan gejala dengan	Sistem menampilkan halaman hasil diagnosis	Sesuai

Skenario	*Input*	*Output* yang Diharapkan	Status
Diagnosis yang Cocok	kombinasi yang sesuai dengan salah satu aturan diagnosis dalam sistem	dengan informasi penyakit yang teridentifikasi, gejala yang dialami, serta rekomendasi pencegahan dan perawatan	
[URS-K-04] Hasil Diagnosis Tidak Ditemukan	Menjawab pertanyaan gejala dengan kombinasi yang tidak cocok dengan aturan diagnosis manapun dalam sistem	Sistem menampilkan pesan "Tidak ditemukan penyakit yang sesuai dengan gejala yang dialami" dan memberikan saran untuk konsultasi dengan dokter gigi	Sesuai
[URS-K-05] Mengakses Riwayat Diagnosis Pribadi	Setelah *login* sebagai *user*, mengakses halaman riwayat diagnosis	Sistem menampilkan daftar riwayat diagnosis yang pernah dilakukan oleh *user* tersebut dengan informasi nama anak, usia, tanggal diagnosis, dan hasil diagnosis	Sesuai
[URS-K-05] Melihat Detail Hasil Diagnosis dari Riwayat	Memilih salah satu riwayat diagnosis dari daftar dan menekan tombol "Lihat Detail"	Sistem menampilkan halaman detail hasil diagnosis lengkap dengan data anak, gejala yang dialami, dan informasi penyakit yang terdiagnosis	Sesuai
[URS-K-06] Mengunduh Hasil Diagnosis dalam Format PDF	Pada halaman hasil diagnosis atau detail riwayat, menekan tombol "Cetak PDF"	Sistem *generate* file PDF dengan *layout* hasil diagnosis lengkap	Sesuai
[URS-K-07] *Logout* dari Sistem	Menekan tombol *logout* pada *navigation bar user*	Sistem menghapus sesi *login* dan mengarahkan ke halaman *home*	Sesuai

4.5 *Maintenance*

Tahap *maintenance* merupakan tahap terakhir dalam siklus pengembangan perangkat lunak menggunakan metode *waterfall*. Pada tahap ini, sistem pakar diagnosis penyakit gigi dan mulut pada anak yang telah selesai dikembangkan dan diuji memasuki fase pemeliharaan untuk memastikan sistem tetap berjalan dengan baik, dapat beradaptasi dengan kebutuhan baru, serta diperbaiki jika ditemukan kesalahan yang tidak terdeteksi pada tahap sebelumnya. Pemeliharaan sistem mencakup tiga aspek utama, yaitu perbaikan *bug*, penyesuaian sistem terhadap perubahan lingkungan operasional, dan pengembangan fitur baru yang meningkatkan fungsionalitas sistem.

Perbaikan dilakukan untuk mengatasi kesalahan atau *bug* yang ditemukan setelah sistem mulai dioperasikan. Meskipun tahap *testing* telah dilakukan dengan metode *black-box testing*, kemungkinan masih terdapat *bug* yang belum terdeteksi dalam kondisi penggunaan tertentu atau *edge case* yang tidak terprediksi sebelumnya. Beberapa area yang perlu mendapat perhatian khusus dalam perbaikan *bug* meliputi aspek responsivitas tampilan, terutama perbaikan tampilan yang mungkin tidak konsisten pada *navigation bar*, serta optimasi responsivitas tampilan *mobile* jika ditemukan *layout* yang rusak atau tidak *user-friendly* pada perangkat dengan resolusi layar yang berbeda-beda.

Penyesuaian dilakukan untuk mengadaptasi sistem dengan perubahan lingkungan operasional. Penyesuaian utama yang sangat disarankan untuk sistem ini adalah *deployment* ke server *online*, mengingat sistem saat ini masih berjalan di *localhost* dan belum dapat diakses oleh publik secara luas. Untuk mewujudkan akses publik yang optimal, sistem telah di-*deploy* ke subdomain

resmi Klinik Victory Purwokerto yang memungkinkan akses dari berbagai perangkat dan lokasi tanpa batasan jaringan lokal.

Pengembangan fitur dilakukan untuk menambahkan kemampuan baru atau meningkatkan fungsionalitas yang sudah ada berdasarkan *feedback* pengguna atau kebutuhan bisnis yang terus berkembang seiring waktu. Beberapa pengembangan fitur yang sangat disarankan untuk meningkatkan nilai dan kegunaan sistem pakar ini meliputi implementasi *chatbot interface* dengan *Natural Language Processing* (NLP) sebagai fitur asisten virtual yang memungkinkan pengguna untuk bertanya seputar sistem, seperti jenis penyakit yang dapat didiagnosis, cara kerja proses diagnosis, informasi tentang metode *forward chaining*, atau pertanyaan umum lainnya terkait kesehatan gigi dan mulut anak. Selain itu, pengembangan fitur integrasi dengan Rekam Medis Elektronik (*Electronic Medical Record*/EMR) juga sangat direkomendasikan untuk meningkatkan keterpaduan sistem dengan ekosistem layanan kesehatan yang lebih luas. Integrasi EMR memungkinkan sinkronisasi otomatis data pasien, riwayat diagnosis, hasil pemeriksaan klinis, dan rencana perawatan dengan sistem informasi kesehatan klinik atau rumah sakit. Implementasi integrasi EMR harus memperhatikan aspek keamanan data dengan menerapkan standar protokol dan enkripsi untuk melindungi privasi informasi medis pasien sesuai regulasi perlindungan data kesehatan yang berlaku.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Sistem pakar diagnosis penyakit gigi dan mulut pada anak telah berhasil dikembangkan dengan menerapkan metode *forward chaining* sebagai mesin inferensi yang terbukti efektif dalam melakukan penalaran maju dari berbagai kombinasi gejala menuju diagnosis penyakit yang sesuai.
2. Sistem berhasil mengidentifikasi 10 jenis penyakit gigi dan mulut pada anak (karies gigi, pulpitis, abses gigi, gingivitis, persistensi gigi sulung, trauma gigi, fluorosis dental, hipoplasia email, stomatitis aphthosa, dan maloklusi) yang dipetakan dengan 38 gejala berdasarkan studi literatur dan wawancara dengan pakar dari Klinik Victory Purwokerto.
3. *Platform* berbasis *website* menggunakan *framework* Laravel terbukti tepat karena memberikan aksesibilitas universal melalui berbagai perangkat tanpa instalasi khusus. Integrasi dengan MySQL memastikan pengelolaan data efisien dengan arsitektur MVC (*Model-View-Controller*) terstruktur, fitur keamanan, serta kemudahan pemeliharaan.
4. Pengujian *black-box testing* terhadap seluruh fitur untuk administrator, pakar, dan klien menunjukkan semua skenario menghasilkan *output* sesuai harapan, memvalidasi bahwa sistem memenuhi kebutuhan fungsional yang ditetapkan.

5.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan selama proses pengembangan, beberapa saran yang dapat diberikan untuk pengembangan sistem lebih lanjut adalah sebagai berikut:

1. Disarankan mengembangkan *chatbot interface* dengan *Natural Language Processing* (NLP) sebagai asisten virtual yang memungkinkan pengguna bertanya interaktif seputar jenis penyakit, cara kerja diagnosis *forward chaining*, informasi gejala, atau pertanyaan umum kesehatan gigi anak. *Chatbot* ini melengkapi sistem diagnosis dengan memberikan akses cepat terhadap informasi melalui percakapan natural untuk meningkatkan pemahaman dan edukasi pengguna.
2. Untuk meningkatkan keterpaduan dengan ekosistem layanan kesehatan, direkomendasikan mengembangkan integrasi dengan Rekam Medis Elektronik (*Electronic Medical Record*/EMR) yang memungkinkan sinkronisasi otomatis data pasien, riwayat diagnosis, hasil pemeriksaan, dan rencana perawatan. Implementasi harus memperhatikan keamanan data dengan menerapkan standar protokol dan enkripsi sesuai regulasi perlindungan data kesehatan.
3. Disarankan memperluas cakupan sistem untuk mencakup penyakit gigi dan mulut pada semua kelompok usia, tidak hanya terbatas pada anak. Perluasan dapat mencakup penyakit periodontal (di sekeliling gigi) tingkat lanjut dan kondisi gigi pada lansia dengan menambahkan kategori usia dalam basis pengetahuan serta penyesuaian aturan diagnosis yang spesifik untuk setiap kelompok usia.

DAFTAR PUSTAKA

Aaliyah, D. A., Isnur, H., Sari, G. D., Azizah, A., & Wibowo, D. (2024). Hubungan Tingkat Keparahan Maloklusi dengan Kualitas Hidup. *Dentin Jur. Ked. Gigi*, *8*(2), 102–107.

Abses gigi [Gambar]. (2022). Diakses pada 2 November 2025, dari https://cdn.hellosehat.com/wp-content/uploads/2022/06/1c27a096-shutterstock_1442868839-650x434.jpg

Adi Kurniyanti, V., & Murdiani, D. (2022). Perbandingan Model Waterfall Dengan Prototype Pada Pengembangan System Informasi Berbasis Website. *Jurnal Syntax Fusion*, *2*(08), 669–675. https://doi.org/10.54543/fusion.v2i08.210

Afdhal, M., & Mayola, L. (2022). Sistem Pakar Diagnosa Penyakit Hiperlipidemia Menggunakan Metode Certainty Factor Berbasis Web. *Jurnal KomtekInfo*, *9*(4), 133–139. https://doi.org/10.35134/komtekinfo.v9i4.321

Ahmad Kurniawan, D., & Febriana Sesunan, M. (2022). Sistem Pakar Untuk Mendiagnosa Penyakit Gigi an Mulut Menggunakan (Metode Forward Chaining (Studi Di RSUD Menggala). *Jurnal Sistem Informasi Dan Sains Teknologi*, *4*(2), 77–84.

Alam, H., Ahmadi, R. K., Muhazir, A., & Widya, H. (2021). Sistem Pakar Untuk Membantu Pengambilan Keputusan Guru Bimbingan Konseling Dengan Menggunakan Metode Forward Chaining Studi Kasus SMA Negeri 2 Kisaran. *Seminar Nasional Teknik (SEMNASTEK) UISU*, *4*(1), 242–247.

Andini, A., Dasril, D., & Wahyuni, V. I. (2025). Rancang Bangun Website Sistem Informasi Pendaftaran Pelatihan Kursus Pada Merah Putih International Language School Palopo. *Jurnal Informatika Dan Teknik Elektro Terapan*, *13*(1), 1506–1516. https://doi.org/10.23960/jitet.v13i1.5986

Aqilah, A. al afif fadhil, Bustamin, S., & Sultan sahrir, S. (2023). Sistem Informasi Manajemen Persediaan Berbasis Web di CV. Makmur Sejahtera Palopo. *Jurnal PROCESSOR*, *18*(2), 227–240. https://doi.org/10.33998/processor.2023.18.2.1385

Bukhari, Fatna, N., & Ridha, M. Al. (2025). Hubungan Kebiasaan Jajan Makanan Jenis Kariogenik Dengan Kejadian Karies Gigi Pada Siswa Sd Negeri 5. *Jurnal Kesehatan Akimal (JKA)*, *4*(1), 82–90.

Dan, H., & Tunanetra, K. (2023). Inklusivitas Digital : Pemanfaatan Situs Web Untuk Mendorong Kualitas. *Prosiding Seminar Nasional Kemahasiswaan*, *1*(1), 43–47.

Dewi, A. R., Jeddy, & Badruzzaman, I. T. (2021). Gambaran Tingkat Pengetahuan Dan Sikap Ibu Tentang Trauma Gigi Permanen Pada Anak Usia 8-12 Tahun. *Jkgt*, *3*(1), 17–21. https://trijurnal.lemlit.trisakti.ac.id/jkgt/article/view/9832

Dewi, C. I., Idaryati, N. P., Palgunadi, I. N. P. T., & Pramudya, P. (2024). Prevalensi

Persistensi Gigi Sulung Pada Anak Usia 5- 12 Tahun di Wilayah Kerja Puskesmas Kediri II Tabanan. *Bali Dental Science and Exhibition*, 1350–1357.

Faisal, F., Opitasari, O., & Mufti, A. (2024). Sistem Pakar Pendiagnosa Penyakit Mata Dengan Metode Forward Chaining. *Semnas Ristek (Seminar Nasional Riset Dan Inovasi Teknologi)*, *8*(01), 132–137. https://doi.org/10.30998/semnasristek.v8i01.7146

Firnando, J., Franko, B., Pratama Tanzil, S., Wilyanto, N., Christianto Tan, H., & Hartati Kom, E. M. (2023). Pembuatan Website Menggunakan Visual Studio Code di SMA Xaverius 3 Palembang. *Fordicate*, *3*(1), 1–8.

Florentinsa, M. (2023). Perbedaan Anatomi Rongga Mulut Bayi Dan Anak Berkaitan Dengan Perkembangan Gungsi Menelan: Telaah Literatur. *Perbedaan Anatomi Rongga Mulut Bayi Dan Anak Berkaitan Dengan Perkembangan Gungsi Menelan: Telaah Literatur*.

Fluorosis dental [Gambar]. (2023). Diakses pada 2 November 2025, dari https://primaku.com/parenting/white-spots-pada-gigi-anak--bisa-hilangnggak--ya--1677581838474

Fresno Argadianti, A., Tuti Hendarti, H., & Radithia, D. (2020). Stomatitis aftosa yang diperparah oleh iritasi kimiawi obat tradisional. *Clinical Dental Journal UGM*, *6*(2), 44–51.

Gingivitis [Gambar]. (2024). Diakses pada 2 November 2025, dari https://www.satudental.com/wp content/uploads/elementor/thumbs/PenyakitGusi-Gingivitis-Radang-Gusi-dan-Periodontitis-- qsugjib4si93mgv1lejlmlrrgmb2zl1rpxh0fab4jo.jpg

Habibi, R. et al. (2020). *Penggunaan Framework Laravel untuk Membuat Aplikasi Absensi Terintegrasi Mobile*. https://books.google.co.id/books?id=mZPuDwAAQBAJ

Hakim, M. (2020). Sistem Pakar Mengidentifikasi Penyakit Alat Reproduksi Manusia Menggunakan Metode Forward Chaining. *TEKNIMEDIA: Teknologi Informasi Dan Multimedia*, *1*(1), 59–67. https://doi.org/10.46764/teknimedia.v1i1.16

Hidayati, A. T., Widyantoro, A. E., & Ramadhani, H. J. (2023). Perancangan Sistem Informasi Wirausaha Mahasiswa (Siwirma) Berbasis Web dengan Unified Modelling Languange (UML) Institut Teknologi dan Bisnis Semarang untuk memodelkan sistem [6]. Definisi lainnya , UML merupakan kumpulan diagram yang UML mempunyai. *Jurnal Penelitian Rumpun Ilmu Teknik (JUPRIT)*, *2*(4), 86–107.

Hipoplasia email [Gambar]. (2022). Diakses pada 2 November 2025, dari https://thekidsdentistoflasvegas.com/wpcontent/uploads/2022/12/Screenshot_82.png

Ida Ayu Bulan Kencana Dewi, I. W. A. W. P. (2025). Prevalensi Kasus Abses Periapikal pada Pasien di Poliklinik Gigi UPTD Puskesmas Baturiti I Tabanan

Bali. *Jurnal Kesehatan Gigi (Dental Health Journal)*, *12*(1), 16–23. https://doi.org/10.33992/jkg.v

Ikhsanto, M. N., Sulistiyanto, M., & Nafisa, L. (2022). PENERAPAN METODE FORWARD CHAINING UNTUK MENGIDENTIFIKASI HAMA DAN PENYAKIT TANAMAN PADI (Studi Kasus : Desa Purworejo Kec. Kotagajah Kab. Lampung Tengah). *International Research on Big-Data and Computer Technology: I-Robot*, *5*(1), 48–53. https://doi.org/10.53514/ir.v5i1.151

Isnaini, R. N. (2023). *Peran orang tua dalam merawat kondisi gigi sehat dan karies pada anak usia dini di Posyandu Anggrek 1 Klampok Kasri Kota Malang* [Universitas Islam Negeri Maulana Malik Ibrahim]. http://etheses.uin-malang.ac.id/50705/1/18160023.pdf

Karies gigi [Gambar]. (2025). Diakses pada 2 November 2025, dari https://nos.wjv 1.neo.id/rspi-assets-production/rspi api/uploads/MTc0OTYyNTA5MjI1Mg==.jpg

Karima, N. P. T. C. D. (2023). The relationship of Streptococcus mutans spaP gene presence to dental caries severity on children in Denpasar City. *Makassar Dental Journal*, *12*(2), 233–235. https://doi.org/10.35856/mdj.v12i2.772

Kholil, M., & Akhsani, R. (2021). *Desain dan Pemrograman Web: HTML, CSS, PHP, MySQL, & Bootstrap*. https://books.google.co.id/books?id=ZzKhEAAAQBAJ

Kusuma, Abdi Pandu, M. S. (2019). Perbandingan Metode Forward Chaining Dan Backward Chaining Pada Sistem Pakar Diagnosis Penyakit Ikan Lele Sangkuriang. *Jurnal Ilmiah Teknik Informatika*, *13*(1), 59–71.

Kusumawardani, Dwi Mustika, et al. (2023). Web Dasar Menggunakan HTML, CSS, JS, PHP dan Studi Kasus. *PT. Sonpedia Publishing Indonesia*.

Lestari, W. Y. (2025). Pentingnya Menjaga Kesehatan Gigi dan Mulut. *Traz Media Publishing*, *1*(1). https://ejournal.edelweispublishing.com/index.php/trazmediapublishing/article/view/34

Maloklusi [Gambar]. (2024). Diakses pada 2 November 2025, dari https://klinikjoydental.com/wp-content/uploads/2024/03/Informasi-terkaitmaloklusi-gigi-yang-wajib-diketahui.jpg

Manek, A., Nurcahyo, W., & Faizah, N. (2024). Rancang Bangun Aplikasi Sistem Pakar Diagnosis Penyakit Jantung: Studi Kasus pada Klinik Siaga Menggunakan Metode Case-Based Reasoning (CBR) Berbasis Web. *Design Journal*, *2*(2), 51–59. https://doi.org/10.58477/dj.v2i2.182

Matondang, S. (2023). Sistem Pakar Diagnosis Penyakit Umum Menggunakan Metode Forward Chaining. In *Remik: Riset dan E-Jurnal Manajemen Informatika Komputer* (Vol. 7, Issue 3, pp. 1712–1723). https://doi.org/10.1002/1099-0992(200009/10)30:5<613::aid-ejsp11>3.3.co;2-j

Mulyani et al. (2023). Sistem Pakar Diagnosa Stunting Pada Balita Menggunakan Metode Forward Chaining Dan Certainty Factor. *Digital Intelligence*, *3*(2), 22–31.

Mulyati, R., Rohayani, L., & Pratiwi Santika & Mia. (2022). Hubungan Konsumsi Makanan Kariogenik Dengan Kejadian Karies Gigi Pada Anak Sekolah Dasar: Tinjauan Literatur Cariogenic Foods with Dental Caries Incidence in Elementary School Children : Literature Review. *Ilmiah Keperawatan (Scientific Journal Of Nursing) Edisi Khusus*, *8*(3), 12. https://journal.stikespemkabjombang.ac.id/index.php/jikep/article/view/1155

Nur Adiya, A. Z. D., Anggraeni, D. L., & Ilham Albana. (2024). Analisa Perbandingan Penggunaan Metodologi Pengembangan Perangkat Lunak (Waterfall, Prototype, Iterative, Spiral, Rapid Application Development (RAD)). *Merkurius : Jurnal Riset Sistem Informasi Dan Teknik Informatika*, *2*(4), 122–134. https://doi.org/10.61132/merkurius.v2i4.148

P. P. Adi, A. Fatullah, D. M. H. ri, & P. Winda. (2020). Pengujian Aplikasi Point of Sale Berbasis Web Menggunakan Black Box Testing. *Jurnal Bina Komputer*, *2*(1), 74–78.

Persistensi gigi sulung [Gambar]. (2024). Diakses pada 2 November 2025, dari https://mhdc.co.id/wp-content/uploads/Frenectomies2.webp

Pontoluli, Z. G., Khoman, J. A., & Wowor, V. N. S. (2021). Kebersihan Gigi Mulut dan Kejadian Gingivitis pada Anak Sekolah Dasar. *E-GiGi*, *9*(1), 21–28. https://doi.org/10.35790/eg.9.1.2021.32366

Pulpitis [Gambar]. (2022). Diakses pada 2 November 2025, dari https://cdn.hellosehat.com/wp-content/uploads/2022/09/e340760dshutterstock_1571330368-650x434.jpg

Ramdany, S. (2024). Penerapan UML Class Diagram dalam Perancangan Sistem Informasi Perpustakaan Berbasis Web. *Journal of Industrial and Engineering System*, *5*(1), 30–41. https://doi.org/10.31599/2e9afp31

Romdona, S., Junista, S. S., & Gunawan, A. (2025). TEKNIK PENGUMPULAN DATA: OBSERVASI, WAWANCARA DAN KUESIONER. *JISOSEPOL: Jurnal Ilmu Sosial Ekonomi Dan Politik*, *3*(1), 39–47.

Safa Haqiqah, Q., & Dyah Ayu Murika Sari, N. (2025). Perawatan Pulpotomi pada Gigi Desidui Posterior Mandibula : Laporan Kasus. *Prosiding Dental Seminar Universitas Muhammadiyah Surakarta*, 21–25.

Safira, J. R. D., & Larasati, R. (2022). Hubungan Pengetahuan Makanan Kariogenik dengan Kejadian Karies Gigi pada Anak (Studi Pada Kelas IV SD Negeri Sedatigede 2 Sidoarjo). *Indonesian Journal of Health and Medical*, *2*(4), 613–620.

Saputri, V. D. (2024). Desain dan Implementasi Sistem Pakar Diagnosa Penyakit Menggunakan Forward Chaining. *Jurnal Komputer*, *2*(2), 75–80. https://doi.org/10.70963/jk.v2i2.109

Sari, I. P. (2021). *Buku Ajar Rekayasa Perangkat Lunak*. https://books.google.co.id/books?id=1LVKEAAAQBAJ

Sari Siregar, Y., Oktaviana Sembiring, B., Rahayu, E., & Franchitika, R. (2024). Pemanfaatan Aplikasi MySQL untuk Membantu Siswa SMK Swasta Nur Azizi dalam Pengolahan Data Utilization of the MySQL Application to Assist Nur Azizi Private Vocational School Students in Data Processing. *Desember*, *3*(2), 229–240. https://jurnal.unity-academy.sch.id/index.php/japamas%0Ahttp://creativecommons.org/licenses/by-sa/4.0/

Stomatitis aphthosa [Gambar]. (2023). Diakses pada 2 November 2025, dari https://www.halodoc.com/artikel/cara-ampuh-mengobati-sariawan-pada-anak

Sugito, B., & Shafira, A. (2023). Design of a Fuel Sales and Distribution Business a Pplication Using a Website. *Prosiding Seminar Nasional Manajemen Industri Dan Rantai Pasok*, *4*(1), 59–66.

Suryono, S., & Dewi, L. (2023). Sistem Pakar Untuk Menentukan Tingkat Risiko Ibu Hamil Pada Klinik Bersalin Pratama Umi Rahma Menggunakan Metode Forward Chaining. *JURNAL PENELITIAN SISTEM INFORMASI (JPSI)*, *1*(4), 201–2014.

SYAM, N. Z. (2024). *Prevalensi Anomali Gigi Pada Anak Down Syndrome di Kota Makassar= Prevalence of Dental Anomalies in Down Syndrome Children in Makassar City* [Universitas Hasanuddin]. https://repository.unhas.ac.id/id/eprint/42496/

Syuhriah Syuhriah, Laelatun Nupus, Fadiya Qotrunnada, & Firdha Senja Maelaningsih. (2024). Review Artikel : Analisis Kandungan Fluoride dalam Pasta Gigi dengan Metode Potensiometri dan Elektroda Selektif Ion. *OBAT: Jurnal Riset Ilmu Farmasi Dan Kesehatan*, *3*(1), 15–23. https://doi.org/10.61132/obat.v3i1.899

Trauma gigi [Gambar]. (n.d.). Diakses pada 2 November 2025, dari https://kids360pediatricdentistry.com/wp-content/uploads/2025/07/WhyShould-I-Choose-Trauma-Dental-Injuries-Treatment.webp

Wa Rumiyanti, Andi Tenri Sumpala, Y. P. P. (2023). Sistem Pakar Mendiagnosis Penyakit Gigi Dengan Menerapkan Metode Forward Chaining. *Prosiding Seminar Nasional Pemanfaatan Sains Dan Teknologi Informasi*, *1*(1), 459–468.

Wayahdi, M. R., & Ruziq, F. (2023). Pemodelan Sistem Penerimaan Anggota Baru dengan Unified Modeling Language (UML) (Studi Kasus: Programmer Association of Battuta). *Jurnal Minfo Polgan*, *12*(1), 1514–1521. https://doi.org/10.33395/jmp.v12i1.12870

Wibowo, A. A., Rusip, G., & Erawati, S. (2023). Korelasi karies gigi dan status gizi pada anak usia 3-5 tahun. *Jurnal Prima Medika Sains*, *5*(1), 12–15. https://doi.org/10.34012/jpms.v5i1.3428

LAMPIRAN

Lampiran 1 Lembar Verifikasi Aturan Pakar Pertama

LEMBAR VERIFIKASI ATURAN DIAGNOSIS PENYAKIT GIGI DAN MULUT ANAK

Saya yang bertanda tangan di bawah ini:

Nama : drg. Catur Aditya Ramadhani
No. STR : EL00001311269788
Kualifikasi : Dokter Gigi Klinik Victory Purwokerto

Dengan ini menyatakan bahwa dokumen ini disusun oleh mahasiswa berdasarkan hasil wawancara dengan saya dan/atau referensi literatur medis yang telah saya konfirmasi kebenarannya. Aturan-aturan tersebut mencakup gejala-gejala dan diagnosis untuk sepuluh jenis penyakit gigi dan mulut pada anak, yaitu:

1. Karies Gigi (P01)
2. Pulpitis (P02)
3. Abses Gigi (P03)
4. Gingivitis (P04)
5. Persistensi Gigi Sulung (P05)
6. Trauma Gigi (P06)
7. Fluorosis Dental (P07)
8. Hipoplasia Email (P08)
9. Stomatitis Aphthosa (P09)
10. Maloklusi (P10)

Rincian aturan inferensi untuk masing-masing penyakit sebagaimana tercantum dalam lampiran dokumen ini, telah saya periksa dan dapat dinyatakan sesuai dengan praktik medis yang berlaku serta layak digunakan sebagai dasar dalam pengembangan sistem pakar.

Dokumen ini disahkan untuk keperluan pengembangan sistem diagnosis berbasis *website* di bidang kesehatan.

Purwokerto, 21 November 2025

drg. Catur Aditya Ramadhani
Dokter Gigi Klinik Victory Purwokerto

Lampiran 2 Lembar Verifikasi Aturan Pakar Kedua

LEMBAR VERIFIKASI ATURAN DIAGNOSIS PENYAKIT GIGI DAN MULUT ANAK

Saya yang bertanda tangan di bawah ini:

Nama : drg. Nurulita Ashary
No. STR : 3422100122262815
Kualifikasi : Dokter Gigi Klinik Victory Purwokerto

Dengan ini menyatakan bahwa dokumen ini disusun oleh mahasiswa berdasarkan hasil wawancara dengan saya dan/atau referensi literatur medis yang telah saya konfirmasi kebenarannya. Aturan-aturan tersebut mencakup gejala-gejala dan diagnosis untuk sepuluh jenis penyakit gigi dan mulut pada anak, yaitu:

1. Karies Gigi (P01)
2. Pulpitis (P02)
3. Abses Gigi (P03)
4. Gingivitis (P04)
5. Persistensi Gigi Sulung (P05)
6. Trauma Gigi (P06)
7. Fluorosis Dental (P07)
8. Hipoplasia Email (P08)
9. Stomatitis Aphthosa (P09)
10. Maloklusi (P10)

Rincian aturan inferensi untuk masing-masing penyakit sebagaimana tercantum dalam lampiran dokumen ini, telah saya periksa dan dapat dinyatakan sesuai dengan praktik medis yang berlaku serta layak digunakan sebagai dasar dalam pengembangan sistem pakar.

Dokumen ini disahkan untuk keperluan pengembangan sistem diagnosis berbasis *website* di bidang kesehatan.

Purwokerto, 22 November 2025

drg. Nurulita Ashary
Dokter Gigi Klinik Victory Purwokerto

Lampiran 3 Berita Acara Pengujian Pakar Pertama

BERITA ACARA PENGUJIAN
SISTEM PAKAR UNTUK DIAGNOSIS PENYAKIT GIGI DAN MULUT
PADA ANAK DENGAN METODE *FORWARD CHAINING*
BERBASIS *WEBSITE*

Pada hari ini Jumat , 21 November 2025 bertempat di Klinik Victory Purwokerto. Yang bertanda tangan di bawah ini:

Nama : drg. Catur Aditya Ramadhani
No. STR : EL0000131126 9788
Kualifikasi : Dokter Gigi Klinik Victory Purwokerto

Menyatakan bahwa telah dilaksanakan pengujian (*testing*) terhadap Sistem Pakar untuk Diagnosis Penyakit Gigi dan Mulut pada Anak dengan Metode *Forward Chaining* Berbasis *Website* dengan status sebagai Pakar dalam sistem, dengan hasil bahwa proses pengambilan diagnosis awal yang dilakukan oleh sistem telah sesuai dengan prosedur pengambilan diagnosis awal yang digunakan dalam praktik medis.

Demikian berita acara pengujian dibuat agar digunakan sebaik-baiknya.

Purwokerto, 21 November 2025

Penguji,

drg. Catur Aditya Ramadhani
Dokter Gigi Klinik Victory Purwokerto

Lampiran 4 Berita Acara Pengujian Pakar Kedua

BERITA ACARA PENGUJIAN
SISTEM PAKAR UNTUK DIAGNOSIS PENYAKIT GIGI DAN MULUT
PADA ANAK DENGAN METODE *FORWARD CHAINING*
BERBASIS *WEBSITE*

Pada hari ini Sabtu , 22 November 2025 bertempat di Klinik Victory Purwokerto. Yang bertanda tangan di bawah ini:

Nama : drg. Nurulita Ashary
No. STR : 3422100122262815
Kualifikasi : Dokter Gigi Klinik Victory Purwokerto

Menyatakan bahwa telah dilaksanakan pengujian (*testing*) terhadap Sistem Pakar untuk Diagnosis Penyakit Gigi dan Mulut pada Anak dengan Metode *Forward Chaining* Berbasis *Website* dengan status sebagai Pakar dalam sistem, dengan hasil bahwa proses pengambilan diagnosis awal yang dilakukan oleh sistem telah sesuai dengan prosedur pengambilan diagnosis awal yang digunakan dalam praktik medis.

Demikian berita acara pengujian dibuat agar digunakan sebaik-baiknya.

Purwokerto, 22 November 2025

Penguji,

drg. Nurulita Ashary
Dokter Gigi Klinik Victory Purwokerto